Sistema de Gestão

Documentação funcional e técnica

PUZZLE STICKERS · Maputo, Moçambique
Desenvolvido por **KingBalas Ltd**
20 de outubro de 2026

puzzle-gestao.apps-mz.workers.dev

Índice

1 Visão geral e como usar

2 Regras de negócio, decisões e pedidos em aberto

3 Arquitetura e como continuar o desenvolvimento

4 Dados, nuvem e segurança

5 Documentos PDF, catálogos e cálculos

> Os capítulos 1 e 2 são para o dono da empresa: o que o sistema faz, como se usa, as regras combinadas e o que está pedido. Os capítulos 3 a 5 são técnicos, para quem desenvolve: arquitectura, dados e nuvem, documentos PDF e cálculos. Os mesmos textos estão em formato Markdown na pasta `docs/` do repositório.

CAPÍTULO 1

Visão geral e como usar

Este capítulo descreve o sistema **Puzzle Stickers · Gestão** como estava em outubro de 2026: o que faz, onde está cada coisa e como se trabalha no dia a dia. Foi escrito para o Momad e para quem continuar o desenvolvimento.

Convenções deste capítulo:

- Os botões, separadores e campos aparecem **a negrito**, com o nome que têm no ecrã.
- "MT" quer dizer meticais. "Sem IVA" é o valor base. O IVA é de 16%.
- Cada secção termina com uma linha **Código:** que indica os ficheiros principais. Os caminhos são relativos à raiz do repositório.
- "Computador" quer dizer ecrã largo (Mac, PC, iPad na horizontal). "Telemóvel" quer dizer ecrã estreito, com menos de 900 px (telemóvel, iPad na vertical).

Autor: KingBalas Ltd.

---

1. O que o sistema faz

O sistema é uma aplicação web que se instala como app (PWA). Funciona no iPhone, no iPad, no Mac e no PC, também sem internet. A primeira entrada num aparelho precisa de internet. Os dados da empresa sincronizam-se pela nuvem.

Parte do negócio	O que o sistema faz
Pedidos	Regista o que o cliente pediu antes de haver preço, para nada se perder.
Cotações	Faz cotações no formato da Puzzle Stickers: linhas de serviço, materiais a vermelho, opções (ex.: Taksim / Mactac / Avery), deslocação, urgência e desconto. Gera o PDF.
Trabalhos	Acompanha cada trabalho por fases, com datas, tarefas, ficheiros, notas e histórico.
Produção	Prepara a requisição de prints para a Académica Digital (ou outro fornecedor), com o material de cada linha. Gera um PDF pronto a enviar por WhatsApp. Acompanha o circuito: pró-forma, pagamento (ou crédito), fatura e recibo, levantamento.
Dinheiro dos clientes	Sinal, pagamentos parciais e final, quem deve, datas prometidas, avisos.
Faturas e recibos	Arquiva, com foto ou PDF, as faturas e os recibos emitidos (do livro do Momad) e os recebidos (de fornecedores e compras).
Contas da empresa	Caixa, compras, despesas fixas mensais, gastos pessoais e resultado do mês.
IVA	Relação mensal do IVA (liquidado − dedutível), em PDF e numa folha CSV que abre no Excel, para o contabilista.
Equipa	Perfis dono, gerente e trabalhador, agenda de cada um e relatórios diários.

O que o sistema **não** faz:

- **Não emite faturas nem recibos.** Só regista os que o Momad passa no seu livro de faturas. O campo do número diz "N° do seu livro" e a app nunca cria esse número.
- As cotações e as requisições são documentos comerciais. A cotação imprime a frase "Não serve de comprovativo fiscal".
- Não controla stock.

Código: `src/App.tsx`, `src/state.tsx`, `src/db.ts` (interfaces de todos os registos), `src/calc.ts` (todas as contas).

---

2. Entrar na app

2.1 Endereço e instalação

Endereço: `https://puzzle-gestao.apps-mz.workers.dev`

Se a app ainda não estiver instalada, depois de entrar aparece no topo um aviso com as instruções. Fecha-se com **X** e não volta a aparecer nesse aparelho.

Aparelho	Como instalar
iPhone / iPad (Safari)	**Partilhar** → **Adicionar ao ecrã principal**
Mac (Safari)	Menu **Ficheiro** → **Adicionar à Dock**
Outros navegadores	No menu do navegador, escolher **Instalar app** ou **Adicionar ao ecrã principal**

2.2 Entrar: uma palavra-passe

1. Abra a app. Aparecem o logótipo, a palavra "Gestão" e o campo **Palavra-passe**.
2. Escreva a palavra-passe da empresa e toque em **Entrar**. Enquanto liga, o botão diz "A ligar...".
3. Na primeira vez, o aparelho liga-se à nuvem e recebe os dados da empresa. Se a nuvem já tiver dados, essa cópia substitui o que o aparelho tinha antes. Os ficheiros guardados no aparelho não se perdem.
4. Quando o aparelho volta a entrar (depois de **Sair** ou de a sessão terminar), primeiro envia o que mudou nele e só depois recebe o resto.
5. Enquanto houver só uma pessoa ativa na equipa, a app abre logo, sem PIN.

Notas:

- **Outra empresa** mostra o campo **Código da empresa**. No uso normal não é preciso.
- Depois de 5 tentativas erradas, a entrada fica bloqueada durante 10 minutos.
- Para mudar a palavra-passe, vá a **Definições** → **Empresa** → **cartão Nuvem** → **Mudar palavra-passe da empresa**. Preencha **Palavra-passe atual** e **Nova (mínimo 8 caracteres)** e toque em **Guardar**.
  - Os outros aparelhos terminam a sessão e pedem a palavra-passe nova.
  - O que esses aparelhos ainda não tinham enviado sobe quando voltarem a entrar.

2.3 Quando houver equipa: PIN

Com duas ou mais pessoas ativas em **Definições** → **Equipa**, depois da palavra-passe aparece **Quem está a entrar?**:

1. Toque no seu nome.
2. Escreva o PIN de 4 números, no teclado do ecrã ou do computador. **Trocar** volta à lista de pessoas.
3. O dono e o gerente abrem no **Início**. O trabalhador abre na **Agenda**.

O aparelho lembra-se de quem entrou e não volta a pedir o PIN até essa pessoa tocar em **Sair**. Se a pessoa ainda tiver o PIN inicial, o ecrã lembra que o pode mudar em **Definições → Equipa**.

2.4 Sair

Onde	Como
Computador	Toque no nome, no fundo do menu lateral ("Terminar sessão").
Telemóvel (dono e gerente)	No fim do **Início**, botão **Sair**.
Telemóvel (trabalhador)	Botão **Sair** na barra de baixo.

- **Com uma só pessoa na equipa, Sair** termina a sessão neste aparelho e volta à palavra-passe. Antes de sair, a app tenta enviar o que falta. Os dados ficam no aparelho.
- **Com equipa, Sair** volta a **Quem está a entrar?**. O aparelho continua ligado à nuvem.

2.5 Nuvem e sincronização

- Cada alteração sobe para a nuvem cerca de 1,5 segundos depois. A app também sincroniza a cada 30 segundos, quando volta ao ecrã e quando a internet regressa.
- Sem internet, tudo fica guardado no aparelho e sobe mais tarde.
- No computador, o estado aparece no menu lateral, por baixo do nome. Tocar nessa linha sincroniza logo. No telemóvel, o estado está em **Definições → Empresa → Nuvem**, onde também está o botão **Sincronizar agora**. O estado pode ser:
  - "Na nuvem · hh:mm"
  - "A sincronizar…"
  - "Sem internet · guardado no aparelho"
  - "Erro ao sincronizar · toque para tentar"
  - "Não ligado à nuvem"
- Se dois aparelhos mudarem o mesmo registo, fica a alteração mais recente.
- **Os ficheiros (fotos, scans, PDFs e logótipos dos clientes) ainda não vão para a nuvem.** Ficam só no aparelho onde foram carregados (ver secção 9).

2.6 Atualizações

- Depois de entrar, a app procura uma versão nova ao abrir, ao voltar ao ecrã e a cada 2 minutos. Quando encontra uma, aparece a faixa "Há uma versão nova (…). Os seus dados ficam guardados." com o botão **Atualizar**.
- Também pode atualizar em **Definições → Empresa → Nuvem → Atualizar app**.
- A versão em uso aparece no fundo do menu lateral ("Versão dd/mm hh:mm") e no cartão Nuvem ("Versão da app").
- Atualize sempre com **Atualizar**. **Não apague o ícone da app do ecrã principal.** No iPhone e no iPad isso pode apagar os dados guardados nesse aparelho, incluindo os ficheiros que só existem lá.

Código: `src/App.tsx` (`Ligar`, `Login`, `EstadoNuvem`, `VersaoNova`, `DicaInstalar`), `src/nuvem.ts` (`entrar`, `sincronizar`, `iniciarSincronizacao`, `terminarSessao`, `mudarSenha`, `desligar`), `src/versao.ts` (`atualizarApp`, `versaoNoServidor`), `db/001_puzzle_nuvem.sql` (`puzzle_entrar`, `puzzle_mudar_senha`).

---

3. Perfis: o que cada pessoa vê

Há três perfis. Em outubro de 2026, só o Momad usava o sistema. Os perfis já estão prontos para quando a equipa entrar.

Área → separador	Dono	Gerente	Trabalhador
Início	Sim	Sim, mas sem o botão **Compra**, sem o cartão do IVA, sem os avisos de despesas fixas e sem acertar a caixa	Não (abre na Agenda)
Trabalhos → Trabalhos	Todos	Todos	Só os atribuídos a si ("Os meus trabalhos")
Trabalhos → Agenda	Toda a equipa	Toda a equipa	Só a sua ("A minha agenda")
Trabalhos → Relatórios	Vê os de todos	Vê os de todos	Só os seus
Cotações → Cotações	Sim	Sim	Não
Cotações → Catálogos	Sim	Sim	Não
Clientes	Sim	Sim	Não
Fornecedores → Fornecedores, Requisições	Sim	Sim	Não
Fornecedores → Mapa de crédito	Sim	Sim	Não
Finanças → Recebimentos	Sim	Sim	Não
Finanças → Resumo, Despesas, Faturas e recibos, IVA, Pessoal	Sim	Não	Não
Definições → Empresa, Equipa	Sim	Não	Não
Definições → Catálogo	Vê e edita	Vê (e cria trabalhos a partir dele)	Não
Novo pedido, Novo trabalho, editar ou cancelar trabalhos	Sim	Sim	Não
Valores dentro do trabalho (preços, pagamentos, lucro)	Sim	Sim	Não
Apagar trabalhos, cotações, clientes, fornecedores, pagamentos, documentos e requisições	Sim	Não	Não

O trabalhador:

- avança as fases dos trabalhos atribuídos a si;
- junta fotos, notas e tarefas;
- escreve o relatório do dia.

Não vê valores, a lista de clientes nem as finanças. Se abrir um trabalho que não é seu, vê "Este trabalho não está atribuído a si".

Código: `src/state.tsx` (`PERM`, `pode`), `src/App.tsx` (`AREAS`, `useAreas`), `src/pages/Equipa.tsx` (`PODE`).

---

4. Navegação

Área	Separadores	Para quê
Início	—	O dia de hoje, botões rápidos, avisos e caixa
Trabalhos	Trabalhos · Agenda · Relatórios	Quadro dos trabalhos, agenda e relatórios diários
Cotações	Cotações · Catálogos	Cotações e envio dos catálogos em PDF
Clientes	—	Fichas dos clientes
Fornecedores	Fornecedores · Requisições · Mapa de crédito	Fornecedores e preçários, pedidos de produção, crédito
Finanças	Resumo · Recebimentos · Despesas · Faturas e recibos · IVA · Pessoal	Dinheiro, documentos e IVA
Definições	Empresa · Catálogo · Equipa	Dados da empresa, regras, nuvem, catálogo e equipa

- **No computador:**
  - O menu à esquerda tem as áreas, e os separadores aparecem no topo de cada página.
  - Ao lado de **Trabalhos** aparece o número de trabalhos atrasados.
  - No fundo do menu ficam **Claro / Escuro** (tema), o nome (para sair), o estado da nuvem e a versão.
- **No telemóvel:**
  - A barra em baixo tem Início, Trabalhos, Cotações, Clientes, Fornecedores e Finanças.
  - **Definições**, o botão do tema e **Sair** estão no fim do **Início**.
  - Os botões do topo mostram só o ícone.
- **Botão amarelo +:** só existe no telemóvel. Fica no canto do Início, da Agenda e de Clientes e abre **Novo pedido**. O trabalhador não o tem.
- Nas páginas de detalhe, a seta **Voltar** no topo regressa à lista.

Endereços internos (para quem desenvolve):

- Listas e páginas: `#/painel`, `#/trabalhos`, `#/agenda`, `#/relatorios`, `#/cotacoes`, `#/catalogos`, `#/clientes`, `#/fornecedores`, `#/requisicoes`, `#/contas-pagar`, `#/resumo`, `#/dividas`, `#/despesas`, `#/documentos`, `#/iva`, `#/pessoal`, `#/definicoes`, `#/catalogo`, `#/equipa`.
- Detalhe de um registo: `#/trabalhos/<id>`, `#/cotacoes/<id>`, `#/clientes/<id>`, `#/fornecedores/<id>`, `#/requisicoes/<id>`.
- Parâmetros usados pela app: `?tab=pagamentos`, `?etapa=…`, `?mes=AAAA-MM`, `?entregar=1`.

Código: `src/App.tsx` (`AREAS`, `Shell`, `Page`), `src/state.tsx` (`useRota`, `ir`).

---

5. Os ecrãs, um a um

5.1 Início

Bloco	O que mostra e o que permite
Saudação	"Bom dia / Boa tarde / Boa noite" com o nome, quantas coisas há para hoje e quantas há nos próximos 7 dias.
Botões rápidos	**Novo pedido**, **Cotação** (abre uma cotação nova), **Fatura**, **Recibo** e **Compra**.
Contadores	**Pedidos**, **Em negociação** (com o valor, que "não conta no a receber"), **Em curso** (layout, produção, montagem) e **Para entregar**. Pedidos, Em negociação e Para entregar abrem Trabalhos já filtrado. Em curso abre o quadro sem filtro.
Hoje	Até 10 itens da agenda, atrasados e de hoje. **Agenda** → abre a agenda.
Próximos 7 dias	Até 8 itens desta semana.
Sem data	Até 6 pedidos e tarefas que ainda não têm data.
O que precisa de atenção	Até 8 avisos, com os mais graves primeiro. Tocar num aviso abre o sítio certo. Só o dono e o gerente os veem.
Quem me deve	Os 6 clientes com mais dívida. Tocar num cliente abre a ficha dele. **Tudo** → abre Recebimentos.
IVA de <mês>	Só o dono vê. Mostra liquidado, dedutível e a pagar (ou crédito). Tocar abre o IVA.
Caixa	**Em caixa**: só o dono toca para acertar. **Valor dos trabalhos**: confirmados ainda por receber, e quanto já entrou. **Falta receber**. **Eu devo**: pró-formas por pagar e crédito a fornecedores, abre o Mapa de crédito.

Avisos que podem aparecer em **O que precisa de atenção**:

Aviso	Quando aparece
"T-… · entregue sem o pagamento final"	Trabalho entregue, ainda com dívida e sem data prometida.
"<Cliente> prometeu pagar até <data> — N dias de atraso"	A data prometida passou.
"<Cliente> paga hoje" / "<Cliente> paga a <data>"	Faltam 2 dias ou menos para a data prometida.
"T-… · em negociação há N dias"	7 dias ou mais em negociação, sem data de resposta marcada.
"T-… · a trabalhar sem sinal"	Trabalho confirmado em Layout, Produção ou Montagem sem o sinal pago.
"T-… · pronto para entregar"	Em Entrega, com valor a cobrar.
"T-… · valor de venda por definir"	Trabalho confirmado com total 0.
"T-… · recebido sem fatura/recibo"	Há pagamentos sem documento passado.
"REQ-… · à espera da pró-forma há N dias"	Lista enviada ao fornecedor há 1 dia ou mais, sem pró-forma.

Aviso	Quando aparece
"Pró-forma <nº> por pagar"	Pró-forma registada e não paga.
"<Cliente> já pagou T-... — liquidar o crédito"	Requisição a crédito e o cliente já pagou o trabalho todo.
"<despesa> · por pagar desde <data>" ou "N despesas fixas de <mês> em atraso"	Só para o dono. Despesas fixas que passaram o prazo.
"<despesa> · pagar até <data>" ou "N despesas fixas de <mês> · pagar até <data>"	Só para o dono. O prazo de pagamento já abriu. Verifica o mês anterior e o atual.

Código: `src/pages/Painel.tsx` (`Painel`), `src/pages/Resumo.tsx` (`useCaixa` , `CaixaModal`), `src/calc.ts` (`contaParaReceber` , `contasAPagar` , `fixasDoMes` , `semDocumento`).

5.2 Trabalhos

Separador Trabalhos

- **Vistas: Quadro** ou **Lista**. O aparelho lembra-se da escolha.
- **Procurar:** pelo código, cliente, título ou descrição.
- **Filtros:**
  - por tipo: **Todos os tipos**, cada um com o número de trabalhos;
  - por pessoa: **Toda a equipa**, só para o dono e o gerente.
- **Limpar filtro:** tira os filtros que vieram do Início.
- **Novo trabalho** (dono e gerente) abre o formulário completo (ver 6.1).
- **Trabalhos cancelados:** saem do quadro, da lista e da agenda, mas continuam na ficha do cliente.

Quadro:

- Há uma coluna por fase. Com "Todos os tipos" aparecem só as fases que têm trabalhos, mais Pedido e Entregue. Ao escolher um tipo, aparece o caminho completo desse tipo.
- Pode arrastar um cartão para outra coluna. Se o tipo não tiver essa fase, a app avisa.
- Arrastar para Entregue um trabalho com dívida abre a janela de entrega.
- Os trabalhos entregues há mais de 30 dias saem do quadro, mas continuam na Lista.

Cartão do trabalho:

- Mostra o código e o tipo, o título, o cliente e o prazo ("3d atrasado", "Hoje", "Amanhã", "2 dias" ou a data).
- Para quem vê valores, mostra também "falta X", "Sem sinal", "X em negociação", "paga dd/mm" ou "Valor por definir".
- O raio indica um trabalho urgente. O círculo com iniciais é o responsável.

Lista: colunas Trabalho (com "URGENTE" quando é o caso), Cliente, Etapa, Prazo (com a data de entrada), Responsável e, para quem vê valores, Total e Pago.

Separador Agenda

- **Nova tarefa:** caixa **Nova tarefa...** com **Hoje**, **Amanhã**, data, trabalho (ou **Sem trabalho**) e **Adicionar**.
- **Vistas:**
  - **Lista**, com as secções **Atrasado**, **Hoje**, **Próximos 7 dias**, **Mais tarde** e **Sem data**;
  - **Mês**, em grelha, com as setas e **Hoje** para voltar ao mês atual.
- **Filtro:** o dono e o gerente podem filtrar por pessoa (**Toda a equipa**).

- **O que aparece:** as tarefas por fazer e as datas das fases que ainda faltam em cada trabalho ("Responder ao pedido", "<Fase> — data limite", "Montagem" de/até e "Entrega prevista"). Um pedido sem data aparece em **Sem data** como "Pedido por tratar".
- O círculo à esquerda de uma tarefa marca-a como feita. Tocar num item ligado a um trabalho abre esse trabalho.

Separador Relatórios

- **O meu relatório de hoje: O que fiz hoje**, **Em que trabalhos**, **O que falta / amanhã (opcional)**, **Problemas ou o que preciso (opcional)**, **Fotos (opcional)** e depois **Enviar relatório**. Depois de enviado, aparece **Editar**.
- O dono e o gerente veem o cartão **Hoje**, com cada trabalhador marcado como "enviou" ou "ainda não enviou".
- Em **Anteriores** ficam os relatórios dos últimos 45 dias. O dono e o gerente veem os de todos e podem filtrar por pessoa. O trabalhador vê os seus.

Página de um trabalho

No topo:

- Título, código, cliente e tipo.
- O botão ⋯ ("Mais ações") tem:
  - **Voltar para <fase anterior>**;
  - **Editar trabalho** e **Cancelar trabalho** (dono e gerente);
  - **WhatsApp do cliente** (quando o cliente tem telefone);
  - **Apagar trabalho** (só o dono).
- Um trabalho cancelado mostra uma faixa com **Reativar**.

Se houver dinheiro recebido sem documento, aparece a faixa "Recebido X sem fatura/recibo — falta passar o documento.", com o botão **Passar fatura/recibo** (ver 6.7).

Cartão das fases (linha de passos no topo):

- Cada passo mostra a data em que foi feito ou o limite ("até dd mmm"). Um limite que já passou fica a vermelho.
- Tocar num passo leva o trabalho para essa fase. Para trás, a app pede confirmação.
- Por baixo fica o painel da fase atual, com os botões próprios (ver 6.4).
- **Saltar <fase>** → passa à fase seguinte. Não aparece em Pedido nem em Entrega.
- **Datas de todas as fases** abre os prazos de todas as fases que faltam.

Separadores do trabalho:

Separador	Conteúdo
Detalhes	Cartão **Trabalho**: cliente, tipo, fase, "Prometeu pagar" (quando há), entrada, entrega prevista, responsável, serviço do catálogo, cotação, urgência e descrição. **Partes do trabalho** (só com valores): botão **Parte** (ou **Do catálogo** na criação de imagem) e **Fazer cotação** (ou o atalho para a cotação). Cada parte diz "por cotar", "combinado de boca", "da cotação" ou "catálogo". Seguem-se taxa de urgência, sub-total, IVA e TOTAL, e "Material já pedido: X sem IVA" quando há requisições. **Tarefas** deste trabalho. **Ficheiros e fotos da arte**: arte, layouts, fotos.
Requisições	Requisições a fornecedores deste trabalho: **Nova** e, em cada lista, o ícone de PDF. **Outros custos**: botão **Custo**, com **Descrição**, **Valor sem IVA (MT)** e **Data**. Serve para transporte, consumíveis, aplicação externa.

Separador	Conteúdo
Pagamentos	Pagamentos do cliente e **Registar pagamento**.
Faturas e recibos	Documentos ligados ao trabalho e **Carregar**.
Notas e histórico	Escrever uma nota e tocar em **Guardar**. Linha do tempo com notas, mudanças de fase e pagamentos.

Os separadores Requisições, Pagamentos e Faturas e recibos só aparecem a quem vê valores.

Coluna da direita:

- **Resumo financeiro** (só com valores):
  - total, sub-total + IVA e uma barra com o que está pago;
  - **Pago** / **Falta** e "Sinal pago" ou "Sinal em falta";
  - botões **Pago** (quando há dívida) e **Parcial**;
  - em baixo: Requisições (m²), Outros custos, Custo total e **Lucro** com a margem. Custos e lucro são sem IVA.
- **Tempo:** há quantos dias o trabalho está em curso (ou quanto levou), a entrega prevista e o prazo do catálogo.
- **Cliente:** nome, cidade ou telefone, **WhatsApp** e **Ficha** →.

Tipos de trabalho e as suas fases

Tipo	Fases
Branding / wrap	Pedido → Em negociação → Layout → Produção → Montagem → Entrega → Entregue
Criação de imagem	Pedido → Em negociação → Design → Entrega → Entregue
Fornecimento	Pedido → Em negociação → Layout → Encomenda → Entrega → Entregue
Montagem (só mão de obra)	Pedido → Em negociação → Montagem → Entrega → Entregue

Os trabalhos antigos, com os nomes de fase usados antes de 16 de outubro, são lidos com estes nomes novos. Os registos não são reescritos.

Código: `src/pages/Trabalhos.tsx`, `src/pages/Trabalho.tsx` (`TrabalhoDetalhe`, `PagamentoForm`, `CustoForm`), `src/pages/Fases.tsx` (`FasesTrabalho`, `EntregaModal`), `src/pages/Agenda.tsx` (`Agenda`, `NovaTarefa`, `TarefasDoTrabalho`), `src/pages/Relatorios.tsx`, `src/pages/comum.tsx` (`JobCard`, `PrazoBadge`), `src/calc.ts` (`TIPOS`, `etapasDoTipo`, `normalizarEtapa`, `itensAgenda`, `secaoAgenda`, `resumoTrabalho`).

5.3 Cotações

Separador Cotações (lista)

- **Nova cotação** fica no topo.
- Colunas:
  - Cotação: o número e os assuntos das linhas;
  - Cliente, Data;
  - Validade: diz "expirou" quando a cotação está Enviada e o prazo passou;
  - Estado;

- Total: numa cotação com opções, aparece "desde X" até o cliente escolher.

Editor da cotação

No topo do editor estão o número e o estado, e os botões **Preview**, **PDF** e **Guardar**. **Guardar** só fica ativo quando há alterações.

Do lado esquerdo, por esta ordem:

1. **Cliente (Exmo(s) Sr(s))**: escolher da lista, ou **+ Novo cliente...** / **Novo**. Depois de escolher aparecem:
   - **Cidade** e **País**, que se gravam na ficha do cliente;
   - **Logotipo do cliente (sai junto ao nome)**;
   - **Data** e **Válida por (dias)**.
2. As linhas da cotação. Cada linha fechada mostra o número, "Opção N", "Urgente", o título, a quantidade × preço (quando há mais de uma), a Linha 2, os materiais e o total. Tocar abre a linha.
3. **Adicionar linha** (ver 6.2). Numa cotação nova, a escolha da primeira linha já vem aberta.
4. **Deslocação, urgência e desconto**.
5. **Texto final**.
6. **Totais**, sempre atualizados.

Do lado direito:

- **Estado**: Rascunho, Enviada, Aceite ou Recusada;
- o cartão **Cliente aceitou?**, com o sinal de 50% (da opção escolhida, quando há opções);
- a pré-visualização "Como sai a cotação" (só no computador);
- **Apagar cotação** (só o dono).

Quando as linhas falam de Taksim ou de wrap mas não têm materiais próprios, aparece a caixa **Nota do material no rodapé (TAKSIM® STX570 + STX571, garantia até 2 anos)**.

Separador Catálogos

Os catálogos da Puzzle Stickers prontos a enviar. Os PDFs fazem parte da app e estão em todos os aparelhos.

Catálogo	Ano	Páginas
Criação de Imagem	2026	21
Sticker Kits	2025	10
Produção (marcado "antigo")	2020	10

Botões de cada catálogo:

- **Partilhar PDF** abre a partilha do aparelho. Onde não há partilha, descarrega o PDF.
- **Enviar link** abre o WhatsApp com o link público.
- **Ver** abre o PDF.
- O ícone de copiar copia o link.

Código: `src/pages/Cotacoes.tsx` (`Cotacoes`, `CotacaoEditor`, `novaCotacao`), `src/pages/cotacaoFormulario.tsx` (`EditorCotacao`, `NovoImpresso`, `CatalogoCriacao`, `PreviewCotacao`), `src/materiaisCotacao.ts`, `src/cotacaoPdf.tsx` (`paginasCotacao`, `PaginaCotacao`), `src/pages/Catalogos.tsx` (`CATALOGOS`).

5.4 Clientes

- **Lista:**
  - Procurar cliente.

    - Colunas Cliente (com a cidade), Contacto, Trabalhos (e quantos em curso) e, para quem vê valores, Faturado e Em dívida.
    - "Faturado" soma o valor com IVA de todos os trabalhos não cancelados do cliente, incluindo os que ainda estão em negociação. Não depende das faturas registadas.
    - "Em dívida" só conta trabalhos confirmados.
- **Novo cliente: Nome / empresa**, **Pessoa de contacto**, **Telefone / WhatsApp**, **Email**, **NUIT**, **Cidade** e **País**, **Logotipo do cliente** e **Notas**.
    - Cidade e País saem na cotação como "Cidade, País". O país vem preenchido com Moçambique.
- **Ficha do cliente:**
    - No topo: **WhatsApp** (quando há telefone), **Editar** e **Trabalho** (novo trabalho para este cliente).
    - Números: Trabalhos, Faturado, Em dívida e Último trabalho.
    - Listas de trabalhos, contactos, cotações, e faturas e recibos.
    - **Apagar cliente** só aparece ao dono, e só se o cliente não tiver trabalhos.
- Não é preciso criar o cliente antes. No **Novo pedido** e no **Novo trabalho** basta escrever o nome: se o cliente não existir, é criado ao guardar.

Código: `src/pages/Clientes.tsx` (`Clientes`, `ClienteDetalhe`, `ClienteForm`, `LogoCliente`), `src/format.ts` (`separarMorada`, `juntarMorada`).

5.5 Fornecedores

Separador Fornecedores

Cartões: cada fornecedor tem um cartão com o logótipo, o que fornece, "crédito aberto" (quando há), o número de produtos, o número de requisições e "Deve X" ou "Nada em dívida".

Novo fornecedor:

- **Nome**, **O que fornece**, **Pessoa de contacto**, **WhatsApp / telefone** (serve para enviar as listas pela app), **Email**, **NUIT** e **Morada**.
- Tabela **Produtos e serviços**: nome, descrição, tipo (Impressão / Laminação / Material / Serviço / Outro), unidade e preço sem IVA. **Produto / serviço** acrescenta uma linha.
- **Como costuma pagar**: Pró-forma paga antes de produzir, A crédito, Paga na entrega ou Outra.
- **Prazo de entrega habitual**, **Tenho crédito aberto com este fornecedor**, **Dados para pagamento (contas, NIB...)**, **Logotipo** e **Notas**.

Produtos nas requisições: os produtos de tipo **Impressão** e **Laminação** aparecem nas listas de impressão (por m²). Nas encomendas pode escolher qualquer produto do fornecedor.

Página do fornecedor:

- No topo: WhatsApp e **Editar**.
- Números: **Devo**, Requisições, m² impressos e Total (c/ IVA).
- Cartões **Tabela de preços**, **Requisições**, **Condições e contactos**, **Por pagar** e **Faturas e recibos** (só o dono).
- **Apagar fornecedor** só aparece ao dono, e só sem requisições.

Preçário da Académica Digital: está registado na ficha da Académica com os nomes oficiais (por m², sem IVA). A mesma lista está no código como `PRECARIO_ACADEMICA_2026`.

Material	Tipo	MT/m²
Impressão com material da Puzzle Stickers	Impressão	800
Vinil comum	Impressão	950

Material	Tipo	MT/m²
Vinil comum laminado	Impressão	1.400
Vinil de wrap	Impressão	1.150
Vinil branco premium	Impressão	1.050
Vinil transparente premium	Impressão	950
ContraVision	Impressão	1.150
Banner	Impressão	950
Laminação de wrap	Laminação	850
Laminação transparente premium	Laminação	450

Vinil de wrap + Laminação de wrap = **Bundle Wrap** (2.000 MT/m²).

Separador Requisições

- **Nova requisição** cria uma requisição sem trabalho. Para a ligar a um trabalho, escolha-o no formulário ou crie-a a partir do trabalho.
- Botões de filtro por fase, com contagem: **Todas** e cada fase que tenha requisições.
- As requisições aparecem agrupadas por mês, com o total de m² e de valor.
- Cada linha mostra o número, o fornecedor, a pró-forma, o trabalho (ou "sem trabalho"), os m², a fase, o valor e, nas listas de impressão, o botão de PDF.

Fases de uma requisição:

Fase	Quando
Rascunho	Preparada, ainda não enviada.
Lista enviada	Marcada como enviada (ou com a pró-forma registada).
Pró-forma por pagar	Pró-forma registada e por pagar.
A crédito	Posta a crédito e ainda com saldo, mesmo que o material já tenha sido levantado.
Paga	Paga na totalidade.
Faturada	Paga e com a fatura do fornecedor registada.
Levantada	Material levantado (ou recebido) e nada em dívida.

Separador Mapa de crédito

É o mapa do Excel "Mapa de controlo de crédito", dentro da app.

- **Fornecedor:** quando há mais de um fornecedor no mapa, escolhe-se aqui.
- **Números: Total dos trabalhos**, **Total pago**, **Crédito em aberto** e **Pode liquidar já** (requisições cujo cliente já pagou o trabalho).
- **Vistas: Em aberto** ou **Todos**.
- **Colunas:**
  - Data, com há quantos dias;
  - Trabalho, com "cliente já pagou" a laranja;
  - Fatura / pró-forma;

    - Estado do trabalho: Enviado, Em produção ou Concluído;
    - Valor, Pago e Saldo;
    - Pagamento: Pendente, Parcial ou Pago;
    - Material: data do levantamento.

- **Botões:**
    - **Pagar** abre o pagamento, que pode ser parcial.
    - **Excel** exporta um ficheiro .xlsx no formato da folha original.

- No fim aparecem os **Dados para pagamento** do fornecedor.

Entram no mapa as requisições com pró-forma, as que estão a crédito e as que já têm pagamentos.

Código: `src/pages/Fornecedores.tsx` (`Fornecedores` , `FornecedorDetalhe` , `FornecedorForm`), `src/pages/Requisicoes.tsx` , `src/pages/RequisicaoForm.tsx` (`RequisicaoForm` , `Circuito` , `FaseBadge`), `src/pages/ContasPagar.tsx` (`ContasPagar` , `PagarModal` , `exportarMapa`), `src/requisicaoPdf.tsx` (`PRECARIO_ACADEMICA_2026` , `nomeMaterial`), `src/calc.ts` (`faseRequisicao` , `FASES_REQ` , `mapaCredito` , `contasAPagar`).

5.6 Finanças

Resumo (só o dono)

Mostra o dinheiro que entrou e saiu, mês a mês. As setas mudam o mês.

- **Números do mês:**
    - **Entrou**: pagamentos dos clientes, mais gastos pessoais repostos;
    - **Saiu**: compras, despesas fixas, pagamentos a fornecedores e gastos pessoais;
    - **Saldo do mês**;
    - **Em caixa**: tocar para acertar.

- Quando há gastos pessoais por resolver, aparece uma faixa com o total. Tocar abre Pessoal.
- **O que saiu:** Compras, Despesas fixas, Pagamentos a fornecedores, Gastos pessoais (quando há) e Total. Os valores são com IVA, como saíram da caixa.
- **Movimentos do mês:** cada entrada e cada saída. Tocar abre o trabalho, a requisição ou as despesas.
- **Gastos por categoria** e **Últimos 6 meses**, com barras de entrada e saída. Tocar num mês abre esse mês.

Recebimentos (título "Pagamentos e dívidas")

- **Números:**
    - **Total a receber**: só trabalhos confirmados;
    - **Prontos por cobrar**: trabalhos em Entrega ou entregues;
    - **Em negociação**: não conta no total;
    - **Aguardam sinal**: quantos são e a soma dos sinais.

- Dois botões escolhem a vista:
    - **Quem deve:** cada trabalho com dívida, com a data prometida, o último pagamento, o botão **Pago** e o botão de WhatsApp com a mensagem de lembrete já escrita. Tocar na linha abre os pagamentos do trabalho.
    - **Pagamentos recebidos:** todos os pagamentos, agrupados por mês.

Despesas (só o dono)

- No topo: **Despesas fixas** (abre a lista das despesas fixas) e **Despesa** (nova). As setas mudam o mês.
- Números: **Gasto no mês**, **Despesas fixas do mês**, **IVA dedutível** e **Resultado do mês**.

- **Despesas fixas de <mês>** (com **Gerir**). Cada despesa mostra o estado:
  - "a partir de dd": ainda não é altura de pagar;
  - **Pagar até dd**;
  - **Em atraso · pagar**;
  - "Paga".
- **Todas as despesas**, com filtro por categoria.
- **Resultado de <mês>** (sem IVA): Faturado (sem IVA) − Material dos trabalhos (fornecedores) − Compras e despesas da empresa = Resultado.
- **Por categoria** (tocar filtra a lista) e **Custo fixo mensal**: o mínimo que a empresa precisa de faturar por mês, mais os materiais.
- Os gastos pessoais não aparecem aqui.

Faturas e recibos (só o dono)

- **Carregar** no topo.
- Filtros **Todos / Emitidos / Recebidos**, por mês, e procura por número, cliente ou fornecedor.
- Os documentos aparecem agrupados por mês, com o atalho **Relação do IVA** →.
- Uma faixa avisa quantos documentos ainda não têm foto ou scan.
- Cada linha mostra:
  - o tipo e o número, a data, a entidade e o trabalho;
  - se veio de uma despesa e se tem ficheiro;
  - o total e o IVA (ou "não conta p/ IVA").
- Tocar abre o documento para corrigir. Só o dono pode apagar.

IVA (só o dono)

- **Mês:** com as setas ou escolhido diretamente.
- **Exportar: Excel** (ficheiro CSV que abre no Excel) e **PDF** ("Relação do IVA").
- **Números: IVA liquidado** (faturas emitidas), **IVA dedutível** (faturas de fornecedores) e **IVA a pagar** ou **Crédito a reportar**.
- **Lembrete:** entregar ao contabilista, com uma data sugerida (o último dia do mês seguinte). O prazo confirma-se com o contabilista.
- **Aviso:** trabalhos que tiveram pagamento no mês sem fatura emitida registada.
- **Listas: Emitidos (vendas)** e **Recebidos (compras)**. Os recibos do mês ficam fora da conta.

Pessoal (só o dono)

Gastos pessoais do dono pagos com dinheiro da empresa. Saem da caixa, mas não são custo da empresa, por isso ficam fora das despesas e do resultado do mês.

- **Gasto pessoal** no topo.
- Números: **Por resolver**, **Reposto este ano** e **Autorizado este ano**.
- Cada gasto por resolver tem dois botões:
  - **Repor**: o dinheiro volta à caixa no dia em que toca no botão;
  - **Autorizar**: fica como uso próprio.
- Com mais de um gasto por resolver aparecem **Repor tudo** e **Autorizar tudo**.
- Em baixo, com setas para mudar o mês, ficam os gastos já resolvidos.

Código: `src/pages/Resumo.tsx`, `src/pages/Dividas.tsx`, `src/pages/Despesas.tsx` (`Despesas`, `DespesaForm`, `FixasModal`, `FixaForm`), `src/pages/Documentos.tsx` (`Documentos`, `DocumentoForm`, `DocLinha`),

`src/pages/Iva.tsx`, `src/pages/Pessoal.tsx`, `src/print.tsx` (`imprimirIva`, `exportarIvaExcel`), `src/calc.ts` (`movimentosCaixa`, `saldoCaixa`, `resumoCaixaMes`, `resultadoDoMes`, `ivaDoMes`).

5.7 Definições

Empresa (só o dono)

Guardar, no topo, grava os dois primeiros cartões.

- **Empresa (aparece nos documentos):** Nome comercial, NUIT, Razão social, Morada, Cidade, Telefones, Email e Instagram.
- **Regras do negócio:** Taxa de IVA (%), Sinal para começar (%), Taxa de urgência (%), Validade das cotações (dias) e Nota padrão das cotações.
- **Dados: Descarregar cópia de segurança** e **Recomeçar este aparelho** (ver secção 7).
  - Quando há registos de exemplo, aparece também **Apagar os dados de exemplo (N)**.
  - O texto deste cartão ainda fala em "protótipo", mas a sincronização com a nuvem já funciona (cartão Nuvem).
- **Nuvem:**
  - a que empresa o aparelho está ligado e o estado;
  - **Sincronizar agora**, **Mudar palavra-passe da empresa** e **Desligar este aparelho**;
  - a versão da app e **Atualizar app**.

Catálogo

O catálogo Criação de Imagem 2026 (preços sem IVA), pela ordem do PDF.

- No topo, uma faixa com a taxa de urgência e o sinal.
- Cada serviço mostra:
  - o preço (ou "Sob proposta"), o prazo em dias e o que inclui;
  - **Criar trabalho**, que abre um trabalho novo de criação de imagem com esse serviço (não aparece nos Extras).
- O dono pode acrescentar serviços (**Serviço**) e editá-los: Categoria, Nome, Preço (sem IVA), Dias mín., Dias máx., Inclui (um por linha) e Ativo. O gerente só vê.

Equipa (só o dono)

- Os três perfis e o que cada um pode fazer.
- Tabela com a pessoa, o perfil, a função e os trabalhos em curso.
- **Adicionar** (ou tocar numa pessoa) abre o formulário:
  - **Nome**, **Perfil**, **Função**, **Telefone** e **Cor**;
  - **PIN (4 números)**: não pode repetir o de outra pessoa;
  - **Pode entrar no sistema**.
- O dono não pode mudar o próprio perfil nem desativar-se.
- Assim que houver duas pessoas ativas, a entrada passa a pedir o PIN depois da palavra-passe (ver 2.3).

Código: `src/pages/Definicoes.tsx` (`Definicoes`, `CartaoNuvem`), `src/pages/Catalogo.tsx` (`Catalogo`, `ServicoForm`), `src/pages/Equipa.tsx` (`Equipa`, `UserForm`), `src/seed.ts` (`EMPRESA`).

---

6. O dia a dia, passo a passo

O caminho normal de um trabalho:

Pedido → cotação → cliente aceita → fases → requisição à Académica → pagamentos do cliente → fatura e recibo → entrega → caixa, despesas e IVA.

6.1 Registar um pedido novo

Use quando o cliente pediu algo e ainda não há preço.

1. **Início** → **Novo pedido** (ou, no telemóvel, o botão amarelo **+**).
2. **Cliente:** escreva o nome. Se não existir, aparece "Cliente novo — fica registado".
3. **O que pediu:** o pedido, por palavras. A primeira linha passa a ser o título.
4. **Tipo:** Branding / wrap, Criação de imagem, Fornecimento ou Montagem.
5. **Responder até (opcional):** a data em que quer responder ao cliente.
6. **Guardar pedido.**

O trabalho fica com o código seguinte (T-2026-...), na fase Pedido, e entra na agenda: na data de resposta ou, sem data, em "Sem data".

Para um trabalho com mais detalhe (já com preço, ou que já vai a meio), use **Trabalhos** → **Novo trabalho**:

1. **1. Tipo de trabalho:** o formulário mostra as fases desse tipo.
2. **Em que fase está?** Serve para registar um trabalho que já vai a meio. A partir de Layout (Design na criação), o trabalho conta como confirmado.
3. **2. Cliente:** escreva o nome.
4. **3. Partes do trabalho**, sem IVA:
   - Com preço, a parte fica "combinado de boca". Sem preço, fica por cotar.
   - **Do catálogo (ex.: vectorização)** junta um serviço do catálogo.
   - Na criação de imagem, todas as partes vêm do catálogo, com o preço do catálogo. Nos serviços "sob proposta", escreve o valor.
5. **Urgente (+45%)** e **IVA 16%**. Os totais aparecem por baixo.
6. **Título (opcional)**, **Descrição / detalhes (opcional)**, **Data do pedido** e **Prazo de entrega**.
   - O prazo vem a 7 dias úteis. Quando junta um serviço do catálogo, passa a seguir o prazo máximo do catálogo, em dias úteis, até o mudar à mão.
   - **Responsável** só aparece quando há mais de uma pessoa ativa.
7. **Criar trabalho.** A app abre o trabalho novo.

6.2 Fazer a cotação

Duas maneiras de começar:

- **A partir do trabalho:** na fase Pedido (ou Em negociação), toque em **Fazer cotação**. O botão também está no cartão **Partes do trabalho**.
  - A cotação nasce com as partes do trabalho. Se o trabalho não tiver partes, começa com uma linha com o título dele.
  - Um trabalho em Pedido passa a **Em negociação**.
- **Cotação solta: Início** → **Cotação** ou **Cotações** → **Nova cotação**. Se o cliente aceitar, vira trabalho (ver 6.3).

A cotação recebe um número do mês: COT-001-OUT/26, COT-002-OUT/26... O mês é o do dia em que a cotação é criada, e a contagem recomeça em cada mês.

Passo 1 — Cliente.

- Escolha o cliente ou crie-o com **+ Novo cliente...**.
- Confirme **Cidade** e **País**. Sem cidade aparece o aviso "Falta a cidade: sai na cotação por baixo do nome".

- Carregue o logótipo do cliente, se quiser que saia junto ao nome.
- Acerte **Data** e **Válida por (dias)**.

Passo 2 — Linhas. Toque em **Adicionar linha**. Aparecem quatro escolhas:

Escolha	Para quê
Wrap / Branding	Full Wrap, Half Wrap ou Branding impresso. Escreve o texto e cria uma opção por material.
Criação de imagem	Logotipo, identidade visual, editorial… Vem do catálogo, já com o preço e o que inclui.
Outro serviço ou produto	Linha livre: escreve o texto e o preço.
Título de secção	Faixa de título (ex.: SERVIÇOS MENSAIS). Não tem preço e nunca vira trabalho.

Wrap / Branding, campo a campo (cartão "Serviço impresso"):

1. **Serviço:** Full Wrap (cobertura de 100% dos painéis), Half Wrap (toda a lateral) ou Branding (até 30%, estilo Corporativo). Cada um traz a cobertura e as partes habituais.
2. **Estilo:** Artístico, Desportivo e/ou Corporativo.
3. **Viatura** e **Qt.** (ex.: 5 camiões). Para outro objeto (ex.: um expositor), desmarque **É uma viatura**.
4. **Primeira linha da descrição:** é escrita sozinha. Pode mudá-la, e **Voltar ao texto automático** repõe-na.
5. **Cobertura:** botões (de 100%, toda a lateral, até 30/40/50/55/60/75/85/100%) ou texto livre.
6. **Inclui:** Frente, Traseira, Laterais, Portas, Capom, Bumper Frontal, Bumper Traseiro, Tejadilho, Bagageira, Guarda-lamas, Retrovisores, Pilares, Spoiler, Logotipos, Logotipo no "Chapéu", Stickers Complementares, e **Outra parte…**. A frase sai como "Inclui …", com os dois bumpers juntos e os stickers no fim.
7. **Opções:** marque os materiais a propor (Taksim®, Mactac®, Avery Dennison®, Vynil Comum, Premium; os três primeiros vêm marcados) e escreva o preço sem IVA de cada um. Com Qt. maior que 1, o preço é por viatura e aparece a conta (ex.: "5 × = 200.000,00 MT").
8. **Layout:** a nota final (Layout ainda por definir, Layout já definido, …pelo cliente, …+ melhoria).
9. **Juntar à cotação (N opções):** só fica ativo com a viatura escrita e pelo menos um material marcado.
    - Cada material marcado vira uma linha "Opção 1", "Opção 2"… No PDF, a linha sai como "OPÇÃO 1 - Taksim®", com o material a vermelho e a garantia da marca.
    - Com um só material, a linha não tem opção.

O que está a preencher já aparece na pré-visualização e nos totais antes de tocar em **Juntar**.

Editar uma linha (tocar nela):

- **Opção:** Nenhuma, 1, 2, 3… Uma linha sem opção entra no total de todas as opções.
- **Linha 1**, **Linha 2 (a negrito)** e **Detalhes (em itálico, uma ideia por linha)**.
- **Materiais (saem a vermelho):** da lista **+ Material…**, ou **Outro material (escrever)…**, com Não certificado / Certificado e a garantia, e depois **Juntar**.
- **Nota final**, **Qt.** e **Valor unitário (sem IVA)**.
- **Fechar**, ou o caixote para apagar a linha.

Passo 3 — Deslocação, urgência e desconto.

- **Deslocação: Sem deslocação**, **Até 20 km · 1.500,00 MT**, **21–35 km · 2.500,00 MT**, **36–50 km · 3.500,00 MT** (distância de ida), ou **Outro valor** (escreve os km e o valor). Sai como última linha e soma em todas as opções.
- **Urgência:** toque nas linhas urgentes. Logo a seguir a cada uma aparece a linha "Taxa de urgência (45%)", que vale 45% do preço de **uma** unidade dessa linha (não multiplica pela quantidade).
- **Desconto (%):** é tirado do sub-total, antes do IVA.

- **IVA:** 16%, sempre incluído.

Passo 4 — Texto final. Botões **Padrão**, **Pedir layouts**, **Criar layouts**, **Agendar viatura** e **Projecto**, ou texto livre.

Passo 5 — Ver e enviar.

1. **Totais:** por opção (sub-total + IVA e total), ou Sub-total, Desconto, IVA e TOTAL.
2. **Preview** (ícone do olho, no topo): mostra as páginas A4 tal como vão sair, mesmo antes de guardar. Tem **Fechar** e **PDF**.
3. **PDF:** guarda a cotação e abre a janela de impressão do aparelho. Escolha **Guardar em PDF** (ou **Partilhar**).
   - O ficheiro chama-se "Puzzle_Stickers_Cotação_<cliente>_<Linha 2 da primeira linha, ou a Linha 1>".
4. Depois de enviar ao cliente, mude **Estado** para **Enviada** e toque em **Guardar**. O estado não muda sozinho.

Como sai o PDF:

- Numa cotação com opções, cada opção sai nas suas páginas: as linhas comuns mais essa opção, com SUB-TOTAL, IVA e TOTAL próprios.
- O desconto sai a vermelho (DESCONTO x%), entre o sub-total e o IVA.
- Quando as linhas não cabem numa página, a página diz "continua na página N →" e o cabeçalho mostra "página N/M".

6.3 O cliente aceitou

Cotação feita a partir do trabalho:

1. Abra a cotação. No cartão **Cliente aceitou?**, se houver opções, escolha a opção (ex.: "Opção 2 · 249.400,00 MT").
2. Toque em **Cliente aceitou**.
3. A cotação fica Aceite e o trabalho é atualizado:
   - As partes do trabalho passam a ser as linhas aceites. O desconto já vem nos preços, para o total do trabalho bater com a cotação, e a urgência entra como linha.
   - Se o trabalho estava em Pedido ou Em negociação, passa à fase seguinte.
   - O trabalho conta como confirmado e abre nos Pagamentos. A mensagem mostra o sinal de 50%.

Cotação solta:

1. Confirme o cliente (é obrigatório) e, se houver opções, escolha a opção aceite.
2. Toque em **Converter em trabalho** (ou **Converter em trabalhos**).
3. Cada linha de serviço vira um trabalho, já confirmado, na fase a seguir à negociação, com o cliente, o valor e a descrição.
   - Com opções, só passam as linhas comuns e as da opção escolhida.
   - A linha de urgência vai com a sua linha, e a deslocação vai com o primeiro trabalho.
   - As secções não viram trabalho.
   - No fim, a app abre o primeiro trabalho.
4. A app escolhe o tipo de trabalho pelo texto da linha:
   - "wrap", "branding", "vinil", "viatura" ou "camião" → Branding / wrap;
   - "camisete", "t-shirt" ou "caneca" → Fornecimento;
   - "montagem" ou "PPF" → Montagem;
   - o resto → Criação de imagem. A entrega prevista fica a 10 dias úteis. Confirme o tipo e a data em **Editar trabalho**.

Sem cotação (preço combinado de boca): no trabalho, na fase Em negociação, toque em **Cliente confirmou**. O trabalho passa a contar no "a receber".

Um trabalho conta como confirmado quando acontece uma destas coisas:

- tocou-se em **Cliente confirmou** ou **Cliente aceitou**;
- o cliente pagou alguma coisa;
- o trabalho já passou da negociação.

6.4 Avançar as fases do trabalho

Fase	Botões e datas no painel
Pedido	**Fazer cotação**, **Já está em negociação**, **Responder até**
Em negociação	**Cliente confirmou**, a cotação (ou **Fazer cotação**), **Resposta até**
Layout (Design na criação)	**Aprovado**, **Layout até**
Produção (Encomenda no fornecimento)	Lista das requisições com a fase; **Produção pronta**; **Pedido à Académica**; **Pedido feito a (fora da app)** com **Hoje**; **Produção até**
Montagem	**Montagem feita**, **De** e **Até**
Entrega	**Entregue**, **Pago** (quando há dívida), **Entrega prevista**
Entregue	Data da entrega, quanto falta receber e a data prometida; **Pago**

Outras formas de mudar de fase:

- Tocar num passo da linha das fases (para trás, a app pede confirmação).
- ⋯ → **Voltar para <fase anterior>**.
- Arrastar o cartão no quadro.
- **Saltar <fase>** →.

Passar da negociação para a frente, por qualquer destas formas, marca o trabalho como confirmado.

Entregar:

1. Na fase Entrega, toque em **Entregue**.
2. Na janela "Entregar T-...", confirme a **Data da entrega**.
3. Se o cliente ainda deve, preencha **O cliente prometeu pagar até**. A data vem com hoje + 7 dias.
4. Toque em **Confirmar entrega**.

A data prometida aparece no trabalho, no cartão do quadro, em Recebimentos e nos avisos do Início.

6.5 Requisição à Académica

Criar:

1. Escolha por onde começa:
   - no trabalho, na fase Produção, **Pedido à Académica**;
   - no separador **Requisições** do trabalho, **Nova**;
   - sem trabalho, **Fornecedores** → **Requisições** → **Nova requisição**.
2. No separador **Prints**:
   - **Trabalho** (ou "— sem trabalho (ex.: vários clientes) —"), **Fornecedor** e **Data**.
   - Tipo: **Lista de impressão (m²)** ou **Encomenda (unidades)**. Só se pode mudar antes de a lista ser enviada ou de haver pró-forma.

- **Nome do trabalho:** vem preenchido com o cliente e o título. Serve para os nomes dos ficheiros (NOME_PRINT_1, NOME_PRINT_2…).
- **Material base:** a impressão e a laminação dos prints novos. Vem Bundle Wrap quando o fornecedor tem vinil de wrap e laminação de wrap. **Aplicar a todos os prints** copia o material base para todas as linhas.

3. Os prints, linha a linha:
   - **Qt.**, nome do ficheiro, **Largura** e **Comprimento** (em metros) e os m².
   - Cada linha tem a sua impressão e a sua laminação, com o valor da linha ao lado.
   - **Print** junta uma linha. **Gerar nomes** escreve os nomes dos ficheiros.
   - Uma linha com Qt. 0 fica na lista mas não conta.
4. **Fornecedor cobra IVA (16%)**, **Notas** e os totais (Sub-Total, IVA e TOTAL).
5. **Guardar.**
   - Para guardar, a requisição precisa do fornecedor e de pelo menos um print com medidas. Os prints sem nome recebem "<nome>_PRINT_N".
   - O número é do mês em que a requisição foi criada: REQ-001-OUT/26.

Numa **Encomenda**, as linhas são **Itens**: produto (escolhido do preçário do fornecedor ou escrito), quantidade, unidade e preço.

PDF e WhatsApp:

1. Toque em **PDF**. O botão está no fundo da janela da requisição, no Passo 1 do circuito (**PDF da lista**) e como ícone nas listas.
   - A app guarda a requisição e gera, no próprio aparelho, a "REQUISIÇÃO DE PRINTS".
   - O documento tem o número, a data, o logótipo da Académica (nos outros fornecedores sai o nome), o nome do trabalho e uma tabela com Nº, Ficheiro, Material, Qt., L × C (m), m², Preço m² e Valor.
   - No fim vêm Sub-total, IVA 16% e TOTAL. Só saem os prints com quantidade e medidas.
2. Abre a janela **PDF pronto**, com o ficheiro "REQ-…_<nome do trabalho>.pdf":
   - **Partilhar (WhatsApp, email…)**, onde o aparelho deixa partilhar ficheiros (iPhone, iPad).
   - Onde não há partilha de ficheiros (ex.: Mac), **Descarregar e abrir o WhatsApp da Académica**. Descarrega o PDF e abre a conversa do fornecedor com a mensagem escrita. Só falta anexar o PDF que ficou nas Transferências. Este botão precisa do WhatsApp do fornecedor registado.
   - **Descarregar** e **Ver**.

O circuito fica no separador **Pró-forma, pagamento e faturas** (no telemóvel, **Pagamento e faturas**).

1. Lista enviada

- **PDF da lista**, **WhatsApp <fornecedor>** (mensagem pronta a pedir a pró-forma) e **Marcar como enviada**.
- **Desfazer** volta atrás.

2. Pró-forma

- Preencha **Nº da pró-forma**, **Data**, **Valor total (com IVA)** e o ficheiro (**Carregar a pró-forma (PDF)**). Num PDF digital da Académica, a app lê sozinha o número, a data e o valor.
- Toque em **Guardar pró-forma**. Isto também marca a lista como enviada, se ainda não estava.
- Depois aparece "Igual à lista" ou "Diferença de X face à lista", com **Ver original** e **Editar**.
- A pró-forma não conta para o IVA.

3. Pagamento ao fornecedor

- Abre depois da pró-forma, ou ao tocar em **Ficar a crédito**. **Ficar a crédito** só aparece nos fornecedores marcados com crédito aberto.
- Mostra o valor, os pagamentos e o **Saldo**.

- Preencha **Valor a pagar** (pode ser só uma parte), **Data**, **Método** e **Referência**, e toque em **Registar pagamento**.
- A crédito, mostra há quantos dias está e avisa quando o cliente já pagou o trabalho.

4. Fatura e recibo do fornecedor

- Preencha **Nº da fatura**, **Data** e o ficheiro (**Carregar a fatura (PDF ou foto)**; pode ser fatura + recibo no mesmo PDF).
- Se já tiver o recibo, preencha **Nº do recibo (se já tiver)** e **Data do recibo**.
- Toque em **Guardar fatura**. A fatura entra no IVA dedutível do mês da sua data.
- O recibo pode vir depois: **Nº do recibo**, **Data**, **Vem no mesmo ficheiro da fatura** e **Guardar recibo**.

5. Impressão levantada (numa encomenda, **Recebida / levantada**)

- Preencha **Data** e **Custo do delivery (opcional)** e toque em **Marcar como levantada**.
- O custo do delivery vai para Despesas, na categoria Transporte e combustível.
- **Desfazer** volta atrás.

Pagar créditos mais tarde: use **Fornecedores → Mapa de crédito → Pagar**, ou o Passo 3 da requisição. Quando o cliente paga o trabalho todo, o Início avisa "liquidar o crédito" e o Mapa mostra o valor em **Pode liquidar já**.

Pagar ao fornecedor não é uma segunda despesa:

- O material conta no resultado do mês da data da requisição, desde que ela não esteja em Rascunho.
- Na caixa, o material sai no dia em que é pago (ver 6.9).

6.6 Receber do cliente

A regra da empresa: 50% de sinal para começar, o resto na entrega.

Onde	Botão	O que faz
Trabalho → **Resumo financeiro**	**Pago**	Regista tudo o que falta. Pode mudar o valor.
Trabalho → **Resumo financeiro**	**Parcial**	Abre o separador Pagamentos com um pagamento novo.
Trabalho → separador **Pagamentos**	**Registar pagamento**	Pagamento novo. Tocar num pagamento existente edita-o, e o dono também o pode apagar.
Fases Entrega e Entregue	**Pago**	Igual ao **Pago** do resumo.
Finanças → **Recebimentos** → Quem deve	**Pago**	Igual, sem abrir o trabalho.

Campos do pagamento:

- **Valor (MT)** e **Data**;
- **Tipo**: Sinal (50%), Parcial ou Final (na entrega);
- **Método**: M-Pesa, e-Mola, Transferência, Numerário, POS ou Cheque;
- **Referência (opcional)**: ID da transação, nº do talão;
- atalhos **Sinal 50%** e **Tudo o que falta**.

O sinal sugerido é 50% do total com IVA. Enquanto não estiver pago, o trabalho mostra "Sinal em falta". Se o trabalho já estiver em Layout, Produção ou Montagem, o Início avisa "a trabalhar sem sinal".

6.7 Fatura e recibo (número do livro)

A app não numera faturas. Escreva sempre o número do **seu livro**.

Caso A — o cliente já pagou (fatura-recibo):

1. Registe o dinheiro com **Pago**. O trabalho passa a mostrar "Recebido X sem fatura/recibo".
2. Toque em **Passar fatura/recibo** (na faixa do trabalho). Se o trabalho ainda não tiver fatura, abre **Fatura-recibo**.
3. Preencha **Número** (do livro) e **Data**, confirme o **Valor sem IVA** e carregue a foto ou o PDF.
4. Toque em **Guardar fatura**. Os pagamentos sem documento, até ao total da fatura-recibo, ficam ligados a ela, e o aviso desaparece.

Caso B — fatura primeiro, pagamento depois:

1. **Início** → **Fatura.** Aparecem os trabalhos confirmados que ainda não têm fatura.
2. Escolha o **Cliente** e os **Trabalhos desta fatura**. Uma fatura pode cobrir vários trabalhos.
3. Escolha **Documento: Fatura** e preencha **Número**, **Data**, **Valor sem IVA** (vem calculado, mas pode mudar) e o ficheiro. Toque em **Guardar fatura**.
4. Quando o cliente pagar: **Início** → **Recibo** (ou **Passar fatura/recibo** no trabalho). Escolha a **Fatura** e aparece o que falta.
5. Preencha **Valor recebido**, **Número do recibo**, **Data**, **Método** e o ficheiro, e toque em **Guardar recibo**.
   - O valor pode ser só uma parte. Vários recibos para a mesma fatura = pagamento em prestações.
   - **Já carreguei em "Pago" — é só o recibo** vem marcado quando há pagamentos sem recibo. Deixe-o marcado se já tinha tocado em **Pago**: o recibo fica ligado a esses pagamentos (dos mais antigos para os mais recentes) e o dinheiro não é contado duas vezes.
   - Se a caixa estiver desmarcada, a app regista o pagamento nos trabalhos dessa fatura, pela ordem em que estão na fatura. O último trabalho fica com o resto.

Faturas, faturas-recibo e vendas a dinheiro emitidas contam no IVA liquidado do mês da sua data. Os recibos não contam para o IVA, porque o IVA está na fatura.

Documentos antigos ou de outra origem:

1. Abra **Finanças** → **Faturas e recibos** → **Carregar**, ou, no trabalho, o separador **Faturas e recibos** → **Carregar**.
2. Escolha **Emitido por mim** ou **Recebido**.
3. Escolha o tipo: Fatura, Fatura-recibo, Venda a dinheiro, Recibo, Nota de crédito ou Outro.
4. Preencha o número, a entidade, a data, o NUIT e os valores (**Sei o valor sem IVA** ou **Sei o total**).

A nota de crédito abate no IVA.

6.8 Entrega com saldo em falta

1. Entregue com **Entregue** (ver 6.4) e marque a data em que o cliente prometeu pagar.
2. A partir de dois dias antes, o Início avisa "<Cliente> paga a <data>". No próprio dia, avisa "<Cliente> paga hoje".
3. Se a data passar, o aviso fica vermelho: "prometeu pagar até <data> — N dias de atraso".
4. Em **Recebimentos**, o botão de WhatsApp abre a conversa com o lembrete escrito ("...foi entregue. Valor em falta: X...").
5. Quando o cliente pagar, toque em **Pago** e depois passe a fatura ou o recibo (ver 6.7).

6.9 Caixa, compras, despesas fixas e gastos pessoais

Acertar a caixa (só o dono):

1. **Início** → **Em caixa** (ou **Finanças** → **Resumo** → **Em caixa**).
2. Preencha **Quanto tem hoje? (banco + M-Pesa + numerário)** e toque em **Guardar**.

3. A partir daí, a app soma o que os clientes pagam e tira as compras, as despesas fixas pagas, os pagamentos a fornecedores e os gastos pessoais. Os gastos pessoais repostos voltam a entrar. Pode acertar outra vez sempre que quiser.

Compras e despesas pontuais:

1. **Início** → **Compra** abre "Nova compra", já com Fatura escolhida. **Finanças** → **Despesas** → **Despesa** abre "Nova despesa", com Sem documento.
2. Preencha **Descrição**, **Categoria**, **Valor pago (MT)**, **Data**, **Método**, **Pago a (opcional)** e **Notas**.
3. Em **Documento da compra**, escolha **Fatura**, **Venda a dinheiro**, **Recibo** ou **Sem documento**. Depois o número e a foto ou o PDF.
4. Com Fatura ou Venda a dinheiro:
   - a compra fica também em Faturas e recibos;
   - **Tem IVA (conta como IVA dedutível)** vem marcado, e o **IVA incluído** é calculado (16% dentro do valor). Pode corrigir esse valor;
   - com IVA, a compra entra no IVA dedutível do mês.

Despesas fixas (salários, renda, EDM, internet…):

1. **Finanças → Despesas → Despesas fixas → Nova despesa fixa.**
2. Preencha:
   - **Nome**, **Categoria** e **Valor mensal (MT)**;
   - **Paga-se a partir do dia** (vazio = só há um dia de pagamento) e **Até ao dia**;
   - **A partir de** (mês), **Pago a (opcional)** e **Trabalhador (opcional)** (só em Salários);
   - **Notas** e **Ativa (aparece todos os meses)**.

   Exemplo da empresa: de dia 25 a dia 5. Quando o primeiro dia é maior do que o último, o prazo é no mês seguinte: as despesas de outubro pagam-se de 25 de outubro a 5 de novembro.
3. Todos os meses, cada despesa fixa aparece no cartão do mês.
   - Para a pagar, toque em **Pagar até dd** (ou **Em atraso · pagar**), confirme os dados e toque em **Marcar como paga**.
   - Para corrigir, abra-a e toque em **Desmarcar**.

Gastos pessoais:

1. **Finanças → Pessoal → Gasto pessoal**: preencha **O que foi**, **Valor**, **Data**, **Como pagou** e **Notas**.
2. O gasto fica "por resolver" e aparece no Resumo.
3. Depois resolve-o de uma de duas maneiras:
   - **Repor**: devolveu o dinheiro à empresa, e ele volta à caixa nesse dia;
   - **Autorizar**: fica como uso próprio.

   Para desfazer, abra o gasto e toque em **Voltar a por resolver**.

Duas formas de ver o dinheiro:

Ecrã	Pergunta a que responde	Como conta
Finanças → **Resumo**	Quanto dinheiro entrou e saiu?	Com IVA. O fornecedor conta no dia em que é pago. Inclui os gastos pessoais.

Ecrã	Pergunta a que responde	Como conta
Finanças → **Despesas** → **Resultado do mês**	A empresa ganhou ou perdeu?	Sem IVA. Faturado − material dos trabalhos − compras e despesas da empresa. O material conta no mês da data da requisição (os rascunhos não contam) e inclui os outros custos dos trabalhos. As despesas fixas contam no mês a que se referem. Não entram gastos pessoais.

O lucro de cada trabalho (no Resumo financeiro do trabalho) é o valor de venda sem IVA, menos as requisições (valor da lista, sem IVA), menos os outros custos. A mão de obra não entra no custo do trabalho: os salários são despesas fixas e entram no resultado do mês.

6.10 O IVA do mês

1. Confirme que todas as faturas emitidas no mês estão registadas (ver 6.7). O ecrã do IVA avisa quais trabalhos receberam dinheiro no mês sem fatura emitida.
2. Confirme as faturas recebidas: as da Académica (Passo 4 da requisição), as das compras (Fatura ou Venda a dinheiro com IVA) e as carregadas à mão.
3. Abra **Finanças** → **IVA** e escolha o mês.
4. Veja o **IVA liquidado**, o **IVA dedutível** e o **IVA a pagar** (ou **Crédito a reportar**).
5. Gere o **PDF** (Relação do IVA) e o **Excel** para o contabilista. O PDF diz que é um resumo de apoio e que não substitui a declaração oficial.

Código (secção 6): `src/pages/Rapido.tsx` (`NovoPedido` , `EmitirFatura` , `EmitirRecibo` , `documentarPagamentos`), `src/pages/TrabalhoForm.tsx` (`TrabalhoForm`), `src/pages/Partes.tsx` (`EditorPartes` , `LinhaParte` , `soCatalogo`), `src/pages/Trabalho.tsx` (`fazerCotacao` , `PagamentoForm`), `src/pages/Cotacoes.tsx` (`aceitarNoTrabalho` , `converter` , `tipoDe`), `src/pages/cotacaoFormulario.tsx` (`NovoImpresso` , `ItemEditor` , `Materiais`), `src/materiaisCotacao.ts` (`IMPRESSO` , `DESLOCACAO` , `TEXTOS_FINAIS` , `NOTAS_ITEM` , `frasePartes`), `src/print.tsx` (`imprimirCotacao` , `imprimirLista`), `src/pdfPartilha.tsx` (`emitirRequisicao` , `renderizarPdf` , `PdfHost`), `src/requisicaoPdf.tsx` (`PaginaRequisicao`), `src/leitura.ts` (`lerFicheiro`), `src/acoes.ts` (`moverEtapa`), `src/calc.ts` (`totaisCotacao` , `comUrgencia` , `numeroDoMes` , `totaisRequisicao` , `janelaFixa`).

---

7. Cópia de segurança e cuidados com os dados

Onde estão os dados:

- Os registos (trabalhos, cotações, clientes, pagamentos, documentos, despesas, tarefas, relatórios…) estão na nuvem e em cada aparelho ligado. A nuvem é a cópia principal.
- Os ficheiros (fotos, scans, PDFs e logótipos) estão só no aparelho onde foram carregados. Os catálogos em PDF são a exceção: fazem parte da app.

Descarregar cópia de segurança (Definições → Empresa → Dados):

- Descarrega o ficheiro `puzzle-gestao-copia-AAAA-MM-DD.json` .
- Inclui: empresa, equipa, clientes, fornecedores, catálogo, trabalhos, pagamentos, requisições, outros custos, faturas e recibos (só os dados, sem o ficheiro) e cotações.
- **Não inclui:** despesas (nem as fixas nem os gastos pessoais), tarefas, relatórios diários, o registo de ações nem os ficheiros.
- A app não tem botão para repor esta cópia. Repor a partir dela é trabalho de programação.
- O ficheiro tem os dados da empresa e da equipa, incluindo os PINs. Guarde-o num lugar seguro.

Botões que mexem nos dados do aparelho:

Botão	O que faz	Cuidado
Sair	Termina a sessão. Os dados ficam no aparelho.	Nenhum.
Desligar este aparelho (cartão Nuvem)	Envia o que falta, deixa de sincronizar e volta a pedir a palavra-passe.	Ao voltar a entrar, se a nuvem tiver dados, o aparelho fica com a cópia da nuvem.
Recomeçar este aparelho (cartão Dados)	Tenta enviar o que falta, desliga da nuvem e apaga todos os dados deste aparelho.	Os registos voltam da nuvem quando entrar outra vez. Perdem-se as alterações que não chegaram a subir (por exemplo, sem internet). **Perdem-se os ficheiros que só existiam neste aparelho.**
Apagar um registo (só o dono)	O registo fica marcado como apagado e desaparece em todos os aparelhos.	A app não tem "desfazer" para isto.

Antes de **Recomeçar** ou de trocar de aparelho, confirme que o estado diz "Na nuvem" e que os ficheiros importantes existem noutro sítio.

Código: `src/pages/Definicoes.tsx` (`exportar` , `CartaoNuvem`), `src/seed.ts` (`repor`), `src/nuvem.ts` (`TABELAS` , `desligar`), `src/db.ts` (`apagar` , `guardarFicheiro`).

---

8. Regras e números em resumo

Regra	Valor atual	Onde se muda
IVA	16%; nas cotações está sempre incluído	Definições → Empresa → **Taxa de IVA (%)**
Sinal para começar	50% do total com IVA	**Sinal para começar (%)**
Taxa de urgência	45%. No trabalho, sobre o valor das partes. Na cotação, uma linha própria com 45% de uma unidade da linha urgente	**Taxa de urgência (%)**
Validade das cotações	30 dias de origem; pode mudar em cada cotação	**Validade das cotações (dias)**
Deslocação	Até 20 km: 1.500 · 21–35 km: 2.500 · 36–50 km: 3.500 · mais longe: valor escrito	Código: `src/materiaisCotacao.ts` (`DESLOCACAO`)
Desconto	% sobre o sub-total da cotação, antes do IVA	Em cada cotação
Código dos trabalhos	T-2026-001, T-2026-002… (o ano é o da criação)	Automático
Número das cotações	COT-001-OUT/26; recomeça todos os meses	Automático
Número das requisições	REQ-001-OUT/26; recomeça todos os meses	Automático

Regra	Valor atual	Onde se muda
Faturas e recibos	Número do livro do Momad, escrito à mão	Nunca automático
"A receber"	Só trabalhos confirmados	—
Fotos carregadas	As grandes ficam com no máximo 2000 px (JPEG); os logótipos dos clientes com no máximo 800 px (PNG)	Código: `src/db.ts` (`comprimirImagem`), `src/pages/Clientes.tsx` (`guardarLogo`)

---

9. Limitações atuais e pontos em aberto

Coisas que convém saber no uso de todos os dias (confirmadas no código de outubro de 2026):

- **Os ficheiros não sincronizam.** Um logótipo carregado no Mac não aparece na cotação feita no iPhone, e uma fatura fotografada no iPhone não abre no Mac. (`src/nuvem.ts` : `TABELAS` não inclui `ficheiros` .)
- **O PDF da cotação** abre a janela de impressão, onde se escolhe "Guardar em PDF". O PDF da requisição já sai como ficheiro pronto a partilhar.
- **O estado da cotação** (Enviada, Recusada) muda-se à mão, no campo **Estado**, seguido de **Guardar**. **Cliente aceitou** e **Converter** marcam Aceite sozinhos.
- **Requisição** (`src/pages/RequisicaoForm.tsx` , `Circuito` , `guardarFatura` ; `src/calc.ts` , `faseRequisicao` , `mapaCredito`):
  - O Passo 3 (pagar) só abre com pró-forma ou a crédito.
  - No Passo 4, o valor da fatura do fornecedor é o da pró-forma (ou o total da lista); não se pode escrever outro valor.
  - Registar a fatura não marca a lista como enviada (registar a pró-forma marca).
  - Uma requisição só com fatura, sem pró-forma nem pagamento, aparece como "Lista enviada" (ou "Rascunho") e não entra no Mapa de crédito.
- **O PDF da requisição soma sempre o IVA de 16%**, mesmo com **Fornecedor cobra IVA** desmarcado. (`src/requisicaoPdf.tsx` : `totaisLinhas` não lê `comIva` .)
- **O logótipo do fornecedor** carregado na ficha não sai no PDF da requisição. O PDF usa uma imagem fixa da Académica e, nos outros fornecedores, o nome. (`src/requisicaoPdf.tsx` , `logoFornecedor` .)
- **Custo do trabalho:** usa sempre o valor da lista (sem IVA), mesmo quando a fatura do fornecedor é diferente. (`src/calc.ts` , `resumoTrabalho` .)
- **Requisições sem trabalho** (ex.: "Várias") aparecem em Requisições e no Mapa de crédito, mas em nenhum trabalho.
- **Leitura automática:** só funciona com PDFs digitais da Académica (pró-forma, fatura, recibo). Scans e fotos guardam-se, mas os campos preenchem-se à mão. (`src/leitura.ts` .)
- **Pagamentos aos fornecedores** não guardam comprovativo (foto do talão).
- **"Guardar na lista de materiais"** (em **Outro material**, na cotação) ainda não guarda nada: o material escrito fica só nessa linha. (`src/pages/cotacaoFormulario.tsx` , `Materiais` .)

Pontos em aberto: o Momad levantou 5 pontos a 19 de outubro de 2026. Foram organizados num plano de 6 partes, ainda por fazer, à espera da aprovação dele:

1. Fatura da Académica sem pró-forma, pagamento com comprovativo, uma fase nova "Fatura por pagar" e atalhos a partir do trabalho.
2. Cartão "Percurso" no trabalho (cotação → requisição → faturas da produção → fatura ao cliente), com a linha do tempo completa.
3. Cotação com data de envio, um botão **Negociar** separado de **Editar** e as versões ligadas no trabalho.

4. Requisição como a cotação: faixa de estado, pré-visualização A4, marcação de "enviada" ao partilhar e **Guardar** só quando há alterações.
5. Partes do trabalho com estado próprio: marcar uma parte como "feita na empresa / sem custo" e ligá-la à requisição e à linha da cotação.
6. Pagamento do cliente por parte (ex.: "pagou só o logotipo").

Perguntas que o Momad ainda tem de responder para este plano:

- A cotação negociada leva número novo (com "substitui a COT-..." registado na app) ou mantém o número com "v2"?
- O PDF da cotação negociada diz ao cliente que substitui a anterior, ou isso fica só interno?
- Como se marca uma cotação como "enviada": a app pergunta depois de gerar o PDF, marca sozinha (com Desfazer), ou só por botão?
- Ao corrigir uma cotação já enviada, basta avisar, ou bloqueia-se e obriga-se a usar **Negociar**?
- Na requisição, chega melhorar a janela atual, ou passa a ter uma página inteira como a cotação?
- Quando a fatura do fornecedor difere da lista, o custo e o lucro do trabalho passam a usar a fatura (sem IVA)?
- Nas partes, chega um estado simples por parte, ou cada parte precisa das fases completas?
- Os trabalhos feitos por lotes (vários camiões do mesmo cliente) continuam como trabalhos separados por lote, ou passam a um trabalho com partes por lote?
- Num pagamento por parte, cada pagamento corresponde a uma só parte, ou um pagamento pode ser repartido por várias partes?
- As requisições "Várias" (uma lista para vários trabalhos) ficam só no Mapa de crédito, ou ligam-se a mais de um trabalho?
- Os ficheiros ficam só no aparelho por agora, ou a nuvem de ficheiros entra neste plano?
- Liga-se já a leitura de vários PDFs da Académica de uma vez, ou fica para depois? Só lê PDFs digitais. O código está em `src/pages/Importar.tsx`, mas não está ligado a nenhum ecrã.

CAPÍTULO 2

Regras de negócio, decisões e pedidos em aberto

Este capítulo junta as regras que o Momad definiu para a Puzzle Stickers em Outubro de 2026 e o que ainda está por fazer.

- **Parte A**: regras e decisões em vigor. Cada uma foi confirmada no código, com o ficheiro e a função onde está.
- **Parte B**: pedidos em aberto. Inclui os 5 pontos de 19 de Outubro, o plano para os resolver, as perguntas por responder e outros pendentes.

Notas de leitura:

- Os caminhos são relativos à raiz do repositório. Os valores estão em meticais (MT).
- Os dados reais da empresa vivem na nuvem e não estão neste repositório. Também não estão aqui os scripts de carregamento (`admin/dados`), os ficheiros reais (`admin/ficheiros`) nem os documentos reais da Académica (`test/fixtures`). Os testes que lêem esses PDFs também não estão aqui. A fonte Gotham (comercial) também não: copie os ficheiros `gotham-*.woff` da empresa para `public/fonts/` .
- Algumas operações sobre os dados reais foram feitas por esses scripts. Dois exemplos: a renumeração das cotações e requisições antigas, e a passagem dos materiais da Académica para os nomes oficiais. O capítulo assinala-as como tal.

Parte A — Regras e decisões em vigor

Resumo

#	Regra	Onde está no código	Decidida em
A1	Cotações e requisições numeradas por mês	`src/calc.ts` · `numeroDoMes`	16 Out
A2	IVA de 16% sempre incluído nas cotações	`src/calc.ts` · `totaisCotacao`	17 Out
A3	"Não serve de comprovativo fiscal" na cotação	`src/cotacaoPdf.tsx` · `PaginaCotacao`	14 Out (layout dele), confirmada a 16 Out
A4	A app nunca cria nem numera faturas	`src/pages/Rapido.tsx` · `EmitirFatura` , `EmitirRecibo`	13–14 Out, confirmada a 16 Out
A5	Fases do trabalho	`src/calc.ts` · `ETAPAS` , `TIPOS` ; `src/pages/Fases.tsx` · `FasesTrabalho`	16 Out
A6	"A receber" só conta trabalho confirmado	`src/calc.ts` · `contaParaReceber` , `confirmado`	16 Out
A7	Serviços impressos (Wrap / Branding)	`src/materiaisCotacao.ts` · `IMPRESSO` , `frasePartes`	17 Out
A8	Materiais com Certificado / Não Certificado e garantia	`src/materiaisCotacao.ts` · `MATERIAIS_COTACAO` ; `src/cotacaoPdf.tsx` · `linhasDoItem`	17 Out

#	Regra	Onde está no código	Decidida em
A9	Uma opção por material, cada uma nas suas páginas	`src/cotacaoPdf.tsx` · `paginasCotacao`	17 Out
A10	Quantidade por linha	`src/pages/cotacaoFormulario.tsx` · `NovoImpresso`	17 Out
A11	Urgência: 45% de uma unidade, em linha própria	`src/calc.ts` · `comUrgencia`	18 Out
A12	Deslocação por escalões	`src/materiaisCotacao.ts` · `DESLOCACAO`	17 Out
A13	Desconto antes do IVA	`src/calc.ts` · `totaisCotacao`	17 Out
A14	Textos finais e notas de layout	`src/materiaisCotacao.ts` · `TEXTOS_FINAIS`, `NOTAS_ITEM`	17–18 Out
A15	Da cotação aceite ao trabalho	`src/pages/Cotacoes.tsx` · `aceitarNoTrabalho`, `converter`	14–18 Out
A16	Requisição com material por linha e preçário da Académica 2026	`src/requisicaoPdf.tsx`; `src/pages/RequisicaoForm.tsx`	16 Out
A17	Despesas fixas pagas de dia 25 a dia 5	`src/calc.ts` · `janelaFixa`, `fixasDoMes`	16 Out
A18	Caixa	`src/calc.ts` · `movimentosCaixa`, `saldoCaixa`, `resumoCaixaMes`	16 Out
A19	Gastos pessoais: repor ou autorizar	`src/pages/Pessoal.tsx`	18 Out
A20	Catálogos prontos a enviar	`src/pages/Catalogos.tsx` · `CATALOGOS`	18 Out
A21	Uma só palavra-passe	`src/App.tsx`; `src/nuvem.ts` · `terminarSessao`	18 Out

A1. Numeração das cotações e requisições

- **Formato:** `COT-002-OUT/26` e `REQ-001-OUT/26`. O número tem o prefixo, 3 dígitos, o mês abreviado (JAN … DEZ) e o ano com 2 dígitos.
- **Contagem:** recomeça todos os meses. O número novo é o maior do mesmo mês e prefixo, mais 1 (`numeroDoMes` em `src/calc.ts`). O teste está em `src/calc.test.ts`: "numbers quotes and orders per month, restarting every month".
- **Quando é atribuído:** quando o registo é criado, com a data desse dia. Acontece em três sítios:
  - nova cotação: `novaCotacao`, em `src/pages/Cotacoes.tsx`;
  - cotação feita a partir de um trabalho: `fazerCotacao`, em `src/pages/Trabalho.tsx`;
  - nova requisição: `RequisicaoForm`, em `src/pages/RequisicaoForm.tsx`.
- **Onde aparece:**
  - No PDF da cotação, ao lado da palavra "Cotação".
  - No PDF da requisição, numa etiqueta amarela por baixo de "REQUISIÇÃO DE PRINTS". Entra também no nome do ficheiro, por exemplo `REQ-001-OUT-26_<nome do trabalho>.pdf` (`nomePdf` em `src/pdfPartilha.tsx`).
  - O nome do ficheiro da cotação não leva o código. Segue o formato dos ficheiros do Momad: `Puzzle_Stickers_Cotação_<cliente>_<assunto da primeira linha>` (`imprimirCotacao` em `src/print.tsx`).

- **Trabalhos:** têm outro código, anual, como `T-2026-001` (`proximoCodigo` em `src/state.tsx`).
- São números internos. Não são números de fatura (ver A4).
- As cotações e requisições que já existiam foram renumeradas na nuvem, por ordem de data, por um script de carregamento que não está no repositório.
- **Cuidado:** se `SEED_VERSAO` subir, a função `renumerar` em `src/seed.ts` volta a escrever estes números no formato antigo (`COT-2026-001` , `REQ-2026-001`). Antes de subir essa versão, é preciso tirar as cotações e as requisições dessa função.

A2. IVA sempre incluído nas cotações

- **Regra (17 Out):** toda a cotação inclui IVA. Não há opção de cotação sem IVA.
- **Taxa:** `Empresa.taxaIva` (16%), editável em Definições, no cartão "Regras do negócio".
- **Preços das linhas:** escrevem-se sem IVA, no campo "Valor unitário (sem IVA)" (`ItemEditor`). O formulário mostra "16% · sempre incluído" no campo IVA (`EditorCotacao` em `src/pages/cotacaoFormulario.tsx`).
- **Cálculo** (`totaisCotacao` em `src/calc.ts`): bruto → desconto → sub-total → IVA → total.
- **PDF:** mostra SUB-TOTAL, IVA 16% e a barra amarela TOTAL (`blocoTotais` em `src/cotacaoPdf.tsx`).
- **Sinal:** 50% do total com IVA (`Empresa.sinalPct`).
- **Trabalho criado a partir da cotação:**
  - os trabalhos criados com "Converter" ficam com IVA (`comIva: true` em `converter`);
  - com "Cliente aceitou", o trabalho mantém a sua caixa de IVA, que vem ligada nos trabalhos novos.

A3. "Não serve de comprovativo fiscal"

- Todas as páginas da cotação levam `**Não serve de comprovativo fiscal` por baixo do título "Cotação" (`PaginaCotacao` em `src/cotacaoPdf.tsx`).
- A lista de Cotações repete-o no subtítulo: "No formato da Puzzle Stickers · não servem de comprovativo fiscal" (`src/pages/Cotacoes.tsx`).
- O relatório de IVA impresso diz que não substitui a declaração oficial nem o programa de faturação certificado (`imprimirIva` em `src/print.tsx`).
- O PDF da requisição (`PaginaRequisicao` em `src/requisicaoPdf.tsx`) não leva esta frase (ver B5).

A4. A app nunca cria nem numera faturas

- **Regra (desde os requisitos de 13–14 Out, confirmada a 16 Out):** a app regista as faturas e os recibos que o Momad passa no seu livro. Guarda o número desse livro e o ficheiro (foto ou PDF). Nunca gera um número de fatura nem um documento fiscal.
- **Base legal:** Código do IVA, art. 27 (Lei n.º 32/2007, alterada pela Lei n.º 10/2025, em vigor desde 1 de Janeiro de 2026).
  - A numeração sequencial, as tipografias autorizadas (n.º 7) e os programas certificados (n.º 9) aplicam-se a faturas e documentos equivalentes.
  - Cotações e requisições podem ter numeração própria, desde que levem "Não serve de comprovativo fiscal".
  - Esta leitura deve ser confirmada pelo contabilista.
- **No código:**
  - **Botão "Fatura" do Início** abre a janela "Emitir fatura" (`EmitirFatura` em `src/pages/Rapido.tsx`):
    - O campo "Número" tem a ajuda "O número do seu livro de faturas — a app não cria números". Sem número, não grava.
    - Só lista trabalhos confirmados, com valor, não cancelados e que ainda não têm fatura.
    - Uma fatura pode cobrir vários trabalhos do mesmo cliente (`Documento.trabalhoIds`).
    - Tem o modo "Fatura-recibo (já pago)", que fica como documento do dinheiro já recebido.

- **Botão "Recibo" do Início** abre a janela "Emitir recibo" (`EmitirRecibo`):
  - O recibo fica sempre ligado a uma fatura (`Documento.faturaId`). Vários recibos na mesma fatura correspondem a pagamento em prestações.
  - O campo "Número do recibo" tem a ajuda "O do seu livro".
  - Com a caixa "Já carreguei em 'Pago' — é só o recibo", o recibo documenta os pagamentos já registados. Sem ela, a app cria os pagamentos nos trabalhos da fatura, a começar pela dívida mais antiga.
- **Carregar documento** (janela "Carregar fatura ou recibo", `DocumentoForm` em `src/pages/Documentos.tsx`): o número escreve-se à mão; se ficar vazio, grava "s/n". A janela diz: "O sistema não emite faturas: guarda e organiza as do seu programa oficial e as dos fornecedores."

- **Dinheiro recebido sem documento:**
  - O botão "Pago" regista o pagamento.
  - Enquanto o pagamento não tiver recibo (`Pagamento.reciboId`), o trabalho mostra "Recebido X sem fatura/recibo — falta passar o documento." e o Início avisa "recebido sem fatura/recibo" (`semDocumento` em `src/calc.ts`).
  - "Passar fatura/recibo" abre o recibo da fatura do trabalho, se houver, ou então a fatura-recibo. Os dois ligam esses pagamentos ao documento (`documentarPagamentos` em `src/pages/Rapido.tsx`).

A5. Fases do trabalho

- **Fases** (16 Out; `ETAPAS` em `src/calc.ts`): Pedido → Em negociação → Layout → Produção → Montagem → Entrega → Entregue.
- **Por tipo:** cada tipo de trabalho salta as fases de que não precisa (`TIPOS`). Todos terminam em Entregue.

Tipo	Fases
Branding / wrap	Pedido, Em negociação, Layout, Produção, Montagem, Entrega, Entregue
Criação de imagem	Pedido, Em negociação, Design (é o Layout), Entrega, Entregue
Fornecimento	Pedido, Em negociação, Layout, Encomenda (é a Produção), Entrega, Entregue
Montagem	Pedido, Em negociação, Montagem, Entrega, Entregue

- **Botões de cada fase** (`FasesTrabalho` em `src/pages/Fases.tsx`):
  - Pedido: "Fazer cotação" ou "Já está em negociação", e a data "Responder até".
  - Em negociação: "Cliente confirmou", que grava `confirmadoEm` .
  - Layout: "Aprovado", que grava `layoutAprovadoEm` .
  - Produção: "Pedido à Académica" abre uma nova requisição. "Pedido feito a (fora da app)" grava `pedidoFornecedorEm` . Para avançar há "Produção pronta".
  - Montagem: datas "De" e "Até", e o botão "Montagem feita".
  - Entrega: "Entregue" pede a data da entrega. Se o cliente ainda dever, pede também a data em que prometeu pagar (`pagamentoPrometido` , por defeito daqui a 7 dias).

- **Datas-limite:**
  - Cada fase pode ter uma data-limite (`Trabalho.prazos`), em "Datas de todas as fases".
  - `prazo` é a entrega prevista e pode ficar vazio.
  - Um trabalho está atrasado quando passou a data-limite da fase em que está ou a entrega prevista (`prazoAtual` , `resumoTrabalho`).

- **Confirmação automática:** passar além da negociação marca o trabalho como confirmado (`moverEtapa` em `src/acoes.ts`).

- **Trabalhos antigos:** os gravados antes de 16 de Outubro têm nomes antigos de fase. A app lê-os como os novos (`normalizarEtapa` , aplicado em `src/state.tsx`) e não reescreve os registos.
- **Sinal:** 50% para começar e o resto na entrega (`Empresa.sinalPct`). O Início avisa "a trabalhar sem sinal" quando um trabalho confirmado está em Layout, Produção ou Montagem sem o sinal pago (`src/pages/Painel.tsx`).

A6. "A receber" só conta trabalho confirmado

- **Regra (16 Out):** só entra no "a receber" o trabalho que cumpre tudo isto (`contaParaReceber` em `src/calc.ts`):
  - não está cancelado;
  - tem valor;
  - tem dívida;
  - está confirmado.
- **Confirmado** (`confirmado`) quer dizer uma destas três coisas:
  - o Momad carregou em "Cliente confirmou";
  - o cliente já pagou alguma coisa;
  - o trabalho já passou da negociação.
- **Valor em negociação:** aparece à parte no Início, com a nota "não conta no a receber".
- **Onde se aplica:**
  - Início (`Painel`);
  - Recebimentos (`src/pages/Dividas.tsx`);
  - Clientes;
  - a etiqueta "falta" nos cartões de trabalho (`src/pages/comum.tsx`).
- **Faturas:** só se fatura trabalho confirmado (`EmitirFatura` , ver A4).
- **Teste:** "only counts confirmed or delivered work in 'a receber'" (`src/calc.test.ts`).

A7. Serviços impressos (Wrap / Branding)

- **Onde:** no formulário da cotação, "Adicionar linha" → "Wrap / Branding" (`NovoImpresso` em `src/pages/cotacaoFormulario.tsx`). Os dados estão em `IMPRESSO` (`src/materiaisCotacao.ts`).
- **Serviço:**
  - Full Wrap;
  - Half Wrap (às vezes toda a lateral);
  - Branding (o conjunto de elementos que formam o visual).
- **Estilo:** pode escolher-se mais do que um.
  - Artístico;
  - Desportivo (público do drift);
  - Corporativo (empresas).
  - De início vêm marcados Artístico e Desportivo. Ao escolher Branding, fica só Corporativo.
- **Cobertura:** escolhe-se por botões e o texto continua editável. Opções: "Cobertura de 100% dos painéis"; "Cobertura de toda a lateral"; "Cobertura até 30, 40, 50, 55, 60, 75, 85 ou 100% dos painéis".
- **Valores iniciais por serviço:**

Serviço	Cobertura inicial	Partes iniciais
Full Wrap	Cobertura de 100% dos painéis	Laterais, Capom, Bumper Frontal, Bumper Traseiro, Tejadilho, Bagageira, Stickers Complementares
Half Wrap	Cobertura de toda a lateral	Laterais, Stickers Complementares
Branding	Cobertura até 30% dos painéis	Laterais, Logotipos

- **Partes disponíveis:** Frente, Traseira, Laterais, Portas, Capom, Bumper Frontal, Bumper Traseiro, Tejadilho, Bagageira, Guarda-lamas, Retrovisores, Pilares, Spoiler, Logotipos, Logotipo no "Chapéu" e Stickers Complementares. O campo "Outra parte..." permite escrever uma parte à mão.
- **Frase "Inclui..."** (`frasePartes`):
  - As partes seguem a ordem da lista. As partes escritas à mão vêm depois.
  - Com os dois bumpers marcados, juntam-se em "Bumper Frontal e Traseiro".
  - "Stickers Complementares" fica sempre em último.
  - Exemplo: "Inclui Laterais, Capom, Bumper Frontal e Traseiro, Tejadilho, Bagageira e Stickers Complementares".
- **Texto de cada linha gerada:**
  - Linha 1: a marca do material (ex.: "Taksim®"). Com várias opções, o PDF imprime "OPÇÃO n - <marca>".
  - Linha 2, a negrito: o serviço e a viatura (ex.: "Full Wrap Toyota Corolla").
  - Descrição:
    - primeira linha automática "<Serviço> <Estilos> em Viatura <viatura>". É editável e tem o botão "Voltar ao texto automático";
    - `** <cobertura>` ;
    - a frase "Inclui...";
    - "Impresso em vynil com aplicação do respectivo protector material 100% dedicado a wrap."
  - Depois vêm o material a vermelho (A8) e a nota de layout (A14).
- **Outros objectos:** a caixa "É uma viatura" desliga-se para objectos que não são viaturas (ex.: um expositor com as medidas).
- O botão "Juntar à cotação (N opções)" põe as linhas na cotação.

A8. Materiais: Certificado / Não Certificado e garantia

- **Lista inicial** (`MATERIAIS_COTACAO`), escrita como o Momad a imprime nas cotações:

Material	Certificado	Garantia	Título da opção
TAKSIM® STX570 + STX571 (60 Microns)	Não	até 2 anos	Taksim®
Mactac® JT10700 WG BFG XR + LF10700 UV Series (80 Microns)	Sim	até 5 à 7 anos	Mactac®
Avery Dennison® MPI 1105 SuperCast Series + DOL1460Z (82 Microns)	Sim	até 7 anos	Avery Dennison®
Taksim® Economy White Vinyl + Premium Lamination (80 Microns)	Não	acima de 12 meses	Vynil Comum

Material	Certificado	Garantia	Título da opção
TAKSIM® Premium Bubble Free + Premium Lamination (80 Microns)	Não	acima de 3 anos	Premium
Vinyl Holográfico Genérico (50 Microns)	Não	sem garantias	—
Avery Dennison® Supreme™ PPF Xtreme Gloss (165µm)	Sim	—	—
HEXIS® BODYFENCE X	Sim	—	—
HEXIS® HEXLIGHTS2 (70µm)	Sim	—	—
HEXIS® BODYFENCE WINDSHIELD (70µm)	Sim	—	—

- **No gerador Wrap / Branding:** só aparecem como opção os materiais com título (os cinco primeiros). Os três primeiros vêm marcados.
- **Noutras linhas:** escolhe-se qualquer material no campo "Materiais (saem a vermelho)".
- **No PDF** (`linhasDoItem` em `src/cotacaoPdf.tsx`):
  - O nome sai a vermelho e sublinhado, seguido de " - Certificado" ou " - Não Certificado", a cinzento.
  - Um material Não Certificado sem garantia leva também "(Sem garantias)".
  - Com garantia, segue-se a linha "(A marca garante vida útil <garantia> se usado e conservado dentro dos padrões recomendados)".
- **Vários materiais:** uma linha pode ter mais do que um (ex.: impressão + cortes).
- **Nota de garantias:** quando a cotação tem trabalho com material (`comMaterial`), a última página leva uma nota. Diz que as garantias se dão com base nos certificados e que a Puzzle Stickers não dá garantia para materiais não certificados.
- **Cotações antigas de wrap** sem materiais por linha têm a caixa "Nota do material no rodapé (TAKSIM® STX570 + STX571, garantia até 2 anos)" (`Cotacao.notaMaterial`).
- **Material que não está na lista:** usa-se "Outro material (escrever)...", com Certificado / Não certificado e garantia. Ainda não fica guardado na lista (ver B5).

A9. Opções por material, cada uma na sua página

- **Como se criam:** no gerador, cada material marcado cria uma linha com o seu preço, e cada linha fica numa opção (Opção 1, 2, 3; `ItemCotacao.opcao`). Com um só material marcado, a linha não fica numa opção. Também se pode pôr qualquer linha numa opção, nos botões "Opção" da linha.
- **As opções são alternativas e não se somam.** O total de cada opção soma as linhas sem opção (comuns, deslocação) e as linhas dessa opção (`totaisCotacao` → `opcoes`).
- **PDF (17 Out):** cada opção sai nas suas próprias páginas. Cada uma leva as linhas comuns, as linhas dessa opção e o seu próprio SUB-TOTAL / IVA / TOTAL (`paginasCotacao`).
  - O Momad não quis uma tabela de opções no lugar dos totais.
  - O cartão "Totais" do formulário lista o total de cada opção.
- **Lista de cotações:** mostra "desde <opção mais barata>" até o cliente escolher. Depois mostra o total da opção aceite (`valorCot` em `src/pages/Cotacoes.tsx`).
- **Aceitar:**
  - Escolhe-se a opção em "Qual opção o cliente escolheu?" (`Cotacao.opcaoAceite`). Sem escolha, a app avisa "Escolha a opção que o cliente aceitou".
  - Só as linhas comuns e as dessa opção passam ao trabalho.
  - O sinal mostrado é 50% da opção escolhida.

A10. Quantidade

- **Valor da linha:** cada linha tem "Qt.", e o Valor Total é Qt × Valor Unitário. No PDF a quantidade sai com dois dígitos ("01").
- **No gerador:** o campo "Qt." fica ao lado da viatura.
  - Exemplo: branding em 5 viaturas → cada opção vale 5 × o preço por viatura.
  - O formulário mostra a conta enquanto se escreve (ex.: "5 × = 200.000,00 MT").
  - A linha recolhida mostra "5 × 40.000,00 MT".
- **Teste:** "sums each option across several cars" (`src/calc.test.ts`).

A11. Taxa de urgência

- **Regra (18 Out):** a urgência marca-se por linha, não na cotação inteira (`ItemCotacao.urgente`). Faz-se nos botões do campo "Urgência (+45% do preço unitário da linha, uma vez)", no cartão "Deslocação, urgência e desconto".
- **Cálculo** (`comUrgencia` em `src/calc.ts`):
  - A app escreve "Taxa de urgência (45%)" como linha própria, logo a seguir à linha urgente.
  - Essa linha tem Qt 1 e vale 45% do preço unitário da linha urgente. Nunca se multiplica pela quantidade.
  - Pertence à mesma opção.
  - É gerada no cálculo e nunca é gravada (`gerada`).
- **Percentagem:** 45% vem de `Empresa.urgenciaPct` (Definições, cartão "Regras do negócio").
- **Conversão:** a linha de urgência vai para o trabalho da sua linha.
- **Diferença a conhecer:** num trabalho criado à mão, a caixa "Urgente (+45%)" aplica 45% sobre o total das partes do trabalho (`totaisTrabalho`). Na cotação, a urgência é sempre por linha.
- **Teste:** "writes the taxa de urgência as a line after the urgent item only (45%)".

A12. Deslocação por escalões

- **Valores** (`DESLOCACAO` em `src/materiaisCotacao.ts`), pela distância de ida até ao cliente:

Distância (ida)	Ida e volta	Valor
Até 20 km	até 40 km	1.500 MT
21–35 km	42–70 km	2.500 MT
36–50 km	72–100 km	3.500 MT
Mais longe	—	"Outro valor": distância e valor escritos à mão

- **No formulário:** botões "Sem deslocação", os três escalões e "Outro valor".
- **Na cotação:**
  - Fica uma linha "Deslocação", com a distância na linha 2 e sem descrição.
  - Fica sempre em último lugar (`ordenar` em `src/pages/cotacaoFormulario.tsx`).
  - É marcada `extra: true` .
  - Não pertence a nenhuma opção, por isso soma no total de todas.
- **Conversão:** junta-se ao primeiro trabalho. Nunca é um trabalho próprio.

A13. Desconto antes do IVA

- **Cálculo:** percentagem sobre o bruto, antes do IVA (`Cotacao.desconto` , `totaisCotacao`). Aplica-se a todas as linhas da opção, incluindo a urgência e a deslocação.
- **PDF:** SUB-TOTAL, DESCONTO n% a vermelho, IVA e TOTAL (`blocoTotais`).
- **Trabalho:** não tem desconto próprio. O desconto entra nos preços unitários (`precoNoTrabalho` em `src/pages/Cotacoes.tsx`), para o total do trabalho bater com o da cotação.
- **Teste:** "takes the discount off before IVA".

A14. Textos finais, notas e aspecto da cotação

- **Texto final** (`TEXTOS_FINAIS`):
  - Há cinco textos: Padrão, Pedir layouts, Criar layouts, Agendar viatura e Projecto.
  - Todos são editáveis.
  - Por defeito usa-se o texto padrão da empresa (`Empresa.notaCotacao` , "Nota padrão das cotações" em Definições).
  - Um texto curto sai ao lado dos totais. Um texto longo (ex.: Projecto) sai a toda a largura, acima dos totais (`termos` em `src/cotacaoPdf.tsx`).
- **Nota final da linha** (`NOTAS_ITEM`), em negrito itálico:
  - `** Layout ainda por definir` ;
  - `** Layout já definido` ;
  - `** Layout já definido pelo cliente` ;
  - `** Layout já definido pelo Cliente (Inclui melhoria, ou criação de novo Layout)` .
- **Título de secção:** faixa sem valores (ex.: "SERVIÇOS MENSAIS"). Nunca vira trabalho.
- **Validade:** 30 dias por defeito (`Empresa.validadeCotacao`). A lista mostra "expirou" quando uma cotação enviada passou do prazo.
- **Aspecto:**
  - Sempre A4, sem fotografias do trabalho. O Momad envia as imagens à parte.
  - O logótipo do cliente sai junto ao nome. Carrega-se no próprio formulário: "Logotipo do cliente (sai junto ao nome)".
  - O cliente tem "Cidade" e "País" (por defeito Moçambique), guardados juntos em `Cliente.cidade` (`separarMorada` / `juntarMorada` em `src/format.ts`).
- **Pré-visualização** (`PreviewCotacao`):
  - O botão "Preview" mostra as páginas A4 antes de guardar, incluindo o que está a ser preenchido no gerador.
  - No computador, as páginas aparecem também ao vivo ao lado do formulário ("Como sai a cotação").

A15. Da cotação ao trabalho

- **Cotação feita dentro do trabalho:**
  - "Fazer cotação" (`fazerCotacao` em `src/pages/Trabalho.tsx`) junta todas as partes do trabalho numa só cotação, incluindo as combinadas de boca (`ItemTrabalho.origem`). Um trabalho em Pedido passa a Em negociação.
  - "Cliente aceitou" (`aceitarNoTrabalho`) troca as partes do trabalho pelas linhas aceites e confirma o trabalho. Se o trabalho ainda estiver em negociação ou antes, passa à fase seguinte.
- **Cotação solta:** o botão "Converter em trabalho(s)" (`converter`) pede primeiro o cliente e depois faz o seguinte:
  - cria um trabalho por linha de serviço;

    - adivinha o tipo pelo texto (`tipoDe`);
    - põe a entrega prevista a 10 dias úteis;
    - começa o trabalho já depois da negociação, confirmado.
- **Secções:** nunca viram trabalho.

A16. Requisição de prints: material por linha e preçário da Académica 2026

- **Regra (16 Out):** cada print tem o seu próprio material, impressão + laminação (`LinhaImpressao.impressao` , `laminacao`). Um só material por lista era uma limitação do Excel.
- **Preçário da Académica 2026** (`PRECARIO_ACADEMICA_2026` em `src/requisicaoPdf.tsx`), por m², sem IVA, com os nomes oficiais:

Produto	Tipo	MT/m²
Impressão com material da Puzzle Stickers	impressão	800
Vinil comum	impressão	950
Vinil comum laminado	impressão	1.400
Vinil de wrap	impressão	1.150
Vinil branco premium	impressão	1.050
Vinil transparente premium	impressão	950
ContraVision	impressão	1.150
Banner	impressão	950
Laminação de wrap	só laminação	850
Laminação transparente premium	só laminação, sobre qualquer vinil	450

- **Bundle Wrap:** Vinil de wrap + Laminação de wrap = 2.000 MT/m². Aparece com esse nome no formulário e no PDF (`nomeMaterial`).
- **Nomes antigos:** a app lê os nomes das listas Excel ("Taksim Wrap", "Laminação (PREMIUM)") como os oficiais (`nomeOficial`).
- **Requisições antigas:** as anteriores a 16 de Outubro têm um material por lista. Esse material passa para cada print (`linhasComMaterial`).
- **De onde vêm os preços:**
    - `PRECARIO_ACADEMICA_2026` é a referência no código. A app não o usa directamente.
    - O formulário lê os preços da tabela do fornecedor (`Fornecedor.materiais`), editável em Fornecedores.
    - Na nuvem, os materiais da Académica e as requisições existentes passaram para os nomes oficiais, sem mudar os preços, por um script que não está no repositório.
- **Formulário** (`src/pages/RequisicaoForm.tsx`):
    - O material base é o Bundle Wrap quando o fornecedor o tem (`baseDe`). Os prints novos ficam com ele.
    - O botão "Aplicar a todos os prints" copia o material base para todas as linhas.
    - Cada linha tem as suas escolhas de impressão e laminação e mostra o valor ao vivo.
- **Contas** (`totaisRequisicao` em `src/calc.ts` , `totaisLinhas` em `src/requisicaoPdf.tsx`):
    - Para cada linha: Qt × largura × comprimento = m², e m² × preço/m² = valor sem IVA.
    - No fim soma-se o IVA de 16%, com a caixa "Fornecedor cobra IVA (16%)" marcada. A caixa vem marcada por defeito.

- Teste: 7,92 m² em Bundle Wrap = 15.840 MT (`src/requisicaoPdf.render.test.tsx`).

- **Documento** (`PaginaRequisicao`), layout aprovado pelo Momad:
  - título "REQUISIÇÃO DE PRINTS";
  - na caixa do fornecedor, só o logótipo (`public/img/academica.png` para a Académica; nos outros fornecedores, o nome);
  - nome do ficheiro em destaque, a negrito com barra amarela;
  - material em duas linhas;
  - sem total de m² e sem resumo por material, porque os materiais são diferentes;
  - rodapé "Preços por m², sem IVA, do preçário 2026 da ...".

- **PDF:** sai como ficheiro real, não como janela de impressão (`emitirRequisicao` e `PdfHost` em `src/pdfPartilha.tsx`). Abre a janela "PDF pronto", com estas saídas:
  - "Partilhar (WhatsApp, email...)", pela folha de partilha, onde o aparelho o permite;
  - "Descarregar e abrir o WhatsApp" do fornecedor, onde não há partilha e o fornecedor tem telefone;
  - "Descarregar" e "Ver", sempre.

- **Circuito do fornecedor** (`Circuito` em `src/pages/RequisicaoForm.tsx` ; fases em `faseRequisicao` e `FASES_REQ`):
  - Passos: lista enviada → pró-forma → pagamento (ou crédito) → fatura + recibo → levantada.
  - Fases mostradas: Rascunho, Lista enviada, Pró-forma por pagar, A crédito, Paga, Faturada, Levantada.
  - A fatura do fornecedor fica como documento recebido e conta no IVA dedutível.
  - A pró-forma não conta para o IVA.
  - O custo do levantamento (delivery) fica como despesa em "Transporte e combustível".

A17. Despesas fixas: de dia 25 a dia 5

- **Regra (16 Out):** as despesas fixas pagam-se do dia 25 de um mês ao dia 5 do mês seguinte.
- **Dados:**
  - `DespesaFixa.abre` é o primeiro dia de pagamento e `dia` é o último.
  - Quando `abre` é maior que `dia` , o prazo cai no mês seguinte (`janelaFixa` em `src/calc.ts`).

- **Estados** (`fixasDoMes`): pendente, a pagar (janela aberta), atrasada e paga.
- **Nova despesa fixa:** começa com 25 a 5 (`FixaForm` em `src/pages/Despesas.tsx`).
- **Início:** verifica o mês anterior e o actual, porque a janela de 25 a 5 atravessa o início do mês.
- **Resultado do mês** (`resultadoDoMes`):
  - Conta a despesa fixa no mês a que se refere (`Despesa.mes`), não na data em que foi paga.
  - Conta as requisições na data delas, sem os rascunhos.

- **Teste:** "pays fixed expenses from the 25th to the 5th of the next month".

A18. Caixa

- **Regra (16 Out):** a app mostra o dinheiro da empresa como uma caixa: quanto há, o que entrou e o que saiu.
- **Movimentos** (`movimentosCaixa` em `src/calc.ts`):
  - Entradas: pagamentos de clientes e reposições de gastos pessoais.
  - Saídas: compras, despesas fixas pagas, pagamentos a fornecedores (`Requisicao.pagamentos`) e gastos pessoais.

- **Saldo** (`saldoCaixa`):
  - Parte do valor que o Momad escreve uma vez em "Acertar caixa", a pergunta "Quanto tem hoje? (banco + M-Pesa + numerário)" (`Empresa.caixa` ; `CaixaModal` em `src/pages/Resumo.tsx`).

    - A partir daí, soma as entradas e tira as saídas.

- **Início:** faixa "Caixa" com Em caixa, Valor dos trabalhos, Falta receber e Eu devo.
- **Finanças → Resumo** (só o dono):
    - Entrou, Saiu, Saldo do mês e Em caixa;
    - O que saiu;
    - movimentos;
    - gastos por categoria;
    - últimos 6 meses.

- **Duas leituras do custo do fornecedor:**
    - No caixa, conta quando é pago.
    - No resultado do mês (página Despesas), conta na data da requisição.
    - Pagar a um fornecedor nunca conta como segunda despesa.

- **Compra** (botão "Compra" do Início, `DespesaForm` em `src/pages/Despesas.tsx`):
    - Documento da compra: Fatura, Venda a dinheiro, Recibo ou Sem documento.
    - Fatura e venda a dinheiro ficam também como documento recebido em Faturas e recibos. Com IVA, contam no IVA dedutível.

A19. Gastos pessoais: repor ou autorizar

- **Regra (18 Out):** são gastos pessoais do dono pagos com dinheiro da empresa. Saem do caixa, mas não são custo do negócio.
- **Onde:** Finanças → Pessoal (`src/pages/Pessoal.tsx`), só para o dono.
- **Dados:** `Despesa.pessoal = true`, categoria "Pessoal", sem IVA.
- **Fora das contas:** ficam fora da lista de Despesas e do resultado do mês (`resultadoDoMes`).
- **Como se resolvem:** cada gasto fica "por resolver" até uma destas acções:
    - "Repor": grava `pessoalEstado = 'reposto'` e `pessoalEm`. O dinheiro volta a entrar no caixa nesse dia, como "Reposição pessoal".
    - "Autorizar": grava `pessoalEstado = 'autorizado'`. Fica como uso do dono e não volta ao caixa.
    - Com mais de um gasto por resolver há também "Repor tudo" e "Autorizar tudo". No gasto há "Voltar a por resolver".

- **Resumo:** mostra o cartão "Gastos pessoais por resolver". O cartão Entrou indica "clientes + X repostos por si".

A20. Catálogos

- **Separador Cotações → Catálogos** (`src/pages/Catalogos.tsx`, lista `CATALOGOS`). Os perfis de trabalhador não o vêem.

Catálogo	Ano	Páginas	Nota
Criação de Imagem	2026	21	—
Sticker Kits	2025	10	—
Produção	2020	10	marcado "antigo"

- **Ficheiros:** os PDFs e as capas estão em `public/catalogos` e vêm com a app. O link funciona para qualquer cliente e o ficheiro é igual em todos os aparelhos.
- **Acções:**
    - "Partilhar PDF", pela folha de partilha, ou descarga onde não houver partilha;

- "Enviar link", pelo WhatsApp;
- "Ver";
- copiar o link.

- **Links:** abrem o PDF e não a app (`navigateFallbackDenylist` em `vite.config.ts`).
- **Catálogo novo:** põe-se o PDF e a capa JPG em `public/catalogos` e acrescenta-se uma entrada em `CATALOGOS` .
- **Catálogo de serviços:** é outro, em Definições → Catálogo. Tem os serviços e preços do "Catálogo Criação de Imagem 2026" (`CATALOGO` em `src/seed.ts` , ids fixos `srv-…` , pela ordem do PDF).
  - Nos trabalhos de criação, todas as partes vêm deste catálogo (`soCatalogo` em `src/pages/Partes.tsx`).
  - Nas cotações, a escolha "Criação de imagem" acrescenta o serviço com o seu preço (`CatalogoCriacao`).

A21. Uma só palavra-passe

- **Regra (18 Out):** a entrada pede só a palavra-passe da empresa.
- **Ecrã de entrada** (`Ligar` em `src/App.tsx`):
  - Pede só a "Palavra-passe".
  - O código da empresa fica escondido em "Outra empresa".
- **Um só utilizador activo** (`unico`): a palavra-passe basta e não há PIN.
- **Sair** (`terminarSessao` em `src/nuvem.ts`):
  - Sincroniza e termina a sessão.
  - Os dados ficam no aparelho.
  - Ao voltar, a app só pede a palavra-passe.
- **Equipa:** o ecrã de PIN só aparece quando houver mais do que um utilizador activo. Os perfis dono / gerente / trabalhador já existem (`PERM` em `src/state.tsx`).
- **Mudar a palavra-passe:** no cartão "Nuvem" da página Definições, botão "Mudar palavra-passe da empresa" (`mudarSenha`). A nova tem de ter pelo menos 8 caracteres.
- **Servidor:** guarda a palavra-passe cifrada (bcrypt). Depois de 5 tentativas erradas, bloqueia a entrada durante 10 minutos (`db/001_puzzle_nuvem.sql`).

Parte B — Pedidos em aberto

Nada desta parte está construído, salvo onde se diz o contrário.

B1. Os 5 pontos de 19 de Outubro

A 19 de Outubro o Momad pediu que o trabalho funcione como um dossier, com tudo o que lhe diz respeito num só sítio. Pediu estes cinco pontos:

Ponto	O que o Momad pediu	Partes do plano (B2)
1	**Requisição como a cotação:** ver o A4 antes de enviar, guardar só quando algo muda, e o sistema saber se a lista já foi enviada	Parte 4
2	**Faturas da produção e o trabalho com tudo:** onde registar a fatura que pagou à Académica, pagar com comprovativo, e ver no trabalho as faturas do cliente separadas das da produção	Partes 1 e 2
3	**Trabalho como conjunto:** cotação, requisição, pró-forma, pagamentos, faturas e recibos visíveis dentro do trabalho	Parte 2

Ponto	O que o Momad pediu	Partes do plano (B2)
4	**Estado e pagamento por parte:** cada parte com o seu estado, partes feitas na empresa sem custo (ex.: o logotipo), e o caso "o cliente pagou só o logotipo"	Partes 5 e 6 (e a cadeia de versões da Parte 3)
5	**Negociar a cotação com versões:** "Editar" corrige erros e "Negociar" cria uma versão nova ligada à anterior (ex.: 10 camiões passam a 5). A primeira fica registada como enviada	Parte 3

Estado: o plano está feito e nada está construído. Espera o OK do Momad e as respostas às perguntas de B4.

B2. O plano em 6 partes

Princípio:

- O trabalho já é o dossier. Cotação, requisição, pró-forma, pagamentos (dos dois lados), faturas e recibos já apontam para ele por id. Não se cria base nova: a app junta essa informação e mostra-a dentro do trabalho.
- Cada "já enviei?" passa a ser um acto com data, gravado no próprio registo (requisição e cotação).
- "Negociar" cria uma versão nova, ligada à anterior, em vez de reescrever.
- O circuito do fornecedor passa a aceitar a ordem real: fatura antes da pró-forma, pagamento depois.
- Tudo é aditivo e opcional. A app lê os registos reais da nuvem sem migração, sem tabela nova e sem subir `ESQUEMA` (`src/nuvem.ts`).

Pré-requisito: antes da Parte 1, um commit de base do estado actual e `npm test` a passar. Os testes cobrem `calc`, `leitura` e os dois renders de PDF.

Ordem	Parte	Pontos	Esforço
1	Fatura da produção sem pró-forma: pagamento com comprovativo, fase "Fatura por pagar", entradas a partir do trabalho	2	médio
2	Dossier do trabalho: cartão "Percurso" e histórico completo	2, 3	médio
3	Cotação: "Enviada" com data, botão "Negociar" separado de "Editar", cadeia de versões	5, 4	médio
4	Requisição como a cotação: faixa de estado, preview A4, "enviada" ao partilhar, Guardar só se mudou	1	médio
5	Partes com identidade: estado por parte, "feito na empresa / sem custo", ligação à requisição e à linha da cotação	4	grande
6	Pagamento do cliente por parte ("pagou só o logotipo")	4	pequeno

Parte 1 — Fatura da produção sem pró-forma

- **Hoje** (verificado em `src/pages/RequisicaoForm.tsx` e `src/calc.ts`):
  - O Passo 3 "Pagamento ao fornecedor" recusa pagar sem pró-forma ou crédito ("Primeiro registe a pró-forma").
  - O valor da fatura é sempre o da pró-forma ou o total da lista (`guardarFatura`).
  - Registar a fatura não marca a lista como enviada. Registar a pró-forma marca.
  - `faseRequisicao` não tem "fatura por pagar" e mostra "Lista enviada".
  - `contasAPagar` e `mapaCredito` deixam essa requisição de fora, por isso o "Pagar" do Mapa de crédito não aparece.
  - O Início avisa "à espera da pró-forma".

- `PagamentoFornecedor` não guarda comprovativo.
- O Passo 4 (fatura e recibo) já funciona sem pró-forma. O aspecto "em espera" é só visual.

- **Proposta:**
  - **Circuito da requisição:**
    - O Passo 4 ganha um campo editável "Valor da fatura (com IVA)". Guardar a fatura marca também a lista como enviada.
    - O Passo 3 abre com pró-forma, fatura ou crédito. Cada pagamento aceita um comprovativo (foto ou PDF). O "Pagar" do Mapa de crédito fica igual.
    - O Passo 5 deixa de depender da pró-forma.
    - Nova fase "Fatura por pagar": na etiqueta, no Mapa de crédito e num alerta do Início. Quando já há fatura, o alerta "à espera da pró-forma" deixa de aparecer.
  - **No trabalho:**
    - Cada requisição mostra uma sub-linha (enviada · PF · FT · RC · levantada) e os botões "Fatura da Académica" e "Pagar".
    - "Carregar" em Faturas e recibos pergunta primeiro qual é o documento: fatura passada ao cliente, ou fatura/recibo da Académica desta produção.
    - O Resumo financeiro mostra "lista X · fatura Y".

- **Dados novos:**
  - `Requisicao.fatura { numero, data, valor }`;
  - `PagamentoFornecedor.ficheiroId`;
  - fase `fatura` em `FASES_REQ`;
  - `valorRequisicao` = valor da fatura; sem fatura, o da pró-forma; sem pró-forma, o total da lista.

- **Riscos:**
  - O lucro dos trabalhos reais muda quando a fatura difere da lista (pergunta 6).
  - Os comprovativos ficam só no aparelho onde foram carregados (pergunta 11).

Parte 2 — Dossier do trabalho: cartão "Percurso"

- **Hoje:**
  - O trabalho só conhece uma cotação (`Trabalho.cotacaoId`).
  - A informação está espalhada por 5 separadores.
  - "Faturas e recibos" mistura emitidos e recebidos.
  - O histórico só tem notas, fases e pagamentos do cliente.
  - Os pagamentos à Académica não aparecem no trabalho.
  - As requisições só se vêem na fase Produção.

- **Proposta:**
  - **Ficheiro novo** `src/dossier.ts` com `dossierTrabalho(t, d)`: só junta o que já existe, não grava nada, e tem teste próprio.
  - **Cartão "Percurso do trabalho"** no topo de Detalhes, só para quem tem a permissão `financas`, com três blocos:
    - Cotação: a cadeia de versões.
    - Produção: cada requisição, com as suas acções.
    - Dinheiro: duas colunas, Cliente e Académica.
  - **Faturas e recibos:** divide-se em "Que passei ao cliente" e "Da produção (Académica)", agrupadas por requisição.
  - **Pagamentos:** ganha a secção "Pagos à Académica".

- **Histórico:** passa a incluir todos os passos: cotação criada, enviada, negociada e aceite, lista enviada, pró-forma, pagamentos, faturas e levantamento. Cada passo abre o registo.
- Não se acrescenta um 6.º separador.

- **Dados novos:** nenhum.
- **Riscos:**
  - A página fica comprida no telemóvel. Por isso as secções ficam recolhidas, com uma linha de resumo.
  - As requisições "Várias" sem trabalho continuam fora (pergunta 10).

Parte 3 — Cotação: "Enviada", "Negociar" e versões

- **Hoje:**
  - O estado escolhe-se à mão numa lista (Rascunho, Enviada, Aceite, Recusada).
  - O PDF e o Preview não mudam o estado.
  - Não há data de envio.
  - "Guardar" sobrescreve a cotação mesmo depois de enviada ou aceite.
  - Um trabalho tem uma só cotação.

- **Proposta:**
  - **Cartão "Estado"** com "Marcar como enviada", que grava a data.
    - Depois de gerar o PDF, a app pergunta "Já enviou ao cliente?" (pergunta 3).
    - O Preview nunca marca.
  - **"Negociar"**, visível em cotações enviadas ou recusadas:
    - Copia a cotação para uma nova, com número novo e em rascunho, ligada à anterior.
    - A anterior passa a "Substituída": só leitura, com Ver e PDF.
    - Os trabalhos passam a apontar para a nova. Se forem vários, a app pergunta quais.
  - **Editar uma cotação já enviada** pede confirmação (avisar ou bloquear: pergunta 4).
  - **PDF:** pode dizer "Substitui a cotação ..." (pergunta 2).
  - **Cadeia no trabalho:** "v1 COT-003 · enviada · 10 camiões → v2 COT-005 · 5 camiões · aceite".
  - **Alerta por cotação** enviada há dias sem resposta.

- **Dados novos:**
  - `Cotacao.enviadaEm`, `aceiteEm`, `recusadaEm`, `substituiId`, `substituidaPorId`;
  - estado `substituida`.

- **Riscos:**
  - Ao aceitar, as versões anteriores têm de passar a "substituida". Senão continuam pendentes na lista, na ficha do cliente e no Início.
  - Os totais e as listas que contam cotações enviadas têm de deixar de fora as substituídas.

Parte 4 — Requisição como a cotação

- **Hoje:**
  - A requisição é uma janela com 2 separadores.
  - "Guardar" está sempre activo.
  - "Marcar como enviada" está escondido no Passo 1 do 2.º separador.
  - A janela "PDF pronto" não diz se o ficheiro foi partilhado.
  - As linhas identificam-se pela posição e saltam quando se remove uma.

- **Proposta:**
  - **Faixa de estado fixa** nos dois separadores (ex.: "REQ-002 · 4 prints · 12,8 m² · 34.800 MT · Enviada a 10 Out"). Se ainda não foi enviada, diz "Ainda não enviada" e mostra o botão "Marcar como enviada".

- **Botão "Preview"** com o A4 tal como vai sair, reutilizando `PaginaRequisicao`. No computador, o A4 fica ao vivo ao lado do formulário.
- **Ao partilhar ou descarregar o PDF:** a caixa "Enviei à Académica" vem marcada e pode desmarcar-se. Ao fechar, grava "Enviada hoje", com Desfazer.
- **"Guardar"** só fica activo quando há alterações.
- **Lista já enviada:** alterá-la mostra um aviso e pede confirmação.
- **Linhas** com id estável.
- **Lista de requisições** mostra "Enviada 10 Out".
- **Opção 4b:** página inteira `/requisicoes/:id`, como a cotação (pergunta 5).

- **Dados novos:**
  - `LinhaImpressao.id`;
  - `Requisicao.enviadaVia` e `enviadaPor`, opcionais;
  - `PdfPronto.onPartilhado`.

- **Riscos:**
  - Não mexer em `PaginaRequisicao` nem em `renderizarPdf`. O preview só os embrulha.
  - O teste `src/requisicaoPdf.render.test.tsx` tem de continuar a passar.

Parte 5 — Partes com identidade

- **Hoje:**
  - `ItemTrabalho` não tem id: cada parte identifica-se pela posição.
  - Aceitar a cotação substitui todas as partes.
  - Requisições e custos só se ligam ao trabalho.
  - Há uma fase por trabalho.
  - Os lotes de um mesmo cliente estão hoje como trabalhos separados que partilham a cotação.

- **Proposta:**
  - **Tabela de partes** com duas colunas novas:
    - "Estado": Por iniciar, Em curso, Feito ou Entregue.
    - "Produção": "Feito na empresa · sem custo", ou a requisição e a sua fase.
  - **Caixa "Feito na empresa (sem custo de produção)"** na parte (o caso do logotipo).
  - **Ao aceitar a cotação,** as partes juntam-se por id e mantêm o estado e os pagamentos.
  - **Nova requisição a partir do trabalho:** pergunta "Para que partes?".
  - As fases continuam por trabalho (pergunta 7). Os lotes continuam a ser trabalhos (pergunta 8).

- **Dados novos:**
  - `ItemTrabalho.id`, `estado`, `semCusto`, `cotacaoItemId`;
  - `ItemCotacao.id`, `semCusto`;
  - `Requisicao.itemIds`;
  - `Custo.itemId`.

- **Riscos:**
  - É a única parte que mexe no centro do modelo (os itens).
  - Os ids das partes existentes têm de ser gravados na primeira abertura do trabalho.
  - Ao aceitar, as partes juntam-se por id, nunca pelo texto.

Parte 6 — Pagamento do cliente por parte

- **Proposta:**
  - O "Pago" pergunta "Este pagamento é de que parte?". Por defeito fica "Todo o trabalho".

- A tabela de partes ganha a coluna "Pago".
- O Resumo financeiro do trabalho não muda.
- Exemplo: o cliente paga só o logotipo → o valor regista-se nessa parte → o logotipo fica "Pago" e o trabalho continua com o resto em falta.

- **Dados novos:** `Pagamento.itemId` (uma parte por pagamento).
- **Depende da Parte 5.**

B3. Factos verificados no código que o plano tem de respeitar

- **Ficheiros só no aparelho:**
  - Os ficheiros (fotos e PDFs) não vão para a nuvem: `TABELAS` em `src/nuvem.ts` não inclui `ficheiros`, e o servidor rejeita registos com mais de 300 000 caracteres.
  - O que se carrega num telemóvel não abre no computador. O cartão "Nuvem" em Definições já o diz.
- **Tabela nova:** obriga a mudar `TABELAS` em `src/nuvem.ts` e a lista aceite em `puzzle_enviar` (`db/002_puzzle_tarefas.sql`), e a subir `ESQUEMA`. Nenhuma parte do plano cria tabela nova.
- **Última gravação ganha:** a sincronização guarda o registo inteiro. Se dois aparelhos editarem o mesmo trabalho, perde-se a alteração de um deles. Já hoje é assim.
- **Faturas digitalizadas:** a fatura real da Académica é uma imagem sem texto, e a leitura automática (`lerFicheiro` em `src/leitura.ts`) não a lê. O caminho que funciona é registá-la à mão no Passo 4. O teste dessa fatura não está neste repositório, porque usa o documento real.
- **Total de faturas digitais:** `carregarLido` (`src/pages/RequisicaoForm.tsx`) ignora o total lido numa fatura digital. A Parte 1 precisa desta correcção.
- **Ordem das versões:**
  - As cotações vêm ordenadas pelo número em texto (`src/state.tsx`). Entre meses essa ordem falha: "COT-001-NOV/26" fica antes de "COT-003-OUT/26".
  - A cadeia de versões tem de seguir as ligações (`substituiId`), nunca o número.
  - Recomenda-se número novo para a versão negociada e não um sufixo "v2". O sufixo partiria `renumerar` em `src/seed.ts` e o PDF.
- **Cotação partilhada:** uma cotação pode estar ligada a vários trabalhos. O "Negociar" tem de perguntar que trabalhos passam a apontar para a nova.
- **Importação em massa:** `src/pages/Importar.tsx` (`ImportarModal`, `BotaoCarregar`) existe, mas não está ligado a nenhuma página. Nunca correu com dados reais e não lê imagens digitalizadas.

B4. Perguntas por responder ao Momad

1. **Número da versão negociada:** número novo (ex.: COT-005-OUT/26, com "substitui a COT-003" registado na app) ou o mesmo número com "v2"? Recomenda-se número novo, porque a numeração mensal e o PDF já funcionam assim.
2. **"Substitui" no PDF:** o PDF da cotação negociada deve dizer ao cliente "Substitui a cotação COT-003-OUT/26", ou isso fica só interno?
3. **Marcar "enviada":** depois de gerar o PDF, a app pergunta "Já enviou? [Sim, hoje]"? Marca sozinha, com Desfazer? Ou só por botão? Recomenda-se perguntar.
4. **Correcção numa cotação já enviada:** basta avisar, ou a app bloqueia e obriga a usar "Negociar"?
5. **Requisição:** chega melhorar a janela actual (faixa de estado, preview, "enviada" ao partilhar, Guardar só se mudou)? Ou prefere uma página inteira, como a cotação?
6. **Custo e lucro:** quando a fatura da Académica difere da lista, o custo e o lucro do trabalho passam a usar a fatura (sem IVA)? Hoje usam sempre a lista. A app mostraria "lista X · fatura Y".
7. **Estado das partes:** chega um estado simples por parte (por iniciar / em curso / feito / entregue), com as fases a continuarem por trabalho? Ou prefere as fases completas em cada parte?

8. **Encomendas em lotes** (ex.: vários camiões do mesmo cliente feitos por fases): cada lote continua a ser um trabalho separado? Ou passam a 1 trabalho com uma parte por lote?
9. **"Pagou só o logotipo":** basta marcar cada pagamento como "da parte X" (uma parte por pagamento)? Ou é preciso repartir um pagamento entre várias partes?
10. **Requisição "Várias"** (uma lista da Académica para vários trabalhos): continua só no Mapa de crédito, ou deve ficar ligada a mais de um trabalho e aparecer em cada um?
11. **Ficheiros na nuvem:** faturas, recibos e comprovativos ficam só no aparelho onde são carregados (o computador não vê o que foi carregado no telemóvel). Fica assim por agora, ou a nuvem de ficheiros entra neste plano?
12. **Importação em massa:** quer ligar já a carga de vários PDFs da Académica de uma vez, como botão secundário, ou fica para depois? Esta função só lê PDFs digitais.

B5. Outros pedidos pendentes

- **PPF, Proteção (vidros fumados, faróis, para-brisas) e Chrome delete**
  - A pesquisa está feita e a proposta foi enviada ao Momad. Temas recolhidos:
    - PPF: frente parcial, frente completa, "track pack", carroçaria completa, faróis, para-brisas, conchas dos puxadores.
    - Vidros: película tingida, de carbono ou cerâmica, de 5% a 70% de transmissão de luz, faixa de para-brisas e película de segurança.
    - Chrome delete: frisos das janelas, grelha, emblemas, puxadores, capas dos espelhos e barras de tejadilho, em preto brilhante ou mate.
  - Os materiais PPF e HEXIS já estão na lista (A8), mas estes serviços não têm gerador próprio.
  - O Momad não tem cotações antigas destes serviços. Tem amostras e o preçário do fornecedor. Falta uma sessão com ele para definir pacotes, marcas e preços.
  - O limite legal de escurecimento de vidros em Moçambique não foi confirmado. Não deve aparecer na app sem fonte oficial.
- **Página "Legislação"**
  - Ainda não existe. A ideia é listar as leis com links oficiais.
  - Fontes já identificadas:
    - Lei n.º 13/2023 (Lei do Trabalho, em vigor desde 21 de Fevereiro de 2024, revogou a Lei n.º 23/2007);
    - Código do IVA com as alterações da Lei n.º 10/2025 (em vigor desde 1 de Janeiro de 2026).
- **Novo visual**
  - Pedido: a app com o aspecto da cotação (Gotham, gradiente amarelo, padrão claro e ícones melhores no menu).
  - Primeiro faz-se uma maqueta para o Momad aprovar. Só depois se constrói.
- **Ficheiros na nuvem**
  - Estes ficheiros ficam só no aparelho onde foram carregados:
    - fotos e PDFs de faturas, recibos, pró-formas e comprovativos;
    - logótipos dos clientes;
    - ficheiros de arte.
  - Consequência: o logótipo do cliente só sai na cotação no aparelho onde foi carregado.
  - Próximo passo previsto: guardar os ficheiros no servidor, com uma função que os receba e entregue.
  - Os catálogos não são afectados, porque vêm com a app.
- **Catálogo de wrap**
  - O ficheiro do Momad ("Catálogo Wrap Projecto") só tem a capa.
  - Falta o PDF final. Quando chegar, junta-se em `public/catalogos` com uma entrada em `CATALOGOS`.

- **Outros pontos vistos no código:**
  - **PDF da cotação:** ainda abre a janela de impressão do sistema (`imprimirCotacao` em `src/print.tsx`). A requisição já gera o ficheiro pronto a partilhar. O método da requisição (html2canvas) não desenha bem as fontes da cotação, que estão dentro do SVG, por isso é preciso outro caminho.
  - **Lista de materiais da cotação:** é fixa (`MATERIAIS_COTACAO`). A caixa "Guardar na lista de materiais" de "Outro material" ainda não grava nada. Está prevista uma lista guardada nos dados da empresa.
  - **Frase fiscal na requisição:** o PDF da requisição não leva "Não serve de comprovativo fiscal". Falta confirmar com o Momad, e se preciso com o contabilista, se deve levar.
  - **Equipa:** os perfis e os relatórios diários existem, mas por agora só o Momad usa a app. Os acessos da equipa ficam para quando houver equipa.
  - **Fonte Gotham:** é uma fonte comercial e não está neste repositório. Os ficheiros copiam-se para `public/fonts/` (ver capítulo 3).

B6. Estado de cada pedido

Legenda: **Feito** = já funciona na app; **Planeado** = plano feito, à espera do OK do Momad; **Por fazer** = pedido aceite, ainda sem plano detalhado; **À espera** = depende de algo do Momad.

#	Pedido	Pedido em	Estado	Próximo passo
1	Requisição como a cotação	19 Out	Planeado (Parte 4)	OK do Momad e pergunta 5
2	Faturas da produção	19 Out	Planeado (Partes 1 e 2)	OK do Momad e perguntas 6, 11 e 12
3	Trabalho como conjunto (dossier)	19 Out	Planeado (Parte 2)	OK do Momad e pergunta 10
4	Estado e pagamento por parte	19 Out	Planeado (Partes 5 e 6)	OK do Momad e perguntas 7, 8 e 9
5	Negociar a cotação com versões	19 Out	Planeado (Parte 3)	OK do Momad e perguntas 1 a 4
6	PPF, Proteção e Chrome delete	17–18 Out	À espera	Sessão com o Momad: pacotes, marcas e preços
7	Página Legislação	18 Out	Por fazer	Construir a página com links oficiais
8	Novo visual	18 Out	Por fazer	Maqueta para aprovação
9	Ficheiros na nuvem	15 Out	Por fazer	Guardar os ficheiros no servidor, com uma função própria (pergunta 11)
10	Catálogo de wrap	18 Out	À espera	PDF final do Momad
11	PDF da cotação como ficheiro partilhável	16 Out	Por fazer	Escolher o método de geração
12	Lista de materiais da cotação editável	17 Out	Por fazer	Guardar materiais novos nos dados da empresa
13	Importação em massa de PDFs da Académica	—	O código existe, mas não está ligado	Pergunta 12
14	Acessos da equipa e relatórios diários	14 Out	Feito, por activar	Quando houver equipa a usar a app

#	Pedido	Pedido em	Estado	Próximo passo
—	Regras A1 a A21	13–18 Out	Feito	Manter os testes a passar (`npm test`)

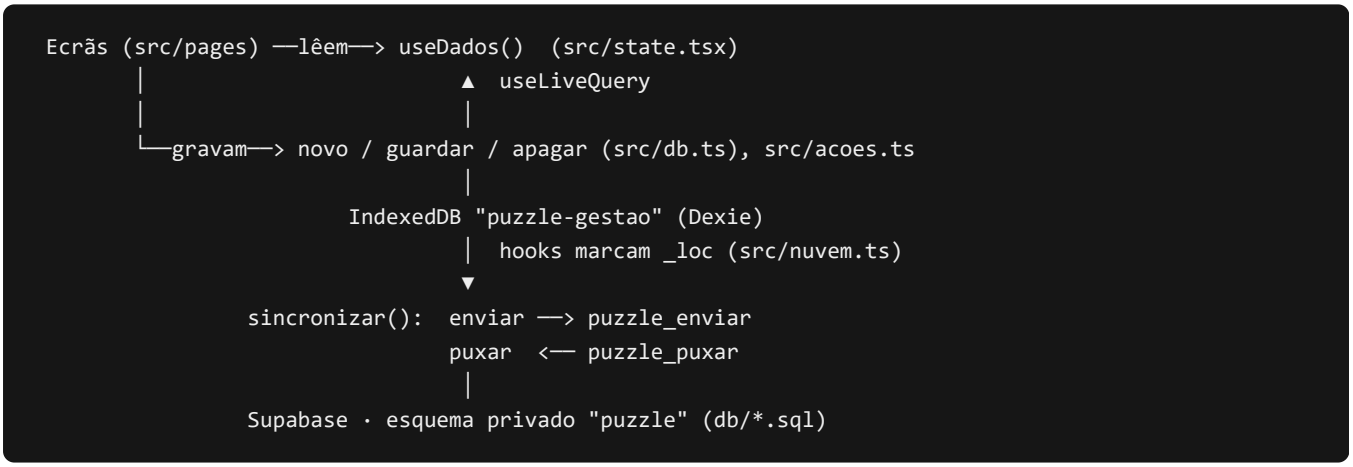
CAPÍTULO 3

Arquitetura e como continuar o desenvolvimento

Documento técnico do sistema **Puzzle Stickers · Gestão**, preparado pela KingBalas Ltd em outubro de 2026. É para o Momad e para quem continuar o desenvolvimento. Os caminhos de ficheiros são relativos à raiz do repositório.

Visão geral

- É uma aplicação web instalável (PWA) de página única, feita em React + TypeScript e construída com Vite.
- Funciona sem internet. Todos os registos vivem no próprio aparelho, numa base IndexedDB chamada `puzzle-gestao` (biblioteca Dexie).
- Há uma cópia da empresa na nuvem (Supabase). Cada aparelho envia e recebe registos dessa cópia, e a última escrita ganha.
- Os documentos que a app produz (cotação, requisição de prints, relação do IVA, Mapa de crédito em Excel) são gerados no próprio aparelho.
- Os ficheiros carregados (fotos, PDFs, logótipos) ficam só no aparelho onde foram carregados.
- Não há servidor próprio. A app é um conjunto de ficheiros estáticos (`dist/`), e a nuvem só expõe cinco funções SQL.



Stack e dependências

Versões tal como estão em `package-lock.json` :

Pacote	Versão	Para que serve
`react` , `react-dom`	18.3.1	Interface
`dexie`	4.0.11	Base de dados IndexedDB no aparelho
`dexie-react-hooks`	1.1.7	`useLiveQuery` : os ecrãs atualizam-se quando a base muda
`jspdf` + `html2canvas`	2.5.2 + 1.4.1	PDF real da requisição, gerado no aparelho
`pdfjs-dist`	4.10.38	Visualizador de PDFs, miniaturas e leitura do texto de PDFs

Pacote	Versão	Para que serve
`xlsx` (SheetJS)	0.20.3	Ler listas de impressão em Excel (`src/leitura.ts`) e exportar o Mapa de crédito
`@fontsource/montserrat`	5.1.1	Fonte da interface, incluída no build
`vite` (dev)	5.4.11	Servidor de desenvolvimento e build
`@vitejs/plugin-react` (dev)	4.3.4	Suporte React no Vite
`vite-plugin-pwa` (dev)	0.21.1	Service worker (Workbox) e manifesto
`typescript` (dev)	5.6.3	Verificação de tipos (modo `strict` , `noUnusedLocals`)
`vitest` (dev)	2.1.8	Testes. Traz também o `vite-node` , que o carregador `admin/carregar.ts` usa
`fake-indexeddb` (dev)	6.0.0	IndexedDB em Node, para o carregador de dados

Notas:

- O `xlsx` não vem do registo npm. Instala-se a partir de `https://cdn.sheetjs.com/xlsx-0.20.3/xlsx-0.20.3.tgz` , por isso o `npm install` precisa de acesso a esse endereço.
- `jspdf` , `html2canvas` , `pdfjs-dist` e `xlsx` só são carregados quando são precisos (`import()` dinâmico). Não pesam no arranque.
- O projeto não usa biblioteca de rotas, framework de CSS nem gestor de estado. As rotas são por hash (`src/state.tsx`), o estilo é um único `src/styles.css` e o estado vem de contextos React.
- O Wrangler (publicação) não está no `package.json` : é chamado com `npx wrangler@4` .

Scripts (`package.json`):

Comando	Faz
`npm run dev`	`vite --port 5195 --host` : servidor de desenvolvimento, também visível na rede local
`npm run dev:fixed`	`vite --port 5195 --strictPort` : sempre na porta 5195, e falha se ela estiver ocupada
`npm run build`	`tsc --noEmit && vite build` : verifica os tipos e gera `dist/`
`npm run preview`	`vite preview --port 5196` : serve `dist/` localmente
`npm test`	`vitest run` : corre todos os testes uma vez

Estrutura de pastas

```
.
├─ index.html          página única: #root (app) e #print-root (impressão)
├─ vite.config.ts      Vite + React + PWA; carimbo de versão; versao.json; ambiente dos testes
├─ tsconfig.json       TypeScript strict; verifica só src/ (sem os testes)
├─ package.json, package-lock.json
├─ README.md, .gitignore
├─ docs/               esta documentação
├─ db/                 SQL da nuvem: 001_puzzle_nuvem.sql, 002_puzzle_tarefas.sql
├─ admin/carregar.ts   carrega lotes de dados reais para a nuvem (corre em Node)
├─ mock/               pré-visualização do formulário de cotação (só em desenvolvimento)
├─ public/             servido tal como está
│  ├─ img/             logótipos, ícones, arte de fundo da cotação
│  ├─ fonts/           vazia no repositório: aqui entram os 8 gotham-*.woff da empresa
│  └─ catalogos/       3 catálogos em PDF + capas .jpg
├─ src/                código da app
│  └─ pages/           ecrãs
└─ test/               dump.mjs (ajuda) e fixtures/gotham-advances.json
```

O que **não** está no repositório (ver `.gitignore`):

- `admin/dados/` : scripts com os dados reais da empresa.
- `admin/ficheiros/` : documentos reais.
- `test/fixtures/*.pdf` e `test/fixtures/*.xlsx` : documentos reais da Académica Digital, retirados por privacidade. Os testes que os liam também ficaram de fora.
- `public/fonts/gotham-*` : a fonte Gotham é comercial (licença da empresa). Para os documentos saírem iguais aos originais, copie os 8 ficheiros `gotham-*.woff` da empresa para `public/fonts/` . Os nomes estão nos `@font-face` de `src/styles.css` .
- Pastas e ficheiros gerados ou locais: `node_modules/` , `dist/` , `test/out/` , `.wrangler/` , `*.log` , `.env*` .

Os dados reais vivem na nuvem, não no código. Os exemplos do `mock/` e dos testes usam nomes genéricos.

Ficheiros principais de `src/`

Ficheiro	Papel	Exportações principais
`main.tsx`	Arranque: importa o CSS, regista o service worker, instala os hooks da nuvem, corre o seed num aparelho que já tem dados, inicia a sincronização e monta `<App/>` em `StrictMode`	—
`App.tsx`	Portas de entrada (`Ligar` , `Login`), concha da app (`Shell` : menu, separadores, rotas), `Page` (cabeçalho comum), aviso de versão nova, dica de instalação e estado da nuvem	`App` , `Page` , `PAPEL_NOME` , `textoNuvem`
`state.tsx`	Rotas por hash, sessão e permissões, todos os dados em memória, códigos sequenciais, estado online, tema	`useRota` , `ir` , `useSessao` , `SessaoProvider` , `useDados` , `DadosProvider` , `proximoCodigo` , `useOnline` , `useTema`

Ficheiro	Papel	Exportações principais
`db.ts`	Modelo de dados (interfaces), base Dexie (versões 1 a 5) e funções de gravação	`db`, `uid`, `agora`, `novo`, `guardar`, `apagar`, `vivos`, `registar`, `guardarFicheiro`
`nuvem.ts`	Sincronização com a nuvem, sessão da empresa e modo local de desenvolvimento	`TABELAS`, `devLocal`, `instalarHooks`, `sincronizar`, `agendar`, `iniciarSincronizacao`, `entrar`, `desligar`, `terminarSessao`, `mudarSenha`, `useNuvem`, `estadoNuvem`
`seed.ts`	Dados iniciais (empresa, catálogo Criação de Imagem 2026, utilizador dono) e correções que correm em cada arranque	`EMPRESA`, `CATALOGO`, `semear`, `apagarExemplos`, `repor`
`calc.ts`	Regras de negócio puras, sem React: fases e tipos, totais, IVA, despesas, ciclo do fornecedor, agenda, numeração e caixa	`ETAPAS`, `TIPOS`, `normalizarEtapa`, `resumoTrabalho`, `totaisRequisicao`, `totaisCotacao`, `comUrgencia`, `ivaDoMes`, `resultadoDoMes`, `faseRequisicao`, `mapaCredito`, `itensAgenda`, `numeroDoMes`, `movimentosCaixa`, `saldoCaixa`
`acoes.ts`	Ações partilhadas que gravam	`moverEtapa`, `novaTarefa`, `marcarTarefa`, `adicionarNota`
`format.ts`	Formatos em português (valores em MT, datas), leitura de números, ligação de WhatsApp, cidade/país	`mt`, `mtCurto`, `dataLonga`, `dataDoc`, `parseNum`, `waLink`, `separarMorada`, `juntarMorada`
`ui.tsx`	Componentes de base	`Icon`, `Logo`, `Modal`, `Confirm`, `Field`, `Avatar`, `Empty`, `Seg`, `ToastHost` / `useToast`, `UploadZone`, `Thumb`, `useFicheiro`, `abrirFicheiro`, `useMenu`, `useVer`
`Visualizador.tsx`	Visualizador de ficheiros dentro da app (desenha os PDFs página a página com pdf.js) e miniaturas	`VisualizadorHost`, `Miniatura`
`leitura.ts`	Leitura automática de PDFs com texto (pró-forma, fatura e recibo da Académica; listas de impressão) e de listas em Excel	`lerFicheiro`, `lerAcademica`, `lerListaPdf`, `lerListaGrelha`, `lerListaExcel`, `textoPdf`, `carregarPdfjs`, `numeroPt`
`print.tsx`	Documentos pelo diálogo de impressão (`#print-root` + `window.print`) e CSV do IVA	`imprimirCotacao`, `imprimirIva`, `exportarIvaExcel`, `imprimirLista` (que chama `emitirRequisicao`)
`pdfPartilha.tsx`	PDF real gerado no aparelho (html2canvas + jsPDF) e janela "PDF pronto" (Partilhar, Descarregar, abrir o WhatsApp, Ver)	`renderizarPdf`, `PdfHost`, `emitirRequisicao`
`cotacaoPdf.tsx`	Página A4 da cotação em SVG (em pontos de PDF), igual ao original do Momad, com páginas por opção	`PaginaCotacao`, `paginar`, `paginasCotacao`, `blocoTotais`, `medirCanvas`, `carregarFontes`, `logoDoCliente`

Ficheiro	Papel	Exportações principais
`requisicaoPdf.tsx`	Documento "Requisição de prints" em HTML A4 e o preçário da Académica 2026	`PaginaRequisicao`, `CSS_REQUISICAO`, `PRECARIO_ACADEMICA_2026`, `nomeOficial`, `nomeMaterial`, `totaisLinhas`, `linhasComMaterial`
`materiaisCotacao.ts`	Materiais, notas, textos finais, gerador de trabalho impresso e tabela de deslocação da cotação	`MATERIAIS_COTACAO`, `NOTAS_ITEM`, `TEXTOS_FINAIS`, `IMPRESSO`, `frasePartes`, `DESLOCACAO`
`versao.ts`	Versão do build e atualização forçada	`VERSAO`, `versaoCurta`, `versaoNoServidor`, `atualizarApp`
`styles.css`	Todo o estilo da interface, mais os `@font-face` da Gotham	—
`*.test.ts`, `*.test.tsx`	Testes (ver "Testes")	—

Ecrãs em `src/pages/`

Ficheiro	Onde aparece	Papel
`Painel.tsx`	`#/painel` (Início)	Hoje, Próximos 7 dias e Sem data; contadores Pedidos, Em negociação, Em curso e Para entregar; botões rápidos (Novo pedido, Cotação, Fatura, Recibo, Compra); "O que precisa de atenção" (alertas); faixa Caixa
`Trabalhos.tsx`	`#/trabalhos`	Quadro por fase (com arrastar) ou lista; pesquisa; filtros por tipo, responsável, atrasados e sem valor
`Trabalho.tsx`	`#/trabalhos/:id`	Ficha do trabalho com os separadores Detalhes, Requisições, Pagamentos, Faturas e recibos, Notas e histórico. Inclui `PagamentoForm` e `CustoForm`
`TrabalhoForm.tsx`	janela	Criar ou editar um trabalho (tipo, cliente, partes)
`Partes.tsx`	formulários	Editor das partes do trabalho. Em criação de imagem as partes vêm só do catálogo (`soCatalogo`)
`Fases.tsx`	ficha do trabalho	`FasesTrabalho` : fase atual, botão para avançar, prazos por fase, janela de entrega (`EntregaModal`)
`comum.tsx`	vários	`EtapaBadge`, `PrazoBadge`, `JobCard`, `ProgressoPago`, métodos de pagamento
`Agenda.tsx`	`#/agenda`	Tarefas e datas das fases (Atrasado, Hoje, Próximos 7 dias, Mais tarde, Sem data) e vista do mês. Inclui `NovaTarefa` e `TarefasDoTrabalho`
`Relatorios.tsx`	`#/relatorios`	Relatórios diários da equipa
`Rapido.tsx`	botões do Início e botão "+"	`NovoPedido`, `EmitirFatura` (regista a fatura do livro do Momad e liga-a a um ou mais trabalhos), `EmitirRecibo`
`Cotacoes.tsx`	`#/cotacoes`, `#/cotacoes/:id`	Lista e editor; `novaCotacao` ; aceitar a cotação e convertê-la em trabalho(s)

Ficheiro	Onde aparece	Papel
`cotacaoFormulario.tsx`	editor de cotação e `mock/`	`EditorCotacao` (linhas, gerador Wrap / Branding, catálogo, deslocação, urgência, desconto) e `PreviewCotacao`
`Catalogos.tsx`	`#/catalogos`	Catálogos em PDF: Partilhar PDF, Enviar link, Ver, Copiar link
`Clientes.tsx`	`#/clientes`, `#/clientes/:id`	Lista, ficha, `ClienteForm`, `LogoCliente`
`Fornecedores.tsx`	`#/fornecedores`, `#/fornecedores/:id`	Lista e ficha com o catálogo de materiais e as requisições; `FornecedorForm`
`Requisicoes.tsx`	`#/requisicoes`, `#/requisicoes/:id`	Lista por mês com contagem por fase; abre a requisição `:id`
`RequisicaoForm.tsx`	janela	Lista de impressão ou encomenda, e o Circuito de 5 passos: Lista enviada, Pró-forma, Pagamento ao fornecedor, Fatura e recibo do fornecedor, Impressão levantada ("Recebida / levantada" numa encomenda)
`ContasPagar.tsx`	`#/contas-pagar` (Mapa de crédito)	Mapa de crédito aos fornecedores, pagamentos parciais, exportação para Excel no formato da folha do Momad
`Resumo.tsx`	`#/resumo`	Caixa: entrou, saiu, saldo, "Acertar caixa" (`CaixaModal`), últimos 6 meses
`Dividas.tsx`	`#/dividas` (Recebimentos)	Ecrã "Pagamentos e dívidas": quem deve e pagamentos recebidos
`Despesas.tsx`	`#/despesas`	Despesas fixas (com janela de pagamento), compras e resultado do mês; `DespesaForm`
`Pessoal.tsx`	`#/pessoal`	Gastos pessoais pagos com dinheiro da empresa (Repor ou Autorizar)
`Documentos.tsx`	`#/documentos` (Faturas e recibos)	Arquivo de documentos emitidos e recebidos; `DocumentoForm`, `DocLinha`
`Iva.tsx`	`#/iva`	Relação mensal do IVA: botão "PDF" (impressão) e botão "Excel" (CSV)
`Catalogo.tsx`	`#/catalogo`	Catálogo de serviços (Criação de Imagem 2026)
`Equipa.tsx`	`#/equipa`	"Equipa e acessos": utilizadores, papéis e PINs
`Definicoes.tsx`	`#/definicoes`	Dados da empresa, regras do negócio, cópia de segurança, "Recomeçar este aparelho" e o cartão Nuvem
`Importar.tsx`	nenhum	Importação em massa de listas e documentos da Académica. **Não está ligado a nenhum ecrã** (ver armadilhas)

Recursos em `public/`

Ficheiro	Uso
`img/logo.svg`	Logótipo da interface, da cotação e da relação do IVA. Só tem `viewBox` e usa a cor `currentColor`. A interface aplica-lhe o filtro `--logo-filter`, que no tema escuro o inverte
`img/logo-doc.svg`	O mesmo logótipo com `width` / `height` e cor fixa (#111111), para documentos que passam pelo html2canvas (requisição)
`img/academica.png`	Logótipo do fornecedor na requisição, só quando o fornecedor é a Académica (`logoFornecedor` em `requisicaoPdf.tsx`). Nos outros casos aparece o nome
`img/cotacao-fundo.jpg`, `img/cotacao-fundo-livre.jpg`, `img/cotacao-barra-total.jpg`	Arte de fundo da cotação
`img/icon-192.png`, `img/icon-512.png`, `img/icon-maskable-512.png`, `img/apple-touch-icon.png`, `img/favicon.png`	Ícones do manifesto e do `index.html`
`img/cot-pattern.jpg`, `img/mark.png`	O código não os usa
`fonts/gotham-*.woff`	Gotham em 8 estilos (light, book, medium e bold, normal e itálico), só para os documentos. **Não estão no repositório**: copiar os ficheiros da empresa
`catalogos/*.pdf` e `catalogos/*.jpg`	3 catálogos e as capas, para o ecrã Catálogos. A lista está em `CATALOGOS`, em `src/pages/Catalogos.tsx`

Modelo de dados e nuvem (resumo)

Mais detalhe no capítulo 4 (Dados, nuvem e segurança).

Registos no aparelho (`src/db.ts`)

- Os registos das tabelas de negócio herdam `Base`: `id`, `createdAt`, `updatedAt`, `deleted?` e `demo?`. As exceções são `Empresa` (só tem `updatedAt`), `Registo` (usa `em`), `Ficheiro` e `Nuvem`.
- Para criar usa-se `novo()`, para gravar `guardar()` (que atualiza `updatedAt`) e para apagar `apagar()`. Apagar marca `deleted: true` e nunca remove o registo.
- `guardarFicheiro()` guarda um ficheiro em `ficheiros`. Uma imagem com mais de 2000 px, ou com 900 KB ou mais, é comprimida para JPEG com no máximo 2000 px.

Tabela	Desde a versão Dexie	Vai para a nuvem	Conteúdo
`empresa`	1	sim	Registo único `'empresa'`: dados da empresa, IVA, sinal, urgência, validade e nota das cotações, `seedVersao`, `caixa`
`utilizadores`	1	sim	Pessoas, papel, PIN, ativo
`clientes`	1	sim	Clientes. `logoId` aponta para `ficheiros`
`fornecedores`	1	sim	Fornecedores e o seu catálogo (`materiais`)

Tabela	Desde a versão Dexie	Vai para a nuvem	Conteúdo
`servicos`	1	sim	Catálogo Criação de Imagem 2026, com ids fixos `srv-…`
`trabalhos`	1	sim	Tipo, partes (`itens`), fase, prazos, notas, ficheiros
`pagamentos`	1	sim	Pagamentos dos clientes (`reciboId` quando já têm documento)
`requisicoes`	1	sim	Listas de impressão e encomendas, e o ciclo do fornecedor
`custos`	1	sim	Outros custos de um trabalho
`documentos`	1	sim	Faturas, faturas-recibo, recibos, vendas a dinheiro e notas de crédito, emitidos e recebidos. É a base do IVA
`cotacoes`	1	sim	Cotações
`ficheiros`	1	**não**	O conteúdo dos ficheiros (fotos, PDFs, logótipos)
`registos`	1	sim	Histórico de ações, escrito por `registar()`
`despesasFixas`	2	sim	Custos fixos mensais
`despesas`	2	sim	Compras, despesas e gastos pessoais
`relatorios`	3	sim	Relatórios diários
`meta`	4	**não**	Registo `'nuvem'` : sessão do aparelho e cursores de sincronização
`tarefas`	5	sim	Tarefas da agenda

Sincronização (`src/nuvem.ts`)

- **O que sincroniza:** as 16 tabelas em `TABELAS` . Ficam de fora `ficheiros` e `meta` .
- **Carimbo local:** `instalarHooks()` liga os hooks Dexie `creating` e `updating` em todas as `TABELAS` .
  - Cada escrita local recebe `_loc` , e uma atualização recebe também um `updatedAt` novo.
  - Cada escrita agenda uma sincronização para 1,5 s depois.
  - O que chega da nuvem é gravado numa transação marcada `__remoto` , por isso não volta a subir.
  - Não há hook de remoção: um `delete()` físico não chega aos outros aparelhos.
- **Versão de um registo:** `updatedAt` (ou, se faltar, `em` ou `createdAt`). Ganha a versão mais recente, no servidor e no aparelho (`aplicar`).
- **Ciclo `sincronizar()` :**
  1. Envia, em lotes de 200, tudo o que tem `_loc` igual ou posterior a `marca` (o primeiro envio de um aparelho manda tudo).
  2. Puxa por `seq` , a partir de `cursor - 100` , em páginas de 500 (até 100 páginas por ciclo).
  3. Grava `meta.nuvem` .
- **Quando corre:** ao arrancar, a cada 30 s, quando a rede volta, quando a app volta ao primeiro plano e 1,5 s depois de uma escrita. Corre uma vez de cada vez; um pedido que chega a meio fica para logo a seguir.
- `ESQUEMA` (hoje `2`): quando um aparelho tem um número menor, o cursor volta a 0 e ele puxa tudo outra vez.
- **Estado visível:** `useNuvem()` e `textoNuvem()` (`App.tsx`). Os textos são "A sincronizar…", "Na nuvem · hh:mm", "Sem internet · guardado no aparelho", "Erro ao sincronizar · toque para tentar" e "Não ligado à nuvem".
  - No computador aparecem no fundo do menu lateral, e tocar no texto sincroniza.

    - No telemóvel o menu lateral não aparece: o estado e o botão "Sincronizar agora" estão em Definições → Nuvem.

- **Servidor** (`db/001_puzzle_nuvem.sql` , `db/002_puzzle_tarefas.sql`):
    - Esquema privado `puzzle` com as tabelas `empresas` , `sessoes` e `docs` . O RLS está ativo e não há acesso direto.
    - Cada registo é uma linha JSON em `puzzle.docs` , com `empresa_id` , `tabela` , `id` , `dados` , `versao` , `seq` e `servidor_em` .
    - Cinco funções públicas `security definer` : `puzzle_entrar` , `puzzle_puxar` , `puzzle_enviar` , `puzzle_mudar_senha` e `puzzle_sair` . A função privada `puzzle.empresa_do_token` valida o token.
    - A palavra-passe da empresa é guardada com bcrypt. 5 tentativas erradas bloqueiam a entrada durante 10 minutos.
    - O token de sessão é aleatório e o servidor só guarda o hash SHA-256.
    - Mudar a palavra-passe termina as sessões dos outros aparelhos, e a nova tem de ter pelo menos 8 caracteres.
    - `puzzle_enviar` aceita no máximo 500 registos por pedido, só das tabelas da sua lista `tabs` , e ignora registos com mais de 300 000 caracteres. Só substitui um registo quando a versão recebida é mais recente.

Estado global (`src/state.tsx`)

`useDados()` : todos os dados em memória

- `DadosProvider` lê de uma vez, com um único `useLiveQuery` , as 15 tabelas de dados (todas menos `ficheiros` , `registos` e `meta`). Qualquer escrita numa delas volta a desenhar os ecrãs. A ideia é que uma gráfica tem centenas de registos, não milhões.
- Os registos apagados são filtrados com `vivos()` .
- Ordenações:

Tabela	Ordem
`trabalhos`	código, decrescente
`cotacoes`	número (texto), decrescente
`requisicoes` , `documentos` , `despesas` , `relatorios`	data, decrescente
`clientes` , `fornecedores`	nome
`servicos`	`ordem` (a ordem do PDF do catálogo)
`despesasFixas`	dia
`tarefas`	data; as que não têm data vão para o fim

- `resumo(t)` devolve `resumoTrabalho()` de `calc.ts` (totais, pago, em dívida, sinal, custos, lucro, atraso), em cache por `id + updatedAt` .
- Para buscar por id: `cliente(id)` , `user(id)` e `fornecedor(id)` .
- `pronto` fica `false` até à primeira leitura, e a `Shell` não desenha nada até lá. Se não houver registo `empresa` , usa-se `EMPRESA` de `seed.ts` .
- **Nomes antigos das fases** (anteriores a 16 de outubro de 2026) são traduzidos na leitura por `normalizarEtapa()` (`calc.ts` , mapa `ANTIGAS`). A leitura não reescreve os registos, por isso os dados crus na nuvem podem ter nomes antigos.

`useSessao()` : utilizador e permissões

- Dá `user` (o `Utilizador` com sessão aberta), `sair()` e `pode(acao)`.
- As permissões estão em `PERM`:

Ação	Dono	Gerente	Trabalhador
`financas`	sim	sim	—
`todosTrabalhos`	sim	sim	—
`editarTrabalho`	sim	sim	—
`iva`, `despesas`, `equipa`, `definicoes`, `apagar`	sim	—	—

- Um trabalhador só vê os trabalhos de que é responsável (`responsavelId`). Pode mover a fase dos seus trabalhos (`Fases.tsx`), mas não vê valores.
- As permissões só filtram a **interface**. Os dados no aparelho são sempre todos (ver armadilhas).

Rotas por hash

- `useRota()` lê `location.hash` (por defeito `/painel`) e devolve `parts` e `query`. `ir(caminho)` muda o hash. Cada mudança volta ao topo da página.
- As rotas por hash funcionam sem internet e a partir do ícone no ecrã principal, sem regras no servidor.
- Em `App.tsx`, o `<main>` usa como `key` o caminho e os parâmetros. Por isso, cada mudança de rota volta a montar o ecrã.
- A navegação está em `AREAS` (`App.tsx`). Cada área tem separadores. Um separador pode exigir uma ação (`acao`) ou estar escondido do trabalhador (`naoTrab`):

Área	Separadores (rota)	Quem vê
Início	`painel`	dono, gerente
Trabalhos	`trabalhos`, `agenda`, `relatorios`	todos
Cotações	`cotacoes` (financas), `catalogos`	dono, gerente
Clientes	`clientes`	dono, gerente
Fornecedores	`fornecedores`, `requisicoes`, `contas-pagar` (financas)	dono, gerente
Finanças	`resumo` (despesas), `dividas` (financas), `despesas` (despesas), `documentos` (iva), `iva` (iva), `pessoal` (despesas)	conforme a ação
Definições	`definicoes` (definicoes), `catalogo`, `equipa` (equipa)	conforme a ação

- Uma rota desconhecida ou sem permissão mostra "Sem acesso" (`Bloqueado`).
- Parâmetros em uso, por exemplo:
  - `#/trabalhos/<id>?tab=pagamentos` e `?entregar=1`
  - `#/despesas?mes=AAAA-MM` (o mesmo em `iva` e `resumo`)
  - `#/trabalhos?tipo=…&f=atrasados&etapa=…&novo=1` (também `f=semvalor`)
- No telemóvel (até 899 px):
  - o menu lateral dá lugar a uma barra inferior com até 6 áreas, sem Definições;
  - Definições abre-se por um botão no fim do Início;

- nos ecrãs Início, Agenda e Clientes aparece o botão flutuante "+" (Novo pedido), exceto para trabalhadores. Este botão só existe no telemóvel.

Outros

- `proximoCodigo('T', …)` gera o código seguinte de trabalho no formato `T-AAAA-NNN`. Cotações e requisições usam `numeroDoMes()` de `calc.ts`, nos formatos `COT-NNN-MMM/AA` e `REQ-NNN-MMM/AA`, e a contagem recomeça todos os meses.
- `useOnline()` diz se há rede. `useTema()` guarda o tema em `localStorage` (`pg_tema`), põe `data-theme` no `<html>` e atualiza a `theme-color`.
- Chaves de `localStorage`: `pg_sessao` (utilizador com sessão aberta), `pg_tema`, `pg_vista` (quadro ou lista), `pg_dica` (dica de instalação fechada) e `pg_dev_local` (só em desenvolvimento).

Arranque e login

Sequência de arranque (`src/main.tsx`)

1. `registerSW({ immediate: true })` regista o service worker e pede atualizações a cada 60 s e sempre que a app volta ao primeiro plano.
2. `instalarHooks()` corre antes de qualquer escrita.
3. `arrancar()`: se o aparelho já tem o registo `empresa` (ou se está em modo local de desenvolvimento), corre `semear()`. Num aparelho já semeado, isso só aplica `correcoes()`. Um aparelho novo não semeia nada antes de entrar na nuvem.
4. `iniciarSincronizacao()`.
5. Monta `<App/>` dentro de `<StrictMode>`.

Portas de entrada (`src/App.tsx`)

`App` decide o que mostrar:

1. Sem token da nuvem (e fora do modo local), mostra **`Ligar`**.
2. Com token, mas sem utilizador resolvido, mostra **`Login`** (PIN).
3. Caso contrário, monta `ToastHost`, `SessaoProvider`, `DadosProvider`, `VisualizadorHost`, `Shell` e `PdfHost`.

`Ligar`: liga o aparelho à cópia da empresa.

- Pede só a palavra-passe. O código da empresa (`puzzle` por defeito) fica escondido atrás do botão "Outra empresa".
- Ao entrar, corre `entrar()`, depois `semear()` e depois `sincronizar()`.
- `entrar()` (`nuvem.ts`) trata três casos:
  - **Aparelho novo e nuvem com dados:** apaga as tabelas sincronizadas do aparelho (os ficheiros ficam) e desce a cópia da nuvem. Assim não há duplicados.
  - **Aparelho novo e nuvem vazia:** os dados do aparelho passam a ser a primeira cópia.
  - **Aparelho que já esteve ligado** (a sessão tinha terminado): primeiro envia o que mudou cá, depois puxa.
- O aparelho fica identificado no servidor como, por exemplo, "iPhone · Safari".

Utilizador único:

- Com exatamente **um** utilizador ativo, a palavra-passe da empresa é o único login e não se pede PIN (variável `unico`). Foi um pedido do Momad de 18 de outubro de 2026.
- O ecrã de PIN só aparece quando houver equipa.

`Login` (PIN):

- Lista as pessoas ativas (dono, gerente, trabalhador, por esta ordem) e pede um PIN de 4 dígitos.
- O PIN é comparado no aparelho com `Utilizador.pin`.
- A sessão fica em `localStorage` (`pg_sessao`). Um trabalhador entra na Agenda; os outros entram no Início.

Sair, desligar, recomeçar:

Ação	Onde	O que acontece
Sair, com um só utilizador	menu lateral; fim do Início no telemóvel	`terminarSessao()`: sincroniza, fecha a sessão no servidor e deixa o token vazio. A app volta a pedir a palavra-passe e os dados ficam no aparelho
Sair, com equipa	menu lateral; fim do Início no telemóvel; barra inferior do trabalhador	Apaga `pg_sessao` e volta ao ecrã de PIN. O aparelho continua ligado
Desligar este aparelho	Definições → Nuvem	`desligar()`: sincroniza, fecha a sessão e apaga `meta.nuvem`. Ao voltar a entrar, se a nuvem tiver dados, as tabelas sincronizadas do aparelho são substituídas pela cópia da nuvem
Recomeçar este aparelho	Definições → Dados	`desligar()` e depois `repor()`: apaga toda a base do aparelho, **incluindo os ficheiros**, e recarrega
Sessão terminada	automático	O servidor responde `sessao` (por exemplo, a palavra-passe foi mudada noutro aparelho). O token fica vazio, os cursores ficam guardados e o ecrã Ligar mostra "A sessão terminou. Entre outra vez." Ao voltar a entrar, o aparelho envia o que tinha pendente

PWA e atualização

Configuração (`vite.config.ts`)

- `VitePWA` com `registerType: 'autoUpdate'`: o service worker novo instala-se sem pedir confirmação.
- O manifesto tem:
  - `name` "Puzzle Stickers · Gestão" e `short_name` "Puzzle Gestão";
  - `lang` `pt-MZ`, `start_url` e `scope` `/`, `display` `standalone`;
  - cores `#0b0b0c`;
  - ícones de 192, 512 e 512 *maskable*.
- Workbox:
  - `globPatterns`: `**/*.{js,mjs,css,html,png,svg,woff2}`, `img/*.jpg` e `fonts/*.woff`. É isto que fica disponível sem internet.
  - `navigateFallback: '/index.html'`.
  - `navigateFallbackDenylist: [/^\/catalogos\//]`. Sem isto, abrir `/catalogos/<id>.pdf` (por exemplo, o link enviado a um cliente) mostrava a app em vez do PDF.
  - `maximumFileSizeToCacheInBytes`: 4 MB.
- **Não** ficam em cache:
  - os PDFs e as capas de `catalogos/`, que precisam de internet (dois dos PDFs passam dos 4 MB);
  - `versao.json`, de propósito.
- `index.html` tem `noindex, nofollow`, as meta tags `apple-mobile-web-app-*` e dois contentores: `#root` e `#print-root`.

Versão e atualização (`src/versao.ts`)

- No build, `vite.config.ts` cria `VERSAO = new Date().toISOString()` , ou seja, a hora em UTC:
  - injeta-a no código como `__VERSAO__` ;
  - um plugin próprio (`versao`) escreve `/versao.json` com `{ v }` .
- `versaoCurta()` mostra a versão como "dd/mm hh:mm" na hora de Maputo (UTC+2). Aparece no menu lateral e em Definições → Nuvem ("Versão da app").
- `VersaoNova` (`App.tsx`) pergunta ao servidor pela versão (`versaoNoServidor()` , com `cache: 'no-store'`):
  - ao abrir, ao voltar ao primeiro plano e a cada 2 minutos;
  - se a versão do servidor for mais recente, mostra "Há uma versão nova (...). Os seus dados ficam guardados." com o botão **Atualizar**.
- `atualizarApp()` remove o service worker, apaga as caches e recarrega. **Não mexe na IndexedDB**: os dados ficam. O mesmo botão existe em Definições → Nuvem ("Atualizar app").
- No iPhone ou iPad, apagar a app do ecrã principal apaga os dados locais desse aparelho, incluindo os ficheiros, que não estão na nuvem. Para forçar uma versão nova usa-se "Atualizar", nunca reinstalar.

Instalação

`DicaInstalar` (`App.tsx`) explica como instalar:

- no iPhone ou iPad: Partilhar → Adicionar ao ecrã principal;
- no Mac com Safari: Ficheiro → Adicionar à Dock;
- nos outros navegadores: "Instalar app" / "Adicionar ao ecrã principal".

Não aparece se a app já estiver instalada ou se a dica tiver sido fechada (`pg_dica`).

Estilos e fontes

- **Um só ficheiro:** `src/styles.css` (cerca de 660 linhas), importado em `main.tsx` .
- **Tokens em** `:root` :
  - tema escuro por defeito: `--y: #ffc406` (o amarelo da marca), `--bg` , `--card` , `--line` , `--tx` , `--ok` , `--warn` , `--bad` , `--info` , `--r` ;
  - tema claro em `:root[data-theme='light']` ;
  - `--safe-b` e `--safe-t` para as margens seguras do iPhone.
- **Layout:**
  - `.shell` com o menu lateral `.side` no computador;
  - até 899 px: o `.side` desaparece e entram a barra inferior `.tabbar` , o botão flutuante `.fab` e as classes `.hide-m` , `.so-m` e `.so-m-block` .
- **Impressão:** `#print-root` está escondido no ecrã. Em `@media print` esconde-se `#root` e mostra-se `#print-root` . A página é A4 com margem 0.
- **Montserrat** (interface): pesos 400 a 800 de `@fontsource/montserrat` , importados no topo do CSS. O build inclui-os em `assets/` . A relação do IVA (CSS em `print.tsx` , classes `.pr`) também usa Montserrat.
- **Gotham** (documentos):
  - 8 regras `@font-face` com a família `'Gotham PS'` (300, 400, 500 e 700, normal e itálico), que apontam para `/fonts/gotham-*.woff` com `font-display: block` ;
  - usada só pela cotação (`F = 'Gotham PS'` em `cotacaoPdf.tsx`) e pela requisição (`CSS_REQUISICAO`);
  - antes de gerar um documento, a app espera pelas fontes (`carregarFontes()` ; `document.fonts.load` em `renderizarPdf`);

    - os ficheiros não estão no repositório. Sem eles, o navegador usa outra fonte e os documentos perdem o aspeto original.

- **Os estilos dos documentos ficam à parte do CSS da app:**
    - `CSS` em `print.tsx` (classes `.pr`);
    - `CSS_REQUISICAO` em `requisicaoPdf.tsx`, tudo com o prefixo `.rq`, injetado só enquanto o PDF é gerado;
    - a cotação é SVG (`.pr-svg` para imprimir, `.a4-prev` para pré-visualizar, ambas em `styles.css`).

Como cada documento é gerado

Mais detalhe no capítulo 5 (Documentos PDF, catálogos e cálculos).

Documento	Como	Onde
Cotação	SVG A4 em pontos (595,276 × 841,89). O fundo é a arte original. O texto é Gotham, com as medidas do original. Há páginas por opção. Sai pelo diálogo de impressão	`cotacaoPdf.tsx`; `imprimirCotacao` em `print.tsx`
Requisição de prints	HTML A4 (`.rq`), depois html2canvas e jsPDF. Abre a janela "PDF pronto" com "Descarregar" e "Ver". Mostra também "Partilhar" se o aparelho consegue partilhar ficheiros. Se não consegue e o fornecedor tem telefone, mostra "Descarregar e abrir o WhatsApp" (`wa.me` no telemóvel, `whatsapp://` no computador)	`requisicaoPdf.tsx`; `emitirRequisicao` em `pdfPartilha.tsx`
Relação do IVA	HTML A4 pelo diálogo de impressão. O CSV tem BOM, separador `;` e finais de linha `\r\n`	`imprimirIva` e `exportarIvaExcel` em `print.tsx`
Mapa de crédito	`.xlsx` gerado com SheetJS (botão "Excel")	`exportarMapa` em `ContasPagar.tsx`

Regra do negócio: a app **não emite nem numera faturas ou recibos**. Os formulários registam o número do livro do Momad (`Rapido.tsx`). Só trabalhos, cotações e requisições têm numeração interna. A própria cotação imprime "**Não serve de comprovativo fiscal", e a lista de cotações diz que elas "não servem de comprovativo fiscal".

Página de pré-visualização `mock/` (só em desenvolvimento)

- `mock/cotacao.html` e `mock/cotacao.tsx` mostram o formulário de cotação (`EditorCotacao`) e a página A4 (`PreviewCotacao`) com cotações de exemplo. Não pedem login nem usam a base de dados.
- Abre-se com o servidor de desenvolvimento ligado: `http://localhost:5195/mock/cotacao.html`.
    - `?v=pc` (por defeito): formulário ao lado da página A4.
    - `?v=fone`: formulário com a largura de um telemóvel (390 px).
    - `?v=a4`: só as páginas A4. Aceita vários exemplos separados por vírgula e serve para imprimir.
    - `&ex=wrap` (por defeito), `&ex=escola` ou `&ex=mensal` escolhe o exemplo; `&aberto=N` abre a linha N.
- Não entra no build, porque o `vite build` só usa `index.html` como entrada. Também não é verificado pelo `tsc`, que só olha para `src/`.

Como continuar

Requisitos

- Node.js 20 ou mais recente. O `pdfjs-dist` 4.10 pede Node 20. Os testes foram corridos com Node 24.
- Acesso à internet no `npm install` (registo npm e `cdn.sheetjs.com`).
- Os 8 ficheiros `gotham-*.woff` da empresa, para os documentos saírem iguais aos originais.
- Para publicar: uma conta Cloudflare (Wrangler 4, chamado com `npx`).

Instalar e correr

```
npm install
# copiar os 8 gotham-*.woff da empresa para public/fonts/
npm run dev:fixed      # http://localhost:5195
```

- `npm run dev` faz o mesmo com `--host`, o que deixa abrir a app noutro aparelho da mesma rede.
- A origem `localhost:5195` tem a sua própria IndexedDB. Da primeira vez aparece o ecrã **Ligar**.
- Há duas formas de trabalhar:
  1. **Entrar com a palavra-passe da empresa.** Atenção: está a trabalhar com os **dados reais**, e cada alteração sobe para a nuvem.
  2. **Usar o modo local de desenvolvimento** (abaixo).

Modo local de desenvolvimento (`pg_dev_local`)

Na consola do navegador, em `http://localhost:5195`:

```
localStorage.setItem('pg_dev_local', '1'); location.reload()
```

- `devLocal()` (`src/nuvem.ts`) só devolve `true` quando `import.meta.env.DEV` está ativo. No build esse código desaparece, por isso a produção nunca o usa.
- O modo local salta o ecrã Ligar. `semear()` cria a empresa, o catálogo e um utilizador dono, e a app abre sem PIN.
- Sem ligação à nuvem nada sobe ("Não ligado à nuvem"). Os dados ficam só nessa origem do navegador.
- Para sair: `localStorage.removeItem('pg_dev_local')`.
- Cuidado: se essa origem já foi ligada à nuvem com a palavra-passe, a sincronização continua a correr com os dados reais, mesmo em modo local. Para limpar, use Definições → "Recomeçar este aparelho" ou apague os dados do site `localhost:5195`.

Testes

```
npm test          # corre uma vez
npx vitest        # fica a observar alterações
```

No repositório: **4 ficheiros e 30 testes, todos a passar**.

Ficheiro	Testes	O que cobre
`src/calc.test.ts`	27	Lista de impressão (totais ao metical de uma folha, material por linha); trabalho (totais de uma cotação, urgência de 45% antes do IVA, dívida, sinal, custo, lucro); IVA do mês (liquidado − dedutível, crédito); fases e agenda (dias úteis, fases por tipo, nomes antigos, "a receber", itens da agenda, atraso); numeração mensal; caixa; despesas fixas (vencimento, estados, janela do dia 25 ao dia 5) e resultado do mês; ciclo do fornecedor (totais de pró-formas, sequência de fases, encomendas); mapa de crédito (totais da folha); cotação (urgência por linha, desconto antes do IVA, total por opção, várias viaturas)
`src/leitura.test.ts`	1	Leitura de números nos formatos "12 345,00", "1 234,00 MTn", "4,321.00", "56.000,00 MT" e "7,23" (`numeroPt`)
`src/cotacaoPdf.render.test.tsx`	1	Paginação de uma cotação de exemplo: as caixas dos itens ficam a menos de 1,2 pt das posições do modelo (219,4 / 303,5 / 411,1 / 518,2). Mede o texto com `test/fixtures/gotham-advances.json` . Escreve `test/out/cotacao.html` , para imprimir no Chrome e comparar
`src/requisicaoPdf.render.test.tsx`	1	Preço de cada linha com o seu material (7,92 m² de Bundle Wrap = 15.840 MT) e somas por material. Escreve `test/out/requisicao.html`

- Os testes que liam os PDFs e as folhas Excel reais (pró-formas, fatura, listas de impressão) usavam documentos reais e não estão no repositório.
- Os HTML de `test/out/` apontam para `public/fonts/gotham-*.woff` . Sem as fontes, a comparação visual não vale.
- Não há testes automáticos para os ecrãs, para a sincronização (`nuvem.ts`), para o seed, nem para a geração de PDF no aparelho (`pdfPartilha.tsx`). Isso valida-se no navegador, no computador e no telemóvel.
- `test/dump.mjs` imprime o texto que o pdf.js extrai da primeira página de um PDF. Serve para escrever ou verificar leitores: `node test/dump.mjs ficheiro.pdf` .

Build

```
npm run build      # tsc --noEmit && vite build
npm run preview    # http://localhost:5196
```

- `dist/` contém `index.html` , `assets/` , `sw.js` , `workbox-*.js` , `manifest.webmanifest` , `versao.json` , `img/` , `fonts/` e `catalogos/` .
- Pesa cerca de 20 MB, dos quais cerca de 14 MB são os catálogos.
- O aviso do Vite sobre blocos maiores que 500 kB é esperado.
- Sem as fontes Gotham em `public/fonts/` , o build passa na mesma, com avisos `didn't resolve at build time` . Ver a armadilha 10.

Publicar (deploy atual)

```
npm test
npm run build
npx wrangler@4 deploy --assets dist --name puzzle-gestao --compatibility-date 2026-10-01
```

- Antes do build, confirme que `public/fonts/` tem os 8 `gotham-*.woff`.
- Da primeira vez num computador é preciso `npx wrangler@4 login` com a conta Cloudflare da KingBalas Ltd.
- Não há `wrangler.toml`: o comando define tudo. É um Worker só de ficheiros estáticos (static assets).
- Depois de publicar, os aparelhos recebem a versão nova sozinhos (service worker e aviso "Há uma versão nova").
- Para confirmar, compare a "Versão da app" em Definições → Nuvem com `dist/versao.json`. O ficheiro está em UTC e a app mostra a hora de Maputo (+2 h).
- Se o SQL mudou, aplique-o no Supabase **antes** de publicar a app.

Onde está alojado hoje

Peça	Onde	Conta	Notas
App (`dist/`)	Cloudflare Workers, static assets, Worker `puzzle-gestao`	KingBalas Ltd	`https://puzzle-gestao.apps-mz.workers.dev`, com `noindex`
Registos	Supabase, projeto `kb-academy`, esquema privado `puzzle`	KingBalas Ltd	O projeto serve também outros sistemas da KingBalas Ltd, e os dados da Puzzle Stickers estão isolados neste esquema. A app só chama as funções `public.puzzle_*`. O URL e a chave pública estão em `src/nuvem.ts` (constantes `API` e `KEY`). Pode passar para uma conta do Momad (ver abaixo)
Ficheiros	Só nos aparelhos (IndexedDB)	—	Não há cópia na nuvem
Catálogos	`public/catalogos`, publicados com a app	—	—
Fontes Gotham	Publicadas com a app em `/fonts/`	—	Não estão no repositório

A chave em `src/nuvem.ts` é a chave pública (anon) do Supabase. É pública por desenho: sem o token de sessão da empresa, as funções não devolvem nada.

Alojar noutra conta

As duas peças são independentes. Pode mudar só a nuvem, só a app, ou as duas.

Nuvem: projeto Supabase próprio

1. **Criar o projeto** no Supabase, na conta do Momad.
2. **Correr o SQL.** No SQL Editor, correr por ordem `db/001_puzzle_nuvem.sql` e `db/002_puzzle_tarefas.sql`.
   - O 001 já aceita a tabela `tarefas`. O 002 repete a mesma função `puzzle_enviar`, por isso corrê-lo a seguir não faz mal.
   - As funções usam a extensão pgcrypto no esquema `extensions` (`extensions.crypt`, `digest`, `gen_random_bytes`, `gen_salt`). Os projetos Supabase trazem-na. Se faltar: `create extension if not exists pgcrypto with schema extensions;`.
   - Não é preciso expor o esquema `puzzle` na API.
3. **Criar a empresa.** Os ficheiros SQL não a criam:

```
insert into puzzle.empresas (id, nome, senha_hash)
values ('puzzle', 'Puzzle Stickers',
        extensions.crypt('<palavra-passe>', extensions.gen_salt('bf', 10)));
```

- `puzzle` é o código que o ecrã Ligar usa por defeito e o que `admin/carregar.ts` usa. O código vai em minúsculas.
- Use pelo menos 8 caracteres, que é o mínimo que `puzzle_mudar_senha` exige.

4. **Copiar os dados.**
   - Pare a app: todos os aparelhos mostram "Na nuvem" e ninguém está a editar.
   - Exporte do projeto atual as linhas de `puzzle.docs` com `empresa_id = 'puzzle'` (colunas `empresa_id`, `tabela`, `id`, `dados`, `versao`) e insira-as no projeto novo. `seq` e `servidor_em` preenchem-se sozinhos.
5. **Apontar a app para o projeto novo.** Em `src/nuvem.ts`:
   - `API = 'https://<ref-do-projeto>.supabase.co/rest/v1/rpc/'`;
   - `KEY` = a chave pública (anon) do projeto novo.
   - `rpc()` envia a chave nos cabeçalhos `apikey` e `Authorization: Bearer`, que é o formato da chave anon (JWT). Se usar as chaves novas do tipo *publishable*, teste primeiro.
6. **Subir** `ESQUEMA` em `src/nuvem.ts` (de `2` para `3`), no mesmo build.
   - Cada aparelho guarda um cursor (`seq`) do servidor antigo, e no servidor novo os `seq` recomeçam. Um cursor alto faria o aparelho saltar registos.
   - Com `ESQUEMA` maior, o cursor volta a 0 na sincronização seguinte e o aparelho puxa tudo uma vez. Só são aplicadas versões mais recentes, por isso é seguro.
7. **Testar e publicar:** `npm test`, `npm run build` e publicar.
8. **Em cada aparelho**, o token antigo não existe no servidor novo, por isso a app pede a palavra-passe (a nova). Ao entrar, envia o que estiver pendente e puxa tudo.

`admin/carregar.ts` importa `src/nuvem.ts` e usa as mesmas constantes. Passa a apontar para o projeto novo sem mais alterações.

App: qualquer alojamento estático

- `dist/` é só ficheiros estáticos. As rotas são por hash, por isso não precisa de regras de reescrita no servidor.
- Tem de ser servida em HTTPS, na **raiz** de um domínio. O manifesto usa `start_url` e `scope` `/`, e o código pede caminhos absolutos (`/img/…`, `/fonts/…`, `/catalogos/…`, `/versao.json`). Num subcaminho (por exemplo `…/puzzle-gestao/`) estes caminhos partem-se.
- Para usar outra conta Cloudflare: `npx wrangler@4 login` com essa conta e o mesmo comando de deploy. O endereço passa a ser `https://puzzle-gestao.<subdomínio-da-conta>.workers.dev`, ou um domínio próprio.
- **Mudar de endereço é mudar de origem**, e cada origem tem a sua IndexedDB vazia:
  - cada aparelho tem de voltar a entrar (puxa tudo da nuvem) e reinstalar a app no ecrã principal;
  - **os ficheiros carregados no endereço antigo não passam** (ver armadilha 1);
  - mudar só a nuvem, mantendo o endereço, não tem este problema.

Carregar lotes de dados na nuvem (`admin/carregar.ts`)

Serve para meter dados reais na nuvem sem os pôr no código.

```
PUZZLE_SENHA='<palavra-passe da empresa>' npx vite-node admin/carregar.ts admin/dados/<script>.ts --
seco
```

Em PowerShell:

```
$env:PUZZLE_SENHA = '<palavra-passe da empresa>'; npx vite-node admin/carregar.ts
admin/dados/<script>.ts --seco
```

- Corre como mais um aparelho:
  1. abre uma IndexedDB em memória (`fake-indexeddb`);
  2. entra na empresa `puzzle` e puxa tudo;
  3. corre `semear()` e depois a função `export default` do script;
  4. lista o que vai subir e sincroniza;
  5. no fim fecha sempre a sessão (`desligar`), mesmo que o script falhe.
- `--seco` mostra o que subiria e não envia nada. Use-o sempre primeiro.
- Um script de dados é um módulo em `admin/dados/` com `export default async function` . Usa as funções da app: `db` , `novo` , `guardar` e `apagar` de `src/db.ts` , e as regras de `src/calc.ts` . A pasta não está no repositório, porque tem os dados reais.
- Os ficheiros não são carregados: `db.ficheiros` está desativado neste modo.

Regras para mudar o código

- **Regras de negócio** vão para `src/calc.ts` como funções puras, com teste em `src/calc.test.ts` .
- **Gravar** com `novo()` , `guardar()` e `apagar()` de `src/db.ts` (ou com as ações de `src/acoes.ts`).
  - Apagar é sempre marcar `deleted` ; nunca `table.delete()` em tabelas sincronizadas.
  - As únicas remoções físicas estão em `seed.ts` (`apagarExemplos` e `catalogoComIdsFixos`), e não chegam aos outros aparelhos.
- **Campos novos** são opcionais, porque os registos antigos na nuvem não os têm.
  - Um campo novo numa tabela existente não precisa de SQL nem de versão Dexie nova, porque o registo vai inteiro como JSON.
  - Só precisa de versão Dexie se o campo for um índice.
- **Nomes antigos** traduzem-se na leitura, sem reescrever registos. Exemplos: `normalizarEtapa` em `calc.ts` ; `nomeOficial` e `linhasComMaterial` em `requisicaoPdf.tsx` . O tipo `producao` é lido como `branding` (`tipoInfo`), e `correcoes()` em `seed.ts` também o converte no arranque.
- **Faturas e recibos** nunca são emitidos nem numerados pela app; só se regista o número do livro. As cotações levam o aviso de que não servem de comprovativo fiscal.
- **Documentos iguais aos originais:** só mexer na geometria de `cotacaoPdf.tsx` ou de `requisicaoPdf.tsx` com os testes de render a passar e uma comparação visual (`test/out/` , com as fontes Gotham presentes).
- **Textos** em português; valores no formato `56.000,00 MT` (`mt` em `format.ts`).
- **Dados reais** nunca no código: vão para a nuvem com `admin/carregar.ts` .
- **Antes de cada publicação:** `npm test` e `npm run build` . Depois, confirmar a versão num aparelho.

Trabalho planeado (por aprovar)

O plano de 19 de outubro de 2026 parte dos 5 pontos levantados pelo Momad e organiza-os em 6 partes:

- tudo é aditivo e opcional;
- não cria tabelas novas nem sobe `ESQUEMA` ;
- os registos reais na nuvem leem-se sem migração;
- antes da Parte 1: versão de base no git e `npm test` a passar.

Parte	Esforço	Resumo
1	médio	Fatura da Académica registada sem pró-forma; pagamento ao fornecedor com comprovativo; nova fase "Fatura por pagar"; acesso direto a partir do trabalho. Hoje, o Passo 3 do Circuito (`RequisicaoForm.tsx`) só deixa pagar depois de registar a pró-forma ou pôr a crédito
2	médio	Dossier do trabalho: cartão "Percurso" (cotação → requisição → faturas da produção → fatura ao cliente) e linha do tempo completa, derivados por funções puras num ficheiro novo, `src/dossier.ts`
3	médio	Cotação marcada como "Enviada" com data; "Negociar" separado de "Editar"; cadeia de versões ligada por id
4	médio	Requisição como a cotação: faixa de estado, pré-visualização A4, marcada como enviada ao partilhar, Guardar só se mudou
5	grande	Partes com identidade: estado por parte, "sem custo", ligação à requisição e à linha da cotação
6	pequeno	Pagamento do cliente associado a uma parte

O plano tem 12 decisões para o Momad tomar antes. Entre elas: o número da cotação negociada, o custo pela fatura ou pela lista, os ficheiros na nuvem, e se o `Importar.tsx` deve ser ligado ou não.

Armadilhas conhecidas e dívida técnica

1. Os ficheiros ficam só no aparelho

- **Onde:**
  - `TABELAS` em `src/nuvem.ts` não inclui `ficheiros` ;
  - mesmo que um ficheiro fosse metido num registo, `puzzle_enviar` (`db/*.sql`) ignora registos com mais de 300 000 caracteres;
  - `admin/carregar.ts` desativa `db.ficheiros` .
- **O que fica só no aparelho:**
  - fotos e PDFs de faturas, recibos, pró-formas e despesas;
  - ficheiros dos trabalhos e anexos das notas;
  - logótipos dos clientes (`Cliente.logoId` , impressos na cotação) e dos fornecedores;
  - fotos dos relatórios.
- **Consequências:**
  - noutro aparelho o registo aparece, mas o ficheiro não abre, e a cotação sai sem o logótipo do cliente;
  - a cópia de segurança de Definições não inclui ficheiros;
  - os ficheiros perdem-se se a app for apagada do ecrã principal, se os dados do site forem limpos, com "Recomeçar este aparelho" ou ao mudar o endereço da app;
  - Definições → Nuvem já avisa disto.
- **Caminho possível:** um bucket privado no Supabase Storage, com uma função que valide o token da empresa. A sincronização faz-se pelo `id` do ficheiro, que os registos já guardam.

2. Finais de linha (LF e CRLF)

- Todos os ficheiros de texto do projeto estão em LF. O repositório não tem `.gitattributes` .
- O resultado depende da configuração do Git de cada computador:

  - o instalador do Git para Windows costuma propor `core.autocrlf=true`, e é o caso no computador de desenvolvimento da KingBalas Ltd;
  - com essa definição, um clone em Windows fica com CRLF na pasta de trabalho, e o Git volta a converter para LF no commit;
  - num computador com `core.autocrlf=false`, um editor que grave em CRLF faz o ficheiro inteiro aparecer alterado no diff;
  - scripts e ferramentas que procuram texto exato (patches, substituições) falham quando o ficheiro tem outro final de linha.
- **Fazer:** fixar LF com um `.gitattributes` na raiz e, depois de o criar, correr `git add --renormalize .`:

```
* text=auto eol=lf
*.pdf binary
*.woff binary
*.png binary
*.jpg binary
*.xlsx binary
```

- **Exceção intencional:** o CSV do IVA (`exportarIvaExcel` em `src/print.tsx`) usa `\r\n` e BOM para o Excel o abrir bem. Não "corrigir".

3. html2canvas e SVG sem tamanho

- `renderizarPdf` (`src/pdfPartilha.tsx`) desenha o documento com html2canvas. O html2canvas corta um SVG que só tem `viewBox`, sem `width` / `height`.
- `public/img/logo.svg` é assim, e usa `currentColor` para acompanhar o tema. Por isso `PaginaRequisicao` usa `public/img/logo-doc.svg`, que tem tamanho e cor fixos.
- **Regra:** qualquer SVG num documento que passe pelo html2canvas precisa de `width`, `height` e cores fixas.
- A cotação (SVG com texto em Gotham) continua a sair pelo diálogo de impressão (`imprimirCotacao`). Passá-la para PDF real exige outro caminho, porque o html2canvas não aplica as fontes web dentro de um SVG.

4. URL de blob e `StrictMode`

- `src/main.tsx` monta a app em `<StrictMode>`. Em desenvolvimento, o React corre cada efeito, a sua limpeza e o efeito outra vez.
- Se uma URL for criada com `URL.createObjectURL` num `useMemo` e revogada na limpeza de um `useEffect`, a primeira limpeza revoga-a e o `useMemo` não a volta a criar. O link fica morto.
- **Padrão certo:** criar a URL dentro do `useEffect`, guardá-la em estado e revogá-la na limpeza desse mesmo efeito. É o que fazem `PdfProntoModal` (`src/pdfPartilha.tsx`) e `useFicheiro` (`src/ui.tsx`).
- **Pendente menor:** `Miniatura` (`src/Visualizador.tsx`), o logótipo do cliente em `imprimirCotacao` (`logoDoCliente`) e `exportar()` em `src/pages/Definicoes.tsx` criam URLs que nunca são revogadas. É uma pequena fuga de memória.

5. `img.decode()` com a app em segundo plano

- `decode()` espera por um frame pintado. Com a app em segundo plano (por exemplo, o Momad muda para o WhatsApp enquanto o PDF é gerado), esse frame não chega e a promessa fica pendurada.
- `renderizarPdf` limita cada `decode()` a 2,5 s (`Promise.race` com `espera(2500)`).
- `imprimirCotacao` (`src/print.tsx`) faz `decode()` das imagens da cotação **sem** limite. Se o problema aparecer, aplique o mesmo `Promise.race`.

6. `seed.renumerar` é perigoso se `SEED_VERSAO` subir

- **O que faz:** `semear()` (`src/seed.ts`) corre `apagarExemplos()` , `semearReal()` , `renumerar()` e `correcoes()` sempre que `Empresa.seedVersao` é menor que `SEED_VERSAO` (hoje `3`).
  - `renumerar()` reescreve **todos** os códigos por ordem de data, no formato antigo: `T-AAAA-NNN` , `REQ-AAAA-NNN` e `COT-AAAA-NNN` .
  - `apagarExemplos()` apaga fisicamente os registos com `demo: true` . Essa remoção não chega aos outros aparelhos.
- **O problema:** as cotações e as requisições reais usam a numeração mensal (`COT-002-OUT/26` , `REQ-001-OUT/26`), e esses números já foram enviados a clientes e fornecedores. Subir `SEED_VERSAO` reescrevia-os em cada aparelho, e as alterações subiam para a nuvem.
- **Fazer:**
  - não subir `SEED_VERSAO` ;
  - pôr as correções de arranque em `correcoes()` , que corre sempre e tem de poder repetir-se sem efeitos;
  - se um dia for mesmo preciso subir, retirar antes as cotações e as requisições de `renumerar()` .

7. `src/pages/Importar.tsx` sem uso

- `ImportarModal` e `BotaoCarregar` ("Carregar ficheiros": várias listas e documentos da Académica de uma vez) não são importados em lado nenhum. Segundo as notas do projeto, os botões saíram dos ecrãs a 18 de outubro de 2026. O build não os inclui, mas o `tsc` verifica-os.
- A leitura automática (`src/leitura.ts`) continua em uso no Circuito da requisição, para ler a pró-forma ou a fatura da Académica. O pdf.js continua em uso no visualizador.
- Os leitores de listas (`lerListaPdf` , `lerListaGrelha` , `lerListaExcel`) só são usados pelo `Importar.tsx` .
- A leitura só funciona em PDFs com texto. Os documentos digitalizados e as fotos não têm texto e são preenchidos à mão.
- **Decisão em aberto:** ligar como botão secundário depois de testar com os documentos reais, ou remover.

8. Ordem das cotações por número entre meses

- `DadosProvider` ordena `d.cotacoes` pelo texto do número, por ordem decrescente (`src/state.tsx`).
- Com a numeração mensal, essa ordem não é a cronológica:
  - `"COT-001-NOV/26"` < `"COT-003-OUT/26"` ;
  - `"COT-001-JAN/27"` < `"COT-002-DEZ/26"` .
- Quando muda o mês, a lista de cotações (`Cotacoes.tsx`) e a ficha do cliente mostram cotações novas abaixo de antigas.
- **Fazer:** ordenar primeiro por `data` (e `createdAt`) e só depois por número. Qualquer cadeia de versões de cotação deve seguir ligações por id, nunca o número.

9. Uma tabela nova exige SQL, `TABELAS` e `ESQUEMA`

Passos, por esta ordem:

1. `src/db.ts` : criar a interface (com `extends Base`), declarar a tabela em `PuzzleDB` e acrescentar `this.version(6).stores({ … })` . Nunca alterar versões que já existem.
2. `db/003_….sql` : `create or replace` de `public.puzzle_enviar` com o nome novo na lista `tabs` , como em `db/002_puzzle_tarefas.sql` , e correr no Supabase **antes** de publicar. Sem isto, o servidor ignora os registos dessa tabela sem dar erro.
3. `src/nuvem.ts` : acrescentar o nome a `TABELAS` (os hooks passam a cobri-la sozinhos) e subir `ESQUEMA` . Um aparelho com uma versão antiga da app ignora os registos que não conhece mas avança o cursor. Só volta a puxá-los se o cursor recomeçar.

4. `src/state.tsx`: se os ecrãs precisarem da tabela, juntá-la à leitura do `DadosProvider` e à interface `Dados`.
5. `src/pages/Definicoes.tsx`: juntá-la à cópia de segurança (`exportar`).
6. Testes para as regras novas em `calc.ts`.

10. Fontes Gotham fora do repositório

- `public/fonts/gotham-*` está no `.gitignore`, porque a Gotham é uma fonte comercial.
- Num clone novo, a pasta `public/fonts/` fica vazia. O `npm run build` passa na mesma: só avisa que `/fonts/gotham-*.woff` não foi resolvido no build.
- Se publicar esse `dist/`, o site deixa de ter as fontes. A cotação e a requisição passam a sair com outra fonte, e a medição de texto da cotação deixa de bater com o original.
- **Fazer:** copiar os 8 ficheiros da empresa para `public/fonts/` antes de qualquer build que vá ser publicado.

Outra dívida técnica confirmada no código

Assunto	Onde	Situação e o que fazer
Permissões só na interface	`pode()` em `state.tsx`, `AREAS` em `App.tsx`	A nuvem conhece a empresa, não as pessoas: qualquer aparelho ligado recebe todos os registos. Quando os trabalhadores usarem a app, vai ser preciso ter sessões por pessoa e filtrar os dados no servidor
PIN em texto simples	`Utilizador.pin` (`db.ts`), `Login` (`App.tsx`)	O PIN é comparado no aparelho e sincronizado com os outros registos. O utilizador criado pelo seed tem um PIN inicial fixo, e o ecrã de PIN mostra-o enquanto não for mudado. Com um só utilizador, não se pede PIN. Antes de criar equipa, mudar os PINs em Definições → Equipa
Números gerados no aparelho	`proximoCodigo` (`state.tsx`), `numeroDoMes` (`calc.ts`)	Sem sequência no servidor, dois aparelhos a criar ao mesmo tempo (ou um sem internet) podem gerar o mesmo número. Como só contam registos não apagados, o número de um registo apagado pode voltar a ser usado
Cópia de segurança incompleta	`exportar()` em `Definicoes.tsx`	Exporta 11 tabelas e deixa de fora `despesasFixas`, `despesas`, `relatorios`, `tarefas`, `registos` e `ficheiros`. Usar `TABELAS` de `nuvem.ts`
Histórico invisível	`registar()` em `db.ts`	Grava ações em `registos` (que sincroniza), mas nenhum ecrã lê essa tabela. O separador "Notas e histórico" mostra notas, passagens de fase e pagamentos
Textos do protótipo	`Page` em `App.tsx` ("guardado no aparelho" e a dica "Neste protótipo..."); cartão Dados em `Definicoes.tsx`	São de antes da nuvem. Atualizar
Cotação e IVA por impressão	`imprimirCotacao` e `imprimirIva` em `print.tsx`	Só a requisição sai como PDF real com Partilhar. A cotação também deve sair como ficheiro PDF
Fonte Gotham	`public/fonts/gotham-*.woff`	Os ficheiros não estão no repositório. Quem continuar o projeto copia-os da empresa para `public/fonts/`
Catálogos sem internet	`globPatterns` em `vite.config.ts`	Os PDFs e as capas de `catalogos/` não ficam em cache, por isso não abrem sem rede
Imagens sem uso	`public/img/cot-pattern.jpg`, `public/img/mark.png`	O código não as usa, mas entram na pré-cache (cerca de 118 KB)

CAPÍTULO 4

Dados, nuvem e segurança

Este capítulo explica onde ficam os dados do Puzzle Stickers · Gestão, como passam de um aparelho para outro e o que os protege. Tudo o que está aqui foi confirmado no código. Os caminhos dos ficheiros são relativos à raiz do repositório.

Resumo

- Cada aparelho tem uma cópia completa dos dados numa base IndexedDB local, gerida pela biblioteca Dexie (`src/db.ts`). Por isso a app funciona sem internet.
- A cópia da empresa na nuvem fica num schema privado `puzzle`, numa base PostgreSQL do Supabase. Cada registo da app corresponde a uma linha, com o registo inteiro em JSON.
- A app só fala com a nuvem através de cinco funções públicas: `puzzle_entrar`, `puzzle_puxar`, `puzzle_enviar`, `puzzle_mudar_senha` e `puzzle_sair`. Para entrar é preciso a palavra-passe da empresa. Depois disso, cada chamada leva um token de sessão.
- Na sincronização, a app envia o que mudou no aparelho e depois puxa o que mudou na nuvem. Quando há conflito, a versão mais recente fica com o registo inteiro (*last-write-wins* por `updatedAt`).
- Os ficheiros (fotos, PDFs, logotipos) ficam só no aparelho onde foram carregados.
- Cronologia: a nuvem entrou em funcionamento a 15 de Outubro de 2026. A tabela `tarefas` passou a ser aceite a 16 de Outubro de 2026. A entrada passou a pedir só a palavra-passe da empresa a 18 de Outubro de 2026.

```
APARELHO (PWA)                                NUVEM (Supabase)
IndexedDB "puzzle-gestao" (Dexie)             schema puzzle (privado)
 |- 16 tabelas de dados   -- puzzle_enviar -->  |- empresas  (código, hash da palavra-passe)
 |                        <-- puzzle_puxar --   |- sessoes   (hash do token, aparelho)
 |- ficheiros (só local)                        '- docs      (1 linha por registo, em JSON)
 '- meta (sessão e cursores, só local)         schema public: funções puzzle_* (RPC)
```

Ficheiros principais

Ficheiro	Papel
`src/db.ts`	Interfaces, base Dexie (`PuzzleDB`), versões e funções auxiliares
`src/nuvem.ts`	Sincronização, sessão, hooks Dexie e estado da nuvem
`src/state.tsx`	Carrega as tabelas de negócio para memória (`DadosProvider`) e define as permissões por papel (`PERM`)
`src/seed.ts`	Dados iniciais (empresa, catálogo, primeiro utilizador), correcções de arranque, `apagarExemplos()` e `repor()`
`src/main.tsx`	Instala os hooks, corre `semear()` num aparelho que já tem dados e arranca a sincronização
`src/App.tsx`	Ecrã `Ligar` (palavra-passe da empresa), ecrã de PIN (`Login`) e estado da nuvem na barra lateral
`src/pages/Definicoes.tsx`	Cartão Nuvem, cópia de segurança JSON, "Recomeçar este aparelho"

Ficheiro	Papel
`db/001_puzzle_nuvem.sql`	Schema `puzzle`, tabelas e funções
`db/002_puzzle_tarefas.sql`	Acrescenta `tarefas` à lista de tabelas aceites pela nuvem
`admin/carregar.ts`	Carregador de lotes de dados reais, que escreve directamente na nuvem

Base de dados local (Dexie)

A base chama-se `puzzle-gestao` (classe `PuzzleDB` em `src/db.ts`). Cada aparelho e cada navegador tem a sua própria cópia.

Versões, tabelas e índices

Versão	Tabela	Interface	Índices (o primeiro é a chave)	Vai para a nuvem
1	`utilizadores`	`Utilizador`	`id, papel`	sim
1	`clientes`	`Cliente`	`id, nome`	sim
1	`fornecedores`	`Fornecedor`	`id, nome`	sim
1	`servicos`	`Servico`	`id, categoria`	sim
1	`trabalhos`	`Trabalho`	`id, codigo, clienteId, etapa, prazo, responsavelId`	sim
1	`pagamentos`	`Pagamento`	`id, trabalhoId, data`	sim
1	`requisicoes`	`Requisicao`	`id, trabalhoId, fornecedorId, data`	sim
1	`custos`	`Custo`	`id, trabalhoId, data`	sim
1	`documentos`	`Documento`	`id, data, direcao, trabalhoId, clienteId, fornecedorId`	sim
1	`cotacoes`	`Cotacao`	`id, clienteId, data`	sim
1	`ficheiros`	`Ficheiro`	`id`	**não**
1	`registos`	`Registo`	`id, em`	sim
1	`empresa`	`Empresa`	`id`	sim
2	`despesasFixas`	`DespesaFixa`	`id, categoria`	sim
2	`despesas`	`Despesa`	`id, data, fixaId, mes, categoria`	sim
3	`relatorios`	`Relatorio`	`id, data, utilizadorId`	sim
4	`meta`	`Nuvem`	`id`	**não**
5	`tarefas`	`Tarefa`	`id, data, trabalhoId`	sim

Notas:

- Todas as chaves são `id` (texto). Só os campos da tabela acima estão indexados. Os outros campos não precisam de ser declarados, e um campo novo não indexado não pede versão nova.

- A app não faz consultas por índice: não há `.where()` nem `.orderBy()` em `src/`. O `DadosProvider` (`src/state.tsx`) lê 15 tabelas inteiras com `useLiveQuery` e guarda-as em memória. Não lê `registos`, `ficheiros` nem `meta`. Retira os registos apagados com `vivos()`. A empresa tem centenas de registos, não milhões.
- Para acrescentar uma tabela, cria-se uma `this.version(6).stores({...})` nova. As versões antigas nunca se alteram. Ver também Tabela nova.

Funções auxiliares (`src/db.ts`)

Função	O que faz
`uid()`	Gera um id novo com `crypto.randomUUID()`. Se não existir, usa `id-` + texto aleatório + tempo
`agora()`	Devolve a data e hora actuais em ISO 8601 UTC (`toISOString()`)
`novo<T>(dados)`	Cria um objecto com `id`, `createdAt` e `updatedAt` iguais a agora. Os dados passados podem substituir estes campos
`guardar(tabela, reg)`	Põe `updatedAt` igual a agora e grava com `put`
`apagar(tabela, id)`	Apagamento suave: marca `deleted = true`, actualiza `updatedAt` e grava. O registo continua na base e na nuvem
`vivos(lista)`	Retira da lista os registos com `deleted`
`registar(por, acao, alvo?)`	Acrescenta uma linha ao histórico `registos`
`guardarFicheiro(file, nome?)`	Guarda um ficheiro na tabela `ficheiros` e devolve o id. As imagens passam antes por `comprimirImagem`
`comprimirImagem` (interna)	Reduz a imagem a 2000 px no máximo (JPEG, qualidade 0,82). Uma imagem que já cabe em 2000 px e tem menos de 900 KB fica como está. Se não a conseguir ler, guarda o original

Regras práticas:

- Os dados de negócio apagam-se sempre com `apagar()`. `Table.delete()`, `bulkDelete()` e `clear()` não chegam à nuvem (ver Hooks Dexie).
- Os logotipos de clientes seguem outro caminho: `guardarLogo` em `src/pages/Clientes.tsx`. Se a imagem tiver mais de 800 px ou mais de 600 KB, é redesenhada em PNG com 800 px no máximo, para manter a transparência. Caso contrário (ou se for SVG), guarda-se o original.
- Só o dono pode apagar. Um cliente só se apaga sem trabalhos (`src/pages/Clientes.tsx`) e um fornecedor sem requisições (`src/pages/Fornecedores.tsx`).

Modelo de dados

Convenções:

- Os valores em dinheiro estão em meticais (MT).
- As datas de dia usam o formato `AAAA-MM-DD`. As marcas de tempo estão em ISO 8601 UTC.
- Os ids são texto. As ligações entre registos são ids simples, sem integridade referencial na base local nem na nuvem.

Base

A maioria dos registos de negócio estende `Base`. As excepções são `Empresa`, `Registo`, `Ficheiro` e `Nuvem`.

Campo	Tipo	Significado
`id`	`string`	Identificador único
`createdAt`	`string`	Momento da criação (ISO)
`updatedAt`	`string`	Momento da última alteração (ISO). É a versão usada na sincronização
`deleted`	`boolean?`	Registo apagado (apagamento suave)
`demo`	`boolean?`	Registo de exemplo dos primeiros protótipos. `apagarExemplos()` em `src/seed.ts` apaga estes registos só no aparelho

Há também um campo interno, `_loc`, que não aparece nas interfaces. É a marca de alteração local que os hooks escrevem, e é retirado antes de o registo ir para a nuvem.

Empresa

Registo único com id `'empresa'` (tabela `empresa`).

Campo	Tipo	Significado
`id`	`'empresa'`	Chave fixa
`nome`	`string`	Nome comercial
`razaoSocial`	`string`	Razão social
`nuit`	`string`	NUIT da empresa
`morada`, `cidade`	`string`	Morada que aparece nos documentos
`telefones`, `email`, `instagram`	`string`	Contactos que aparecem nos documentos
`taxaIva`	`number`	Taxa de IVA em %. O seed põe 16
`sinalPct`	`number`	Sinal para começar, em %. O seed põe 50
`urgenciaPct`	`number`	Taxa de urgência em %. O seed põe 45
`validadeCotacao`	`number`	Validade das cotações em dias. O seed põe 30
`notaCotacao`	`string`	Texto final que as cotações novas trazem por defeito
`updatedAt`	`string`	Versão do registo
`seedVersao`	`number?`	Versão dos dados iniciais já aplicada (`SEED_VERSAO = 3` em `src/seed.ts`). Como o registo sincroniza, vale para a empresa: um aparelho que desce a empresa da nuvem já a recebe
`caixa`	`{ valor, data, em }?`	Saldo real (banco + M-Pesa + numerário) introduzido em "Acertar caixa", em Finanças → Resumo ou no Início (`CaixaModal` em `src/pages/Resumo.tsx`). `data` é o dia do acerto e `em` o momento. Pode voltar a acertar-se

Utilizador

Campo	Tipo	Significado
`nome`	`string`	Nome
`papel`	`'dono' | 'gerente' | 'trabalhador'`	Papel, que decide o que a pessoa vê (`PERM` em `src/state.tsx`)
`funcao`	`string`	Função, em texto livre
`telefone`	`string`	Telefone
`pin`	`string`	PIN de 4 algarismos, usado para escolher quem está a usar o aparelho (ver Papéis e PIN)
`cor`	`string`	Cor do avatar
`ativo`	`boolean`	Só os utilizadores activos aparecem no ecrã de entrada

Cliente

Campo	Tipo	Significado
`nome`	`string`	Nome do cliente
`logoId`	`string?`	Ficheiro com o logotipo. Aparece na cotação, por baixo de "Exmo (s)Sr(s)"
`contacto`	`string?`	Pessoa de contacto
`telefone` , `email` , `nuit`	`string?`	Contactos e NUIT
`cidade`	`string?`	"Cidade, País" num só campo. No formulário edita-se como dois campos (`separarMorada` e `juntarMorada` em `src/format.ts`)
`notas`	`string?`	Notas

Fornecedor

Campo	Tipo	Significado
`nome`	`string`	Nome
`categoria`	`string`	O que fornece, em texto livre
`contacto` , `telefone` , `email` , `nuit` , `morada` , `site`	`string?`	Contactos
`notas`	`string?`	Notas
`logoId`	`string?`	Ficheiro com o logotipo. Aparece na lista e na ficha de fornecedores (`LogoForn` em `src/pages/Fornecedores.tsx`). O PDF da requisição não o usa: mostra uma imagem fixa da Académica quando o nome coincide e, nos outros casos, o nome (`logoFornecedor` em `src/requisicaoPdf.tsx`)

Campo	Tipo	Significado
`materiais`	`MaterialFornecedor[]`	Preçário do fornecedor
`condicao`	`'proforma' | 'credito' | 'entrega' | 'outra'?`	Forma de pagamento: "Pró-forma paga antes de produzir", "A crédito", "Paga na entrega" ou "Outra"
`temCredito`	`boolean?`	Há crédito aberto com este fornecedor
`prazoEntrega`	`string?`	Prazo de entrega habitual, em texto
`dadosBancarios`	`string?`	Dados para pagamento (contas, NIB)

MaterialFornecedor

Fica dentro de `Fornecedor.materiais` e não tem id.

Campo	Tipo	Significado
`nome`	`string`	Produto ou serviço
`tipo`	`'impressao' | 'laminacao' | 'material' | 'servico' | 'outro'`	Tipo
`precoM2`	`number`	Preço por `unidade`, sem IVA. O nome do campo vem da primeira versão, quando tudo era ao m²
`unidade`	`string?`	m², metro, unidade, rolo…
`descricao`	`string?`	Descrição

Servico

O catálogo de serviços da empresa (tabela `servicos`). Os serviços do catálogo inicial têm ids fixos `srv-<nome-normalizado>` (`idCatalogo` em `src/seed.ts`). Assim, todos os aparelhos geram o mesmo id e a sincronização não cria duplicados.

Campo	Tipo	Significado
`categoria`	`string`	Por exemplo, Logotipos ou Identidade Visual
`nome`	`string`	Nome do serviço
`preco`	`number`	Preço. 0 quer dizer "Sob proposta"
`diasMin`, `diasMax`	`number`	Prazo em dias. 0 quer dizer "Prazo por definir"
`inclui`	`string[]`	O que o serviço inclui
`ativo`	`boolean`	Serviço activo
`ordem`	`number?`	Posição no catálogo impresso. `correcoes()` acerta-a no arranque

Trabalho

Campo	Tipo	Significado
`codigo`	`string`	Código interno `T-AAAA-NNN` (`proximoCodigo` em `src/state.tsx`)

Campo	Tipo	Significado
`titulo`	`string`	Título
`clienteId`	`string`	Cliente
`tipo`	`TipoTrabalho`	`'branding' | 'criacao' | 'fornecimento' | 'montagem' | 'producao'`. O valor `'producao'` é o nome antigo de branding e é convertido no arranque (`correcoes()` em `src/seed.ts`)
`servicoId`	`string?`	Serviço do catálogo
`descricao`	`string`	Descrição
`itens`	`ItemTrabalho[]`	Partes do trabalho
`comIva`	`boolean`	O valor leva IVA
`urgente`	`boolean`	Aplica a taxa de urgência (`Empresa.urgenciaPct`)
`etapa`	`Etapa`	Fase actual
`etapas`	`PassagemEtapa[]`	Histórico das mudanças de fase
`dataEntrada`	`string`	Dia do pedido
`prazo`	`string`	Data de entrega prevista. `''` quer dizer sem data
`responsavelId`	`string?`	Utilizador responsável
`ficheiros`	`string[]`	Ids de ficheiros
`notas`	`NotaTrabalho[]`	Notas do trabalho
`cancelado`	`boolean?`	Trabalho cancelado. Deixa de contar nas dívidas e na agenda
`cotacaoId`	`string?`	Cotação em vigor
`pagamentoPrometido`	`string?`	Dia em que o cliente prometeu pagar o saldo depois da entrega
`confirmadoEm`	`string?`	O cliente confirmou (fim da negociação). A partir daqui o trabalho conta em "a receber". Também conta se já tiver um pagamento ou se tiver passado da negociação (`confirmado` e `contaParaReceber` em `src/calc.ts`)
`layoutAprovadoEm`	`string?`	Dia em que o layout foi aprovado
`pedidoFornecedorEm`	`string?`	Dia do pedido ao fornecedor, quando o pedido não foi feito por requisição na app
`montagemDe` , `montagemAte`	`string?`	Período da montagem
`prazos`	`Partial<Record<Etapa, string>>?`	Prazo opcional de cada fase

Cada tipo de trabalho segue a sua sequência de fases (`TIPOS` em `src/calc.ts`):

Tipo	Sequência
`branding`	pedido, negociacao, layout, producao, montagem, entrega, entregue
`criacao`	pedido, negociacao, layout (com o rótulo "Design"), entrega, entregue
`fornecimento`	pedido, negociacao, layout, producao (com o rótulo "Encomenda"), entrega, entregue
`montagem`	pedido, negociacao, montagem, entrega, entregue

Os trabalhos gravados antes de 16 de Outubro de 2026 podem ter nomes de fase antigos (`EtapaAntiga`). Não há migração. O `DadosProvider` converte-os na leitura com `normalizarEtapa()` (`src/calc.ts`), tanto em `etapa` como em `etapas[]` . Um trabalho só fica gravado com os nomes novos quando volta a ser guardado. Os do tipo antigo `'producao'` são reescritos pelas correcções de arranque. A conversão é:

- entrada → pedido;
- cotacao e aprovacao → negociacao;
- design → layout;
- encomenda e recebido → producao;
- marcacao e acabamento → montagem;
- pronto → entrega.

ItemTrabalho

Uma parte do trabalho. Fica dentro de `Trabalho.itens` e não tem id.

Campo	Tipo	Significado
`descricao`	`string`	Descrição da parte
`detalhe`	`string?`	Detalhe
`qt`	`number`	Quantidade
`precoUnit`	`number`	Preço unitário. 0 quer dizer "por cotar"
`origem`	`'boca' | 'cotacao' | 'catalogo'?`	De onde veio o preço: combinado de boca, cotação ou catálogo
`servicoId`	`string?`	Serviço do catálogo

PassagemEtapa e NotaTrabalho

Ficam dentro do `Trabalho` .

Interface	Campo	Tipo	Significado
`PassagemEtapa`	`etapa`	`Etapa`	Fase para onde o trabalho passou
	`em`	`string`	Quando (ISO)
	`por`	`string`	Id do utilizador
`NotaTrabalho`	`id`	`string`	Id da nota
	`em`	`string`	Quando (ISO)
	`por`	`string`	Id do utilizador

Interface	Campo	Tipo	Significado
	`texto`	`string`	Texto
	`ficheiroId`	`string?`	Ficheiro anexo

Pagamento

Dinheiro recebido de um cliente.

Campo	Tipo	Significado
`trabalhoId`	`string`	Trabalho
`data`	`string`	Dia
`valor`	`number`	Valor
`metodo`	`MetodoPagamento`	`'M-Pesa' | 'e-Mola' | 'Transferência' | 'Numerário' | 'POS' | 'Cheque'`
`tipo`	`'sinal' | 'parcial' | 'final'`	Tipo de pagamento
`referencia`	`string?`	Referência
`reciboId`	`string?`	Documento emitido (recibo ou fatura-recibo) que registou o pagamento (`documentarPagamentos` em `src/pages/Rapido.tsx`). Um pagamento sem documento aparece como "sem fatura/recibo" (`semDocumento` em `src/calc.ts`)

Requisicao

Um pedido a um fornecedor. Tem dois tipos: a lista de impressão (ao m²) e a encomenda (por unidades). O ciclo é: lista, pró-forma, pagamento ou crédito, fatura e recibo, levantamento.

Campo	Tipo	Significado
`numero`	`string`	`REQ-NNN-MÊS/AA` . A numeração recomeça cada mês (`numeroDoMes` em `src/calc.ts`)
`trabalhoId`	`string`	Trabalho. `''` quer dizer que não está ligada a um trabalho (por exemplo, "Várias" no mapa de crédito)
`fornecedorId`	`string`	Fornecedor
`data`	`string`	Dia
`tipo`	`'lista' | 'encomenda'?`	Lista de impressão ou encomenda
`nomeTrabalho`	`string`	Nome do trabalho impresso na lista
`materialImpressao` , `precoImpressaoM2`	`string` , `number`	Impressão base da lista e preço por m². Cada linha pode ter a sua
`materialLaminacao` , `precoLaminacaoM2`	`string` , `number`	Laminação base da lista e preço por m²

Campo	Tipo	Significado
`linhas`	`LinhaImpressao[]`	Prints
`itens`	`ItemEncomenda[]?`	Linhas da encomenda
`comIva`	`boolean`	Leva IVA
`estado`	`'rascunho' | 'enviada' | 'recebida'`	Estado. `'recebida'` quer dizer levantada
`anexoId`	`string?`	Ficheiro anexo
`notas`	`string?`	Notas
`enviadaEm`	`string?`	Dia em que foi enviada
`proforma`	`ProForma?`	Pró-forma do fornecedor
`pagamento`	`'pendente' | 'pago' | 'credito'?`	Situação do pagamento ao fornecedor
`pagamentos`	`PagamentoFornecedor[]?`	Pagamentos ao fornecedor. São permitidos pagamentos parciais
`creditoDesde`	`string?`	Desde quando está a crédito
`pagoEm`, `metodoPagamento`, `refPagamento`	`string?`	Dados do pagamento
`faturaDocId`, `reciboDocId`	`string?`	Documentos recebidos (fatura e recibo do fornecedor)
`levantadaEm`	`string?`	Dia do levantamento
`despesaLevantamentoId`	`string?`	Despesa com o custo do levantamento

LinhaImpressao

Campo	Tipo	Significado
`qt`	`number`	Quantidade
`ficheiro`	`string`	Nome do ficheiro do print. É texto, não um ficheiro da app
`largura`, `comprimento`	`number`	Em metros. A área é qt × largura × comprimento (`m2Linha` em `src/calc.ts`)
`impressao`, `precoImpressao`	`string?`, `number?`	Material de impressão desta linha. Se faltar, vale o da lista. `''` quer dizer nenhum
`laminacao`, `precoLaminacao`	`string?`, `number?`	Laminação desta linha, com a mesma regra

ItemEncomenda, PagamentoFornecedor e ProForma

Ficam dentro da `Requisicao`.

Interface	Campo	Tipo	Significado
`ItemEncomenda`	`descricao`	`string`	Produto
	`unidade`	`string`	Unidade
	`qt`	`number`	Quantidade
	`precoUnit`	`number`	Preço unitário
`PagamentoFornecedor`	`id`	`string`	Id do pagamento
	`data`	`string`	Dia
	`valor`	`number`	Valor
	`metodo`	`string`	Método, em texto livre
	`referencia`	`string?`	Referência
`ProForma`	`numero`	`string`	Número da pró-forma
	`data`	`string`	Dia
	`valor`	`number`	Total com IVA, tal como vem na pró-forma
	`ficheiroId`	`string?`	PDF ou foto

Custo

Os "Outros custos" de um trabalho, além das requisições.

Campo	Tipo	Significado
`trabalhoId`	`string`	Trabalho
`data`	`string`	Dia
`descricao`	`string`	Descrição
`valor`	`number`	Valor

Documento

O arquivo de faturas e recibos, emitidos e recebidos. A app nunca numera faturas: o número é o do livro de faturas ou do documento recebido (os campos dizem "Nº do seu livro", em `src/pages/Rapido.tsx`).

Campo	Tipo	Significado
`direcao`	`'emitido' | 'recebido'`	Passado ao cliente ou recebido de um fornecedor
`tipo`	`TipoDocumento`	`'fatura' | 'fatura-recibo' | 'venda-dinheiro' | 'recibo' | 'nota-credito' | 'outro'`. Contam para o IVA a fatura, a fatura-recibo, a venda a dinheiro e a nota de crédito (`contaParaIva` em `src/calc.ts`)
`numero`	`string`	Número do documento
`data`	`string`	Dia

Campo	Tipo	Significado
`entidade`	`string`	Nome do cliente ou do fornecedor
`nuit`	`string?`	NUIT da entidade
`clienteId`, `fornecedorId`	`string?`	Cliente ou fornecedor
`trabalhoId`	`string?`	Primeiro trabalho coberto
`trabalhoIds`	`string[]?`	Todos os trabalhos cobertos (uma fatura pode cobrir vários)
`base`, `iva`, `total`	`number`	Valores
`ficheiroId`	`string?`	Foto ou PDF
`notas`	`string?`	Notas
`despesaId`	`string?`	Despesa que deu origem ao documento
`requisicaoId`	`string?`	Requisição (fatura ou recibo do fornecedor)
`faturaId`	`string?`	Num recibo, a fatura que paga. Vários recibos para a mesma fatura são prestações

Cotacao

Campo	Tipo	Significado
`numero`	`string`	`COT-NNN-MÊS/AA`. A numeração recomeça cada mês (`numeroDoMes`)
`data`	`string`	Dia
`clienteId`	`string`	Cliente
`itens`	`ItemCotacao[]`	Linhas da cotação
`validadeDias`	`number`	Validade em dias. Começa com `Empresa.validadeCotacao`
`notas`	`string`	Texto final. Começa com `Empresa.notaCotacao`
`estado`	`'rascunho' | 'enviada' | 'aceite' | 'recusada'`	Estado
`trabalhoId`	`string?`	Primeiro trabalho criado ao converter a cotação
`origemTrabalhoId`	`string?`	Trabalho onde a cotação foi feita
`notaMaterial`	`boolean?`	Imprime a nota do material no rodapé. Usa-se em cotações de wrap cujos itens não têm materiais
`desconto`	`number?`	Desconto em %, antes do IVA
`opcaoAceite`	`number?`	Opção que o cliente escolheu

ItemCotacao e MaterialCotacao

Interface	Campo	Tipo	Significado
`ItemCotacao`	`qt`	`number`	Quantidade
	`titulo` , `subtitulo` , `descricao`	`string` (os dois últimos opcionais)	Textos da linha
	`precoUnit`	`number`	Preço unitário
	`materiais`	`MaterialCotacao[]?`	Materiais impressos por baixo da linha
	`nota`	`string?`	Última linha em negrito e itálico
	`opcao`	`number?`	Número da opção. Um item sem opção entra no total de todas as opções
	`secao`	`boolean?`	Linha de título de secção. Só o título é impresso
	`extra`	`boolean?`	Encargo como a deslocação. Vai com o trabalho e nunca vira trabalho próprio
	`urgente`	`boolean?`	Taxa de urgência só neste item, impressa numa linha própria a seguir
	`gerada`	`boolean?`	Linha que a app escreve (a da urgência). Nunca é gravada
`MaterialCotacao`	`nome`	`string`	Nome do material (impresso a vermelho)
	`certificado`	`boolean`	Material certificado ou não
	`garantia`	`string?`	Vida útil garantida pela marca. Vazio quer dizer sem garantias
	`marca`	`string?`	Nome da opção feita com este material (por exemplo, "OPÇÃO 1 - Taksim®")

Tarefa

Um afazer da agenda, com ou sem data, solto ou ligado a um trabalho (`novaTarefa` e `marcarTarefa` em `src/acoes.ts`).

Campo	Tipo	Significado
`texto`	`string`	Texto
`data`	`string?`	Dia
`trabalhoId`	`string?`	Trabalho
`feitaEm`	`string?`	Quando foi marcada como feita
`por`	`string`	Id do utilizador que a criou

DespesaFixa

Um custo mensal recorrente. Em cada mês aparece como pendente, a pagar ou atrasada até existir uma `Despesa` com este `fixaId` e esse `mes` (`EstadoFixa` em `src/calc.ts`).

Campo	Tipo	Significado
`nome`	`string`	Nome
`categoria`	`string`	Categoria
`valor`	`number`	Valor mensal
`dia`	`number`	Último dia para pagar
`abre`	`number?`	Primeiro dia para pagar. Se vier depois de `dia` , o prazo passa para o mês seguinte (por exemplo, de 25 a 5)
`entidade`	`string?`	A quem se paga
`utilizadorId`	`string?`	Trabalhador (na categoria Salários)
`inicio`	`string`	Mês de início, `AAAA-MM`
`ativo`	`boolean`	Despesa activa
`notas`	`string?`	Notas

Despesa

Campo	Tipo	Significado
`data`	`string`	Dia
`descricao`	`string`	Descrição
`categoria`	`string`	Categoria
`valor`	`number`	Total pago, com IVA
`iva`	`number`	IVA dedutível incluído no valor. 0 quando não há fatura
`entidade` , `metodo`	`string?`	A quem se pagou e como
`fixaId`	`string?`	Despesa fixa a que corresponde
`mes`	`string?`	Mês (`AAAA-MM`) a que a despesa fixa se refere
`ficheiroId`	`string?`	Foto ou PDF
`notas`	`string?`	Notas
`documento`	`'fatura' | 'venda-dinheiro' | 'recibo' | 'nenhum'?`	Documento que a loja entregou
`numeroDoc`	`string?`	Número desse documento
`pessoal`	`boolean?`	Gasto pessoal do dono feito com dinheiro da empresa. Sai da caixa mas não conta como custo

Campo	Tipo	Significado
`pessoalEstado`	`'reposto' | 'autorizado'?`	Como ficou resolvido: devolvido à empresa, ou aceite como uso próprio
`pessoalEm`	`string?`	Dia em que foi reposto ou autorizado

Relatorio

O relatório diário de um trabalhador (ou do próprio Momad).

Campo	Tipo	Significado
`data`	`string`	Dia
`utilizadorId`	`string`	Quem escreveu
`feito`	`string`	O que fez ("O que fiz hoje")
`aFazer`	`string?`	O que falta fazer ("O que falta / amanhã")
`problemas`	`string?`	Problemas ou o que precisa
`trabalhosIds`	`string[]`	Trabalhos referidos
`fotos`	`string[]`	Ids de ficheiros

Ficheiro

Tabela `ficheiros`. **Não vai para a nuvem.**

Campo	Tipo	Significado
`id`	`string`	Id, guardado pelos outros registos (`logoId` , `ficheiroId` , `anexoId` , `fotos` , `ficheiros`)
`nome`	`string`	Nome original
`mime`	`string`	Tipo (por exemplo, `image/jpeg` ou `application/pdf`)
`tamanho`	`number`	Bytes
`blob`	`Blob`	O conteúdo
`createdAt`	`string`	Quando foi guardado

Registo

O histórico de acções, escrito por `registar()` (por exemplo, "criou …", "moveu … para …"). Vai para a nuvem, mas nenhum ecrã da app o mostra por agora.

Campo	Tipo	Significado
`id`	`string`	Id
`em`	`string`	Quando (ISO). É a versão na sincronização
`por`	`string`	Id do utilizador
`acao`	`string`	Texto da acção

Campo	Tipo	Significado
`alvo`	`string?`	Id do registo afectado

Nuvem

Tabela `meta`, chave `'nuvem'` (é a única linha). Guarda o estado que só pertence ao aparelho e **não vai para a nuvem**.

Campo	Tipo	Significado
`id`	`'nuvem'`	Chave fixa
`token`	`string`	Token da sessão. `''` quer dizer que a sessão terminou mas o aparelho continua ligado à empresa
`empresa`	`string`	Código da empresa
`nome`	`string`	Nome da empresa na nuvem
`cursor`	`number`	Maior `seq` recebido da nuvem
`marca`	`string`	Hora (ISO) em que começou o último envio. `''` quer dizer "enviar tudo"
`esquema`	`number?`	Valor de `ESQUEMA` com que o aparelho puxou da última vez

Fora da IndexedDB, o aparelho guarda no `localStorage` só preferências:

- `pg_sessao`: utilizador escolhido no ecrã de PIN;
- `pg_tema`: tema;
- `pg_dica`: a dica de instalação foi fechada;
- `pg_vista`: vista quadro ou lista dos trabalhos;
- `pg_dev_local`: modo local, que só funciona no servidor de desenvolvimento.

Como as entidades se ligam

O trabalho está no centro:

```
Trabalho
 |-- clienteId ------------------------> Cliente
 |-- cotacaoId ------------------------> Cotacao (em vigor)
 |-- responsavelId --------------------> Utilizador
 |-- servicoId, itens[].servicoId -----> Servico
 |-- ficheiros[], notas[].ficheiroId --> Ficheiro (só no aparelho)
 |
 |<-- Cotacao.origemTrabalhoId (feita dentro do trabalho)
 |<-- Cotacao.trabalhoId (primeiro trabalho criado ao converter)
 |<-- Pagamento.trabalhoId ---- reciboId ------------> Documento (recibo ou fatura-recibo emitido)
 |<-- Requisicao.trabalhoId --- fornecedorId --------> Fornecedor
 |                          |-- faturaDocId, reciboDocId -> Documento (recebido)
 |                          '-- despesaLevantamentoId ---> Despesa
 |<-- Documento.trabalhoId / Documento.trabalhoIds[]
 |<-- Custo.trabalhoId
 |<-- Tarefa.trabalhoId
 '<-- Relatorio.trabalhosIds[]
```

Outras ligações:

```
Cliente <-- Cotacao.clienteId, Documento.clienteId
Documento (recibo) -- faturaId ------> Documento (fatura)
Documento -- despesaId --> Despesa ;  requisicaoId --> Requisicao ;  fornecedorId --> Fornecedor
Despesa -- fixaId + mes --> DespesaFixa -- utilizadorId --> Utilizador (salários)
Relatorio -- utilizadorId --> Utilizador
PassagemEtapa.por, NotaTrabalho.por, Tarefa.por, Registo.por --> Utilizador.id
Cliente.logoId, Fornecedor.logoId, Documento.ficheiroId, Despesa.ficheiroId,
ProForma.ficheiroId, Requisicao.anexoId, Relatorio.fotos[] --> Ficheiro
```

Sincronização com a nuvem

Servidor: schema `puzzle`

Criado por `db/001_puzzle_nuvem.sql`.

Tabela	Colunas	Notas
`puzzle.empresas`	`id` (chave: código da empresa, em minúsculas), `nome`, `senha_hash`, `falhas`, `bloqueado_ate`, `criado_em`	A palavra-passe só existe em hash bcrypt
`puzzle.sessoes`	`token_hash` (chave: SHA-256 do token, em hex), `empresa_id`, `aparelho`, `criado_em`, `visto_em`	`aparelho` é um texto como "iPhone · Safari", gerado por `aparelho()` em `src/nuvem.ts`
`puzzle.docs`	`empresa_id`, `tabela`, `id` (chave composta), `dados` (jsonb com o registo inteiro), `versao`, `seq`, `servidor_em`	Uma linha por registo da app. Índice `docs_empresa_seq (empresa_id, seq)`

- `sessoes` e `docs` apontam para `empresas` com `on delete cascade`: apagar uma empresa apaga as suas sessões e os seus registos.
- `seq` vem da sequência `puzzle.docs_seq`. A sequência é a mesma para todas as empresas do schema, por isso os números de uma empresa têm saltos. O que conta é que crescem sempre.
- As três tabelas têm RLS activo e nenhuma política. O schema tem `revoke all ... from public, anon, authenticated`. Pela API, ninguém lê nem escreve nestas tabelas directamente.
- As funções usam `extensions.crypt`, `extensions.gen_salt`, `extensions.digest` e `extensions.gen_random_bytes`, por isso a extensão pgcrypto tem de estar no schema `extensions`.
- As funções públicas ficam no schema `public`, que a API já expõe. O schema `puzzle` não precisa de ser exposto.
- Uma empresa cria-se à mão, por SQL. Não há função para isso (ver Administração por SQL).

Funções (RPC)

Todas são `security definer` e têm `set search_path = ''`, com referências sempre qualificadas. As cinco públicas só têm `execute` para `anon` e `authenticated`. Do lado da app, `rpc()` em `src/nuvem.ts` faz um `POST` ao endereço `API` (o caminho `/rest/v1/rpc/` do Supabase) mais o nome da função, com a chave pública nos cabeçalhos `apikey` e `Authorization`. Um erro HTTP vira `ErroNuvem` com a mensagem do servidor.

Função	Parâmetros	Devolve	O que faz
`puzzle.empresa_do_token` (interna)	`p_token`	id da empresa	Procura a sessão pelo SHA-256 do token e actualiza `visto_em`. Se a sessão não existir, lança o erro `sessao` (código 28000). `anon` e `authenticated` não a podem chamar
`puzzle_entrar`	`p_empresa`, `p_senha`, `p_aparelho`	`{token, empresa, nome}` ou `{erro: 'credenciais' | 'bloqueado'}`	Normaliza o código (`lower(trim())`), verifica o bloqueio e compara a palavra-passe com bcrypt. Numa falha, soma-a ao contador. Numa entrada certa, põe as falhas a 0, cria a sessão e devolve o token
`puzzle_puxar`	`p_token`, `p_desde` (0), `p_limite` (500)	`{docs: [{t, d}], ate}`	Devolve os registos com `seq > p_desde`, por ordem de `seq`. O limite fica sempre entre 1 e 1000. `ate` é o maior `seq` devolvido, ou `p_desde` quando não vem nada
`puzzle_enviar`	`p_token`, `p_docs`	`{aceites}`	Grava um lote (insere ou actualiza). Um registo só substitui o guardado se a sua versão for maior. Ver Envio
`puzzle_mudar_senha`	`p_token`, `p_atual`, `p_nova`	`{ok: true}` ou `{erro: 'senha' | 'curta'}`	Confirma a palavra-passe actual, exige pelo menos 8 caracteres na nova, grava o hash bcrypt (custo 10) e apaga as outras sessões da empresa
`puzzle_sair`	`p_token`	nada	Apaga a sessão deste token

Migrações

- `db/001_puzzle_nuvem.sql` cria o schema, as tabelas, os índices e as seis funções.
- `db/002_puzzle_tarefas.sql` volta a criar `puzzle_enviar` com `'tarefas'` na lista de tabelas aceites. Foi aplicada no Supabase a 16 de Outubro de 2026, como migração `puzzle_002_tarefas`. No repositório, o ficheiro 001 já mostra a lista com `'tarefas'`. Numa base nova, correr o 001 e depois o 002 dá o mesmo resultado.

Lista fixa de tabelas

Há duas listas que têm de ser iguais:

- `TABELAS` em `src/nuvem.ts`, do lado da app;
- o array `tabs` dentro de `puzzle_enviar`, do lado do servidor.

As 16 tabelas são:

- `empresa`, `utilizadores`, `clientes`, `fornecedores`;
- `servicos`, `trabalhos`, `pagamentos`, `requisicoes`;
- `custos`, `documentos`, `cotacoes`, `registos`;
- `despesasFixas`, `despesas`, `relatorios`, `tarefas`.

Ficam de fora `ficheiros` (os blobs) e `meta` (o estado do aparelho).

Versão de um registo

`versao(d)` em `src/nuvem.ts` usa o primeiro destes campos que não estiver vazio: `updatedAt`, `em` (no caso dos registos de histórico) ou `createdAt`. As versões são textos ISO 8601 UTC, todos no mesmo formato. Por isso, comparar os textos é o mesmo que comparar as datas.

A app envia a versão no campo `v` de cada elemento. O servidor guarda esse valor na coluna `versao` e não o lê de dentro do registo. O servidor e a app comparam da mesma maneira, e só aceitam uma versão **estritamente maior**.

Hooks Dexie

`instalarHooks()` em `src/nuvem.ts` é chamada em `src/main.tsx` antes de qualquer escrita, e também em `admin/carregar.ts`. Instala dois hooks em cada tabela de `TABELAS`:

- `creating`: numa escrita local, põe `_loc` igual a agora e marca uma sincronização. O `updatedAt` que vem no objecto mantém-se. É por isso que os registos do seed ficam com datas antigas.
- `updating`: numa escrita local que muda alguma coisa, marca uma sincronização e **força** `updatedAt` e `_loc` para agora. Qualquer alteração de um registo que já existe passa assim a ser a versão mais recente, mesmo que o código não tenha chamado `guardar()`.

As escritas que vêm da nuvem (`aplicar()`) correm numa transacção marcada `__remoto`. Os hooks ignoram essas escritas, por isso os registos ficam com a versão que trouxeram, sem `_loc`, e nunca são reenviados.

O que **não** sincroniza:

- não há hook `deleting`, por isso `delete()`, `bulkDelete()` e `clear()` só apagam no aparelho. No código, isto acontece:
  - em `apagarExemplos()` e `catalogoComIdsFixos()` (`src/seed.ts`);
  - em `entrar()`;
  - em `repor()`;
- as escritas nas tabelas `ficheiros` e `meta`.

Envio (`enviar`)

1. Guarda a hora de início do envio.
2. Percorre todas as tabelas de `TABELAS`. Se `marca` estiver vazia (primeiro envio do aparelho para uma nuvem vazia), junta todos os registos. Caso contrário, junta só os que têm `_loc >= marca`.
3. Retira `_loc` e calcula a versão. Um registo sem versão não segue.
4. Envia em lotes de 200 (`LOTE`) para `puzzle_enviar`.
5. No fim, `marca` passa a ser a hora de início. As alterações feitas durante o envio têm `_loc` igual ou posterior a essa hora e vão no envio seguinte.

No servidor, `puzzle_enviar` faz isto:

- recusa o lote inteiro (erro `lote invalido`) se não for uma lista ou se tiver mais de 500 elementos;
- **ignora sem erro** cada elemento com um destes problemas:
  - tabela fora da lista;
  - `id` vazio;
  - versão vazia;
  - mais de **300.000 caracteres** (o elemento `{t, v, d}` inteiro, em texto JSON);
- insere o registo, ou actualiza-o só se `versao` guardada < `versao` nova. Uma actualização aceite recebe um `seq` novo e `servidor_em = now()`;
- devolve `aceites`, mas a app não lê este número.

Descida (`puxar`)

1. Pede `puzzle_puxar` a partir de `cursor - 100`, nunca abaixo de 0. A sobreposição conta-se em números de `seq`, não em linhas. Serve para apanhar uma escrita que recebeu um `seq` mais baixo mas só ficou gravada depois de outra com `seq` mais alto.
2. `aplicar()` agrupa os registos por tabela e ignora tabelas desconhecidas e registos sem `id`. Depois, numa transacção `__remoto`, só grava os que não existem no aparelho ou que têm versão maior do que a local.
3. Se `ate` passar o cursor, o cursor avança.
4. Repete enquanto vierem páginas cheias (`PAGINA = 500`) e o cursor avançar, até 100 voltas por sincronização. O que faltar desce na sincronização seguinte.

Os registos da sobreposição voltam a descer, mas não mudam nada porque a versão é igual.

Conflitos: last-write-wins

- A unidade de conflito é o **registo inteiro**, não o campo. Se dois aparelhos alterarem o mesmo trabalho sem ligação, fica a alteração com o `updatedAt` mais recente e a outra perde-se, mesmo que tenha mexido noutro campo.
- Exemplo:
  1. O aparelho A altera o trabalho X às 10:00, sem internet.
  2. O aparelho B altera o mesmo X às 10:05 e envia.
  3. Quando A volta a ter rede, o servidor recusa a versão das 10:00.
  4. A puxa a versão das 10:05 e a alteração de A desaparece.
- As versões vêm do relógio de cada aparelho (`agora()`). O fuso horário não importa, porque tudo é UTC. Um aparelho com o relógio adiantado ganha os conflitos e um atrasado perde-os.
- Apagar é só mais uma escrita (`deleted = true`). Se outro aparelho, que ainda não viu o apagamento, editar o mesmo registo mais tarde, a sua versão é mais recente e o registo volta a aparecer.

Quando sincroniza

`iniciarSincronizacao()` (chamada em `src/main.tsx`) sincroniza:

- ao abrir a app;
- de 30 em 30 segundos;
- quando volta a rede (evento `online`);
- quando a app volta a ficar visível (`visibilitychange`);
- 1,5 segundos depois da última escrita local (`agendar()` , chamada pelos hooks; o temporizador recomeça a cada escrita);
- quando o utilizador toca no estado da nuvem (barra lateral) ou em "Sincronizar agora" (Definições, cartão Nuvem).

O ecrã `Ligar` também chama `sincronizar()` logo depois de entrar.

`sincronizar()` corre uma vez de cada vez. Se for chamada durante uma corrida, repete 300 ms depois de ela acabar. Cada corrida faz isto:

1. Lê `meta.nuvem`. Sem token, a fase fica `desligado`. Sem rede (`navigator.onLine === false`), fica `offline` e não tenta nada.
2. Se `esquema < ESQUEMA`, põe o cursor a 0. Ver ESQUEMA.
3. Envia, puxa e só no fim grava `meta.nuvem` (cursor, marca, esquema). Se falhar a meio, a corrida seguinte repete o trabalho. Isto é seguro porque as versões impedem que uma escrita velha se sobreponha.
4. Se o erro for `sessao`, ver Sair, sessão terminada e desligar. Os outros erros deixam a fase em `erro` (ou em `offline`, se a rede caiu) com a mensagem.

O texto de cada fase está em `textoNuvem()` (`src/App.tsx`):

Fase	Texto
`parado`	"Na nuvem · HH:MM" (só "Na nuvem" antes da primeira sincronização)
`a-sincronizar`	"A sincronizar…"
`offline`	"Sem internet · guardado no aparelho"
`erro`	"Erro ao sincronizar · toque para tentar"
`desligado`	"Não ligado à nuvem"

ESQUEMA: forçar uma nova descida

`ESQUEMA = 2` em `src/nuvem.ts` . Uma versão antiga da app ignora, ao puxar, os registos de tabelas que não conhece, mas o cursor avança na mesma. Por isso, quando se acrescenta uma tabela, sobe-se `ESQUEMA` .

Cada aparelho com `esquema` menor põe o cursor a 0 uma vez e volta a descer tudo. Só as versões maiores são aplicadas, por isso nada se estraga. O valor passou a 2 quando entrou a tabela `tarefas` .

Tabela nova: passos obrigatórios

1. Acrescentar `this.version(6).stores({ ... })` em `src/db.ts` , com a interface nova.
2. Acrescentar a tabela a `TABELAS` em `src/nuvem.ts` . Os hooks, o envio, `aplicar()` e `entrar()` usam esta lista.
3. Criar `db/003_...sql` , que volta a criar `puzzle_enviar` com a tabela no array `tabs` , e aplicá-lo no Supabase.
4. Subir `ESQUEMA` em `src/nuvem.ts` .
5. Se a interface precisar dos dados, carregar a tabela no `DadosProvider` (`src/state.tsx`). Ponderar também juntá-la à cópia de segurança (`exportar()` em `src/pages/Definicoes.tsx`).

A ordem importa: **aplicar a migração no servidor antes de publicar a app**. Se a app chegar primeiro, o servidor ignora em silêncio os registos da tabela nova e a `marca` avança. Esses registos só voltam a ser enviados quando forem alterados.

Um campo novo e opcional num registo existente não precisa de migração, porque o registo viaja inteiro em JSON.

Ligar um aparelho (`entrar`)

O ecrã `Ligar` (`src/App.tsx`) aparece sempre que o aparelho não tem token (fora do modo local de desenvolvimento). Pede só a palavra-passe. O código da empresa já vem preenchido e só se mostra com o botão "Outra empresa". Depois de entrar, o ecrã chama `semear()` e `sincronizar()` .

Situação	O que acontece
Aparelho novo (sem `meta.nuvem`) e nuvem vazia	Cria `meta` com cursor 0, `marca` vazia e `esquema` actual. Não desce nada. O `semear()` cria a empresa, o catálogo e o primeiro utilizador. A sincronização seguinte envia tudo o que está no aparelho, que passa a ser a primeira cópia
Aparelho novo e nuvem com dados	A app pergunta com `puzzle_puxar` (limite 1) se a nuvem tem dados. Se tiver, **apaga no aparelho as 16 tabelas** numa transacção, põe `marca` igual a agora e desce tudo. Os registos antigos do aparelho, criados antes da nuvem com ids próprios, desaparecem para não haver duplicados. Os ficheiros (blobs) ficam, porque os registos da nuvem podem apontar para eles. O ecrã avisa: "Este aparelho fica com os dados da empresa que estão na nuvem."

Situação	O que acontece
Aparelho que já esteve ligado (`meta` com token vazio)	Mantém o cursor e a marca e grava o token novo. Primeiro **envia** o que mudou no aparelho e só depois desce. O ecrã avisa: "O que mudou neste aparelho sobe para a nuvem quando entrar."

Se a entrada falhar, o ecrã mostra "Código da empresa ou palavra-passe errados." ou "Demasiadas tentativas. Tente outra vez daqui a 10 minutos." (`entrar()` em `src/nuvem.ts`).

No arranque (`arrancar()` em `src/main.tsx`), `semear()` só corre se o aparelho já tiver o registo `empresa` ou estiver no modo local de desenvolvimento. Um aparelho novo não cria dados antes de se ligar.

Sair, sessão terminada e desligar

Acção	Onde	Efeito
Sair, com um só utilizador activo	Botão "Sair" ou cartão do utilizador	`terminarSessao()` : sincroniza, chama `puzzle_sair` e grava o token como `''` (a `meta` fica). Volta ao ecrã da palavra-passe e os dados locais ficam
Sair, com vários utilizadores activos	Mesmo sítio	Só limpa `pg_sessao` e volta ao ecrã de PIN. A sessão na nuvem continua
Sessão terminada pelo servidor	Automático	Uma chamada recebe `sessao` (palavra-passe mudada noutro aparelho, ou sessão apagada). O token fica `''` , os cursores ficam, aparece "A sessão terminou. Entre outra vez." e, ao entrar, as alterações pendentes sobem
Mudar palavra-passe	Definições, cartão Nuvem	`puzzle_mudar_senha` . As outras sessões são apagadas e os outros aparelhos voltam ao ecrã da palavra-passe na sincronização seguinte
Desligar este aparelho	Definições, cartão Nuvem	`desligar()` : sincroniza, chama `puzzle_sair` e **apaga** `meta` . Os dados locais ficam, mas na próxima ligação o aparelho é tratado como novo (ver a tabela anterior)
Recomeçar este aparelho	Definições, cartão Dados	`desligar()` seguido de `repor()` : apaga a base local inteira, incluindo os ficheiros, e recarrega

O cartão Nuvem fica em Definições → Empresa, que só o dono vê (acção `definicoes`).

Limites

Limite	Valor	Onde
Tamanho de um registo enviado	300.000 caracteres (JSON do elemento `{t, v, d}`)	`puzzle_enviar`
Elementos por chamada a `puzzle_enviar`	500 no servidor; a app envia 200	`puzzle_enviar` , `LOTE`
Registos por página ao puxar	a app pede 500; o servidor aceita entre 1 e 1000	`PAGINA` , `puzzle_puxar`
Páginas por sincronização	100	`puxar()`
Sobreposição do cursor	100 números de `seq`	`puxar()`

Limite	Valor	Onde
Texto do aparelho na sessão	160 caracteres	`puzzle_entrar`

Cuidados conhecidos

- **Registos ignorados em silêncio.** Um registo com mais de 300.000 caracteres, ou de uma tabela que o servidor não aceita, não sobe e a app não dá erro. Como a `marca` avança, só volta a ser tentado depois de outra alteração.
- **Registo inteiro e relógios.** Ver Conflitos. Convém que os aparelhos tenham a hora certa automática.
- **Números internos.** `T-AAAA-NNN`, `COT-NNN-MÊS/AA` e `REQ-NNN-MÊS/AA` são calculados no aparelho a partir dos registos que ele conhece. Dois aparelhos sem ligação podem criar o mesmo número. Os ids nunca colidem.
- **Desligar sem internet.** `desligar()` tenta sincronizar, mas sem rede não envia nada e apaga `meta` na mesma. Se depois o aparelho se ligar a uma nuvem com dados, as tabelas locais são apagadas e as alterações pendentes perdem-se. Desligar só com internet.
- **Mudar de empresa no mesmo aparelho.** Os dados locais não estão separados por empresa. Há dois casos:
  - entrar noutra empresa ("Outra empresa") com a `meta` antiga mantém o cursor e a marca da primeira, e envia para a nova as alterações pendentes;
  - depois de "Desligar", uma empresa vazia recebe todos os dados do aparelho como primeira cópia.

  Antes de trocar de empresa, usar "Recomeçar este aparelho".
- **`SEED_VERSAO`.** Subir este valor em `src/seed.ts` faz correr outra vez `apagarExemplos()`, `semearReal()` e `renumerar()`. `renumerar()` refaz, por ordem de data, os códigos dos trabalhos e os números das requisições e cotações no formato `T-/REQ-/COT-AAAA-NNN`, e essas escritas sincronizam. Isto mudaria códigos de trabalhos e estragaria a numeração mensal das cotações e das requisições. Não subir sem rever `renumerar()`.
- **Correcções de arranque.** `correcoes()` (`src/seed.ts`) corre sempre que `semear()` corre: ordem do catálogo, ids fixos do catálogo e conversão de `producao` para `branding`. São escritas normais e sincronizam, excepto o apagamento dos ids antigos do catálogo, que é só local.
- **Apagar a app instalada.** No iPhone e no iPad, remover a app do ecrã principal apaga a base local dessa app, incluindo os ficheiros que só existem lá. Limpar os dados do site no navegador tem o mesmo efeito. Para actualizar a app usa-se "Atualizar app" (`atualizarApp()` em `src/versao.ts`), que remove o service worker e as caches sem tocar na IndexedDB.
- **Textos desactualizados.** O indicador no topo das páginas (`Page` em `src/App.tsx`) e o cartão Dados em Definições ainda descrevem o protótipo sem nuvem ("guardado no aparelho", "Na versão 1 passam a sincronizar").

Segurança

Onde está cada segredo

Elemento	Onde fica	Notas
Chave pública (anon) do Supabase	Constante `KEY` em `src/nuvem.ts`, dentro do código publicado	É pública por desenho. Não deve ser tratada como segredo
Endereço da API	Constante `API` em `src/nuvem.ts`	Público

Elemento	Onde fica	Notas
Palavra-passe da empresa	Só como hash bcrypt em `puzzle.empresas.senha_hash`	Nunca no código nem no repositório
Token de sessão	No aparelho, em IndexedDB (`meta.nuvem.token`). No servidor, só o SHA-256	Dá acesso a todos os dados da empresa enquanto a sessão existir
PIN de cada utilizador	Campo `pin` do registo `Utilizador` , em texto simples, no aparelho e na nuvem	Não protege dados (ver abaixo)
Palavra-passe para o carregador	Variável de ambiente `PUZZLE_SENHA` , só durante a execução	Nunca em ficheiro

Chave pública

A chave anon vai dentro do JavaScript que qualquer pessoa descarrega, e isso está previsto. No que diz respeito a esta app, a chave só permite chamar as cinco funções `puzzle_*` . As tabelas do schema `puzzle` estão fechadas pelo `revoke` do schema e pelo RLS sem políticas.

Sem um token válido, `puzzle_puxar` , `puzzle_enviar` e `puzzle_mudar_senha` devolvem o erro `sessao` . A protecção real é a palavra-passe da empresa, que dá o token, e o próprio token.

Palavra-passe da empresa

- Fica guardada só em bcrypt (`extensions.crypt` com `gen_salt('bf', 10)`).
- Um código de empresa errado e uma palavra-passe errada dão a mesma resposta (`credenciais`). Só uma empresa que existe pode responder `bloqueado` .
- **Bloqueio:** à 5.ª falha, as entradas ficam bloqueadas 10 minutos. O contador `falhas` só volta a zero com uma entrada certa. Por isso, depois do primeiro bloqueio, cada nova tentativa errada bloqueia mais 10 minutos.
- O bloqueio é por empresa, não por aparelho. Tentativas erradas vindas de qualquer lado impedem novas entradas para todos. Os aparelhos que já estão ligados continuam a sincronizar, porque usam o token.
- Mudar a palavra-passe exige uma sessão válida e a palavra-passe actual. A nova tem de ter pelo menos 8 caracteres. Todas as outras sessões terminam. Na mudança não há limite de tentativas, mas é preciso ter um token.
- Não há "esqueci a palavra-passe". Repor faz-se por SQL.

Token de sessão

- São 32 bytes aleatórios (`gen_random_bytes`), entregues em hex. O servidor guarda só o SHA-256.
- No aparelho fica na IndexedDB (`meta`), não no `localStorage` .
- Não expira com o tempo. `visto_em` é actualizado em cada chamada que valida o token, mas só serve para informação. A sessão acaba de uma destas formas:
  - com `puzzle_sair` ;
  - com uma mudança de palavra-passe noutro aparelho;
  - quando é apagada por SQL.
- Quem tiver acesso ao navegador de um aparelho ligado tem acesso aos dados. Num aparelho com um só utilizador, "Sair" apaga o token.

Papéis e PIN

As permissões estão em `PERM` (`src/state.tsx`). Aplicam-se nos menus (`useAreas` em `src/App.tsx`), no ecrã "Sem acesso" e dentro das páginas através de `pode()` : botões de apagar, valores em dinheiro e trabalhos de

outras pessoas.

Papel	Acções permitidas
`dono`	`financas`, `iva`, `despesas`, `equipa`, `definicoes`, `apagar`, `todosTrabalhos`, `editarTrabalho`
`gerente`	`financas`, `todosTrabalhos`, `editarTrabalho`
`trabalhador`	nenhuma (vê Trabalhos, Agenda e Relatórios, e só os trabalhos de que é responsável)

- **As permissões só existem na interface.** A nuvem tem uma palavra-passe por empresa e não sabe quem é o utilizador. Qualquer aparelho ligado desce todos os registos da empresa, incluindo despesas, salários e gastos pessoais, e pode escrever em qualquer tabela.
- O PIN tem 4 algarismos e é único entre os utilizadores (`src/pages/Equipa.tsx`). É comparado no navegador com `Utilizador.pin` (`Login` em `src/App.tsx`) e não tem limite de tentativas.
- Com um só utilizador activo, a app não pede PIN (`unico` em `src/App.tsx`).
- Quando o PIN ainda é o inicial, o ecrã de PIN mostra um aviso com o próprio PIN e pede para o mudar em Definições → Equipa.
- O PIN serve para escolher quem está a usar o aparelho, não para proteger dados. Antes de entregar a app a trabalhadores nos seus próprios telemóveis, convém pôr as permissões no servidor (por exemplo, sessões por utilizador e filtros nas funções `puzzle_*`).

Ficheiros só no aparelho

- Os blobs (`ficheiros`) não sobem para a nuvem e não entram na cópia de segurança JSON. O cartão Nuvem diz isso ao utilizador: "Os ficheiros (PDF e fotos) ainda ficam só no aparelho onde foram carregados."
- Noutro aparelho, o registo aponta para um id que não existe lá:
  - a miniatura não aparece (`Thumb` em `src/ui.tsx` não desenha nada);
  - o visualizador mostra "Ficheiro não encontrado" (`src/Visualizador.tsx`);
  - o logotipo do cliente não sai na cotação (`logoDoCliente` em `src/cotacaoPdf.tsx`).
- Perdem-se com "Recomeçar este aparelho", com a remoção da app instalada ou com a limpeza dos dados do site. Ligar o aparelho a uma nuvem com dados não os apaga.
- Os ficheiros na nuvem são uma das decisões em aberto com o Momad (ver Alterações de dados previstas).

Cópia de segurança JSON

O botão "Descarregar cópia de segurança" fica em Definições, cartão Dados (`exportar()` em `src/pages/Definicoes.tsx`). Só o dono o vê.

- Gera o ficheiro `puzzle-gestao-copia-AAAA-MM-DD.json` , com o campo `exportado` (ISO) e o conteúdo completo de 11 tabelas:
  - `utilizadores` , `clientes` , `fornecedores` , `servicos` ;
  - `trabalhos` , `pagamentos` , `requisicoes` , `custos` ;
  - `documentos` , `cotacoes` , `empresa` .
- Inclui os registos apagados e o campo interno `_loc` .
- **Não inclui** `despesasFixas` , `despesas` , `relatorios` , `tarefas` , `registos` nem os ficheiros.
- A app não tem função para restaurar esta cópia.
- O ficheiro tem os PINs dos utilizadores e os dados financeiros. Deve ser tratado como confidencial.

A cópia completa das 16 tabelas é a que está na nuvem (`puzzle.docs`).

Dados reais fora do repositório

- O código só traz dados de referência:
  - em `src/seed.ts`, os dados da empresa (`EMPRESA`), o catálogo de serviços (`CATALOGO`) e o primeiro utilizador;
  - em `src/requisicaoPdf.tsx`, o preçário 2026 da Académica (`PRECARIO_ACADEMICA_2026`).
- Os clientes, trabalhos, documentos e dívidas reais vivem na nuvem, não no código.
- O repositório **não inclui**:
  - `admin/dados`, com os scripts que carregaram os dados reais;
  - `admin/ficheiros`;
  - os documentos reais da Académica em `test/fixtures`.
  - `admin/referencia-academica.ts` (preçário e dados bancários do fornecedor; a app não o usa).

  Foram retirados por privacidade. Os testes que lêem PDFs reais também não estão aqui; em `src/leitura.test.ts` fica o teste de leitura de números.

Carregador `admin/carregar.ts`

O carregador põe lotes de dados reais directamente na nuvem, sem os meter no código publicado. Funciona como mais um aparelho:

1. Usa uma IndexedDB em memória (`fake-indexeddb/auto`).
2. Desvia os pedidos a caminhos que começam por `/` para a pasta `public/`, como faria o servidor de desenvolvimento. O seed actual já não faz estes pedidos, por isso o desvio só serve a scripts que precisem dele.
3. Desliga a tabela `ficheiros`: `put` devolve só o id e `get` não devolve nada. Um script que chame `guardarFicheiro()` recebe um id, mas o ficheiro não fica guardado em lado nenhum.
4. Lê `PUZZLE_SENHA`. Se faltar, pára com "Falta PUZZLE_SENHA".
5. Instala os hooks e entra com o código da empresa que está fixo no script. Desce a nuvem inteira e mostra a contagem por tabela ("nuvem antes").
6. Corre `semear()` e depois o `export default async function` do script de dados, se lhe passarem um.
7. Lista o que vai enviar ("a enviar: N", com até 80 linhas `tabela/id`, e "(apagado)" nos apagados).
8. Sem `--seco`, sincroniza e mostra o estado final.
9. No fim, mesmo que o script falhe, chama `desligar()` e fecha a sessão. Com `--seco` não sincroniza antes de sair.

Como correr: o `vite-node` vem com o `vitest` (devDependencies). A palavra-passe é a actual da empresa e só existe na variável de ambiente.

```
# bash: pedir a palavra-passe sem a mostrar nem a deixar no histórico
read -rs PUZZLE_SENHA && export PUZZLE_SENHA
npx vite-node admin/carregar.ts admin/dados/AAAA-MM-DD-assunto.ts --seco   # ensaio, não envia
npx vite-node admin/carregar.ts admin/dados/AAAA-MM-DD-assunto.ts          # envia
unset PUZZLE_SENHA
```

```
# PowerShell (Windows)
$env:PUZZLE_SENHA = '...'; npx vite-node admin/carregar.ts admin/dados/AAAA-MM-DD-assunto.ts --seco
Remove-Item Env:PUZZLE_SENHA
```

Os scripts de dados não estão no repositório. Seguem este formato:

```
// admin/dados/AAAA-MM-DD-assunto.ts
import { db, guardar } from '../../src/db'

export default async function () {
  const t = await db.trabalhos.get('<id do trabalho>')
  if (t) await guardar(db.trabalhos, { ...t, prazo: '2026-10-31' })
}
```

Um script que só lê, por exemplo um relatório de dívidas, corre com `--seco`.

Administração por SQL

Estes comandos correm no editor SQL do Supabase, com permissões de administrador. Os valores entre `< >` substituem-se. Nunca se guardam palavras-passe em ficheiros.

```
-- criar uma empresa (código em minúsculas; palavra-passe com 8 caracteres ou mais)
insert into puzzle.empresas (id, nome, senha_hash)
values ('<codigo>', '<Nome>', extensions.crypt('<palavra-passe>', extensions.gen_salt('bf', 10)));

-- repor a palavra-passe (não termina as sessões; ver o comando seguinte)
update puzzle.empresas
   set senha_hash = extensions.crypt('<nova>', extensions.gen_salt('bf', 10))
 where id = '<codigo>';

-- obrigar todos os aparelhos a entrar outra vez
delete from puzzle.sessoes where empresa_id = '<codigo>';

-- ver os aparelhos ligados
select aparelho, criado_em, visto_em from puzzle.sessoes where empresa_id = '<codigo>' order by
visto_em desc;

-- levantar o bloqueio por tentativas erradas
update puzzle.empresas set falhas = 0, bloqueado_ate = null where id = '<codigo>';
```

Projecto Supabase

O schema `puzzle` vive no projecto Supabase `kb-academy`, da KingBalas, partilhado com outros sistemas (comentários no topo de `src/nuvem.ts` e de `db/001_puzzle_nuvem.sql`). Ficou aí por decisão de 15 de Outubro de 2026 e pode passar mais tarde para uma conta da própria empresa. Os dados da empresa ficam separados no schema `puzzle` e não há acesso directo a eles pela API.

Procedimento previsto para a mudança (ainda não feito):

1. Correr `db/001` e `db/002` no projecto novo.
2. Copiar as linhas de `puzzle.empresas` e `puzzle.docs`. Há duas formas de tratar o `seq`:
   - copiar sem a coluna `seq`, para a sequência nova os numerar;
   - copiá-la e acertar `puzzle.docs_seq` acima do maior `seq` copiado.
3. Mudar `API` e `KEY` em `src/nuvem.ts`.
4. Subir `ESQUEMA`, porque os cursores antigos não valem no projecto novo.
5. Publicar a app. Os aparelhos terão de entrar outra vez. As alterações pendentes sobem ao entrar.

Alterações de dados previstas (não implementadas)

O plano para os 5 pontos em aberto com o Momad propõe só campos opcionais. Não há tabela nova, `ESQUEMA` fica em 2 e não há `db.version(6)`. Os registos que já estão na nuvem lêem-se sem migração. **Nada disto está no código.** Depende das respostas do Momad.

Proposta	Campos novos (todos opcionais)
Fatura do fornecedor sem pró-forma e comprovativo de pagamento	`Requisicao.fatura` (`{numero, data, valor}`), `PagamentoFornecedor.ficheiroId` . Uma fase "Fatura por pagar", calculada em `src/calc.ts`
Cotação enviada e negociada (versões)	`Cotacao.enviadaEm` , `aceiteEm` , `recusadaEm` , `substituiId` , `substituidaPorId` . Um estado novo, `'substituida'`
Requisição com envio registado	`LinhaImpressao.id` , `Requisicao.enviadaVia` , `Requisicao.enviadaPor`
Partes com identidade	`ItemTrabalho.id` , `estado` , `semCusto` , `cotacaoItemId` ; `ItemCotacao.id` , `semCusto` ; `Requisicao.itemIds` ; `Custo.itemId`
Pagamento por parte	`Pagamento.itemId`

Perguntas ao Momad que mexem com dados e nuvem:

- Os ficheiros (faturas, recibos, comprovativos) ficam só no aparelho por agora, ou a nuvem de ficheiros entra neste plano?
- Quando a fatura do fornecedor tiver um valor diferente da lista, o custo e o lucro do trabalho passam a usar a fatura (sem IVA)?
- Uma cotação negociada leva número novo, ou mantém o número com "v2"?
- Os lotes de um cliente com vários camiões continuam a ser trabalhos separados, ou passam a ser partes de um só trabalho?
- Basta marcar cada pagamento como sendo de uma só parte, ou é preciso repartir um pagamento entre várias partes?
- Uma requisição "Várias" (uma lista do fornecedor para vários trabalhos) continua só no Mapa de crédito, ou passa a ligar-se a mais de um trabalho?

CAPÍTULO 5

Documentos PDF, catálogos e cálculos

Este capítulo explica como a aplicação produz os documentos que saem para fora: a cotação, a requisição de prints e a relação do IVA. Explica também como envia os catálogos da Puzzle Stickers e como faz as contas. Todas as contas de dinheiro estão em `src/calc.ts`. Os testes que fixam os números estão no fim do capítulo.

Visão geral

Documento	Ficheiros	Como é desenhado	Como sai
Cotação	`src/cotacaoPdf.tsx`, `src/print.tsx`, `src/pages/cotacaoFormulario.tsx`, `src/materiaisCotacao.ts`	Página A4 em SVG, medidas em pontos	Diálogo de impressão do sistema ("Guardar em PDF" / Partilhar)
Requisição de prints	`src/requisicaoPdf.tsx`, `src/pdfPartilha.tsx`	Página A4 em HTML, medidas em mm	Ficheiro PDF gerado no próprio aparelho, na janela "PDF pronto"
Relação do IVA	`src/print.tsx` (botões em `src/pages/Iva.tsx`)	Página A4 em HTML	Diálogo de impressão (botão "PDF") e ficheiro CSV (botão "Excel")
Catálogos	`src/pages/Catalogos.tsx`, `public/catalogos/`	PDFs já prontos	Partilha do ficheiro, link por WhatsApp, ver, copiar link

Regras comuns:

- A aplicação **nunca emite faturas**. Cotações e requisições têm numeração interna (`numeroDoMes`). As faturas e os recibos continuam com o número do livro do Momad.
- A cotação imprime sempre "**Não serve de comprovativo fiscal".
- Os valores saem no formato dos documentos da empresa, `56.000,00 MT` (função `mt` em `src/format.ts`). As datas dos documentos saem como "14 de Outubro de 2026" (`dataDoc`).
- A cotação e a requisição usam o Gotham (família `Gotham PS`). São 8 ficheiros `gotham-*.woff` (light, book, medium e bold, cada um normal e itálico) que têm de estar em `public/fonts/`. Não estão neste repositório por serem de uma fonte comercial: sem eles, o navegador usa outra fonte e os PDFs perdem o aspecto original. Os `@font-face` estão em `src/styles.css`, com `font-display: block`. A relação do IVA usa Montserrat.

Cotação

Ficheiros e responsabilidades

Ficheiro	O que faz
`src/cotacaoPdf.tsx`	Desenha a página (`PaginaCotacao`), pagina (`paginar` , `paginasCotacao`), monta o bloco de totais (`blocoTotais`), mede texto (`medirCanvas`), carrega as fontes (`carregarFontes`) e o logotipo do cliente (`logoDoCliente`)
`src/print.tsx`	`imprimirCotacao` : prepara imagens e fontes, desenha todas as páginas em `#print-root` e chama `window.print()`
`src/pages/cotacaoFormulario.tsx`	Formulário da cotação (`EditorCotacao`), gerador de wrap/branding (`NovoImpresso`), linhas do catálogo de criação (`CatalogoCriacao`), Preview ao vivo (`PreviewCotacao`)
`src/materiaisCotacao.ts`	Textos fixos do Momad: materiais, notas de item, textos finais, serviços impressos, partes da viatura, tabela de deslocação
`src/pages/Cotacoes.tsx`	Lista e página da cotação: botões Preview, PDF e Guardar, aceitação, conversão em trabalhos

A página A4 em SVG

A página foi reconstruída a partir do ficheiro original da cotação da Puzzle Stickers (Illustrator, 14 de Outubro de 2026). Todas as posições foram medidas nesse ficheiro.

- Tamanho: `A4 = { w: 595.276, h: 841.89 }` , em pontos. O SVG usa `viewBox="0 0 595.276 841.89"` .
- Cada texto é um `<text>` desenhado pelo componente interno `T` , com o tamanho, o peso, o itálico, a cor e a linha de base do original.
- O Illustrator do Momad aperta o texto mais do que a fonte pura. Por isso:
  - vários estilos têm um `ls` (letter-spacing) próprio, por exemplo preços −0,42, descrições −0,24 e nome do cliente −0,58;
  - alguns textos fixos usam `textLength` (largura medida no original) com `lengthAdjust="spacing"` ;
  - os textos justificados esticam os espaços entre palavras (`wordSpacing`), nunca as letras.

Estilos principais (objecto `E`)

Estilo	Peso / itálico / tamanho / ls	Cor	Uso
`med14`	500, 14	preto	"Cotação"
`numero`	400, 10	#555555	número COT- ao lado de "Cotação"
`medIt9`	500 itálico, 9	#333333	"**Não serve de comprovativo fiscal"
`lightIt9`	300 itálico, 9, ls −0,21	#333333	"Válida por N dias · página p/t"
`med13`	500, 13	preto	data
`cliente12`	700, 12, ls −0,58	preto	nome do cliente
`book12` / `bold12`	400 / 700, 12	preto	linha 1 / linha 2 do item

Estilo	Peso / itálico / tamanho / ls	Cor	Uso
`preco12`	500, 12, ls −0,42	preto	valor unitário e total do item; valor da última linha de totais
`precoBook12`	400, 12, ls −0,43	preto	valores das outras linhas de totais
`bold13` / `precoBold13`	700, 13 / 700, 13, ls −0,51	preto	rótulo e valor do TOTAL
`descPrimeira`	500 itálico, 10, ls −0,24	#333333	primeira linha da descrição
`descLinha`	300 itálico, 10, ls −0,24	#333333	restantes linhas
`descDestaque`	700 itálico, 10, ls −0,24	#666666	linhas "** Layout..." e nota do item
`descMaterial`	400 itálico, 10, ls −0,24	#ff0000	nome do material
`descSufixo`	400 itálico, 10, ls −0,24	#666666	"- Certificado" / "- Não Certificado"
`notaJust`	300 itálico, 10, ls −0,24	#666666	texto final justificado
`garantia`	300 itálico, 10	#1a1a1a	nota das garantias

Medir e quebrar texto

- `Medir` é o tipo `(texto, estilo) => largura`. A aplicação usa `medirCanvas`: mede com o `measureText` de um canvas 2D (reutilizado) e soma `ls × número de caracteres`.
- As fontes têm de estar carregadas antes de medir. `carregarFontes()` carrega as 8 variantes (4 pesos, normal e itálico) com `document.fonts.load`. O Preview e a impressão esperam por ela.
- `quebrar(texto, estilo, largura, medir)` parte o texto por parágrafos (`\n`) e depois por palavras. Ignora os parágrafos vazios.
- A largura útil das descrições é `LARGURA_DESC = COL.totX1 − 30,39 = 564,9 − 30,39 = 534,51 pt`.
- Os textos copiados do original usam a ligadura "ﬁ" (U+FB01), como no ficheiro dele. Para comparar textos, a função interna `normal()` troca "ﬁ" por "fi", junta espaços e remove hífenes de fim de linha.

Imagens de fundo

Ficheiro	Quando é usado	Conteúdo
`public/img/cotacao-fundo.jpg`	Última página com só duas linhas de totais (SUB-TOTAL e IVA), ou seja, sem desconto. No código é `original = ultima && n === 2`	A arte original do Momad, sem texto, com as caixas de totais e a linha amarela
`public/img/cotacao-fundo-livre.jpg`	Todas as outras páginas: as de continuação e a última página com desconto	O mesmo fundo sem as caixas de totais
`public/img/cotacao-barra-total.jpg`	Só na última página com fundo livre (com desconto)	A barra amarela do TOTAL, recortada do original, a toda a largura, em y = 770,887, com 45,808 de altura
`public/img/logo.svg`	Todas as páginas	Logotipo em (343,76; 31,18), 218,92 × 51,17

`public/img/cot-pattern.jpg` serviu de apoio para reconstruir o fundo livre. O código não a usa.

Com o fundo original, as caixas e a linha amarela já vêm na imagem e a aplicação só escreve os textos. Com o fundo livre, a aplicação desenha as caixas e a linha amarela e coloca a barra do TOTAL.

Cabeçalho

Elemento	Posição (pt)	Notas
"Cotação" + número	x 30,39, y 43,08	O número (`Cotacao.numero`) fica 7 pt à direita da palavra. Só sai se existir
"**Não serve de comprovativo fiscal"	y 57,07	Fixo
"Válida por N dias"	y 69,69	N = `validadeDias` . Acrescenta " · página p/t" quando há mais de uma página
Data	y 91,77	`dataDoc(c.data)`
"Exmo (s)Sr(s):" + nome do cliente	y 114,54 (nome em x 118,08)	Nome em negrito
Logotipo do cliente	(30,9; 116,4), 83,4 × 32,2	Só quando o cliente tem `logoId`
Cidade do cliente	y 158,73 com logotipo, 130 sem	`Cliente.cidade` ("Cidade, País")
Contactos da empresa	alinhados à direita em x 550,5, a partir de y 104,05, de 13,4 em 13,4	`telefones` , `email` , `instagram` , `cidade` da `Empresa`
Cabeçalho da tabela	y 196,5	"Qt.", "Descrição", "Valor Unitário", "Valor Total"

O logotipo do cliente é um ficheiro guardado no aparelho (`db.ficheiros`). `logoDoCliente(cliente)` devolve um URL temporário para ele. Os ficheiros **não** sincronizam com a nuvem (a tabela `ficheiros` não está em `TABELAS` de `src/nuvem.ts`). Por isso o logotipo só aparece no aparelho onde foi carregado.

Tabela de itens

Colunas (`COL` , em pontos):

Coluna	De	Até
Qt.	30,4	64,0
Descrição	67,9	348,9
Valor unitário	352,8	456,9
Valor total	460,8	564,9

Cada item (`ItemCotacao`) desenha:

- quatro caixas cinzentas (#f2f2f2, contorno #9d9d9c) com 39,4 pt de altura;
- a quantidade com dois dígitos ("01"), centrada em x 46,2;
- a linha 1 (`titulo`) em Book 12 e, por baixo, a linha 2 (`subtitulo`) em negrito;
- quando o item pertence a uma opção, o título "OPÇÃO n - título" (as linhas geradas pela aplicação não levam este prefixo);
- os valores unitário e total (`qt × precoUnit`), alinhados à direita;
- por baixo das caixas, as linhas de descrição, a partir de y + 53,3, de 12 em 12 pt.

As linhas por baixo do item (função `linhasDoItem`) saem por esta ordem:

1. **Descrição** (`descricao`), quebrada em linhas. Regras de estilo (`estiloLinhaDesc`):
   - uma linha que começa por "** layout" sai em negrito itálico cinzento;
   - a primeira linha sai em Medium itálico, excepto se começar por "·", "-" ou "–" (as listas ficam todas em Light);
   - as restantes saem em Light itálico.
2. **Materiais** (`materiais`), um por linha:
   - o nome a vermelho, sublinhado a vermelho (0,5 pt);
   - a seguir, em cinzento, "- Certificado", "- Não Certificado", ou "- Não Certificado (Sem garantias)" quando não é certificado e não tem garantia;
   - se o material tem `garantia`, uma linha extra: "(A marca garante vida útil {garantia} se usado e conservado dentro dos padrões recomendados)".
3. **Nota do item** (`nota`), em negrito itálico cinzento (ex.: "** Layout já definido").

Um **título de secção** (`secao: true`) é uma faixa cinzenta (#e6e6e6) de 26,5 pt, com filetes amarelos (#e8c94f) em cima e em baixo e o título em maiúsculas. Não tem valores e não entra nos totais.

Alturas usadas na paginação:

Bloco	Altura ocupada	Espaço depois (`folga`)
Item sem linhas por baixo	39,4 + 12 = 51,4	0
Item com L linhas	39,4 + 13,9 + 12 × (L − 1) + 18,6	18,6
Secção	26,5 + 12 = 38,5	12

A `folga` é o espaço depois do bloco. Não conta para decidir se o bloco cabe na página.

Paginação: `paginar()`

`paginar(c, medir)` recebe a cotação (itens, `notaMaterial`, `desconto`, `notas`) e devolve uma lista de páginas. Cada página é uma lista de `ItemPos`, com a posição `y` de cada item.

Constantes:

Constante	Valor	Significado
`TOPO_ITENS`	219,4	Onde começa o primeiro item de cada página
`FIM_CONTINUA`	790	Até onde descem os itens numa página que continua
`fimUltima`	`topoBloco(n) − 12,8 − (43 se notaMaterial) − altura do texto final a toda a largura`	Até onde descem os itens na última página, para caber o bloco de totais

Passos:

1. `n` = número de linhas de totais: 2 (SUB-TOTAL e IVA), ou 3 com desconto.
2. Cada item vira um bloco com altura e folga (tabela acima).
3. `dispor(itens, fim)` enche as páginas por ordem, a partir de `TOPO_ITENS`. Quando o bloco seguinte não cabe até `fim` e a página já tem algum item, abre página nova. **Um título de secção nunca fica sozinho no fundo da página**: para caber, a secção precisa também da altura do item seguinte.
4. Primeiro dispõe tudo com `FIM_CONTINUA`.

5. Depois verifica a última página. Se o último item passa de `fimUltima`:
   - essa página passa a ser uma página de continuação cheia;
   - só o último item (com os títulos de secção logo antes dele) passa para uma página nova;
   - repete até caber, no máximo 50 voltas.
6. Se a página já só tem esse item e mesmo assim não cabe, junta uma última página só com os totais.

Resultado: as páginas de continuação vão até ao rodapé, e a última página leva, em regra, o último item junto com os totais. Esta regra foi definida a 17 de Outubro de 2026: as páginas seguintes devem continuar a cotação com coerência.

Uma série de páginas por opção: `paginasCotacao()`

`paginasCotacao(cotacao, medir, urgenciaPct)` devolve `{ c, itens, ultima }[]`, a lista de todas as páginas a imprimir.

1. Aplica `comUrgencia` aos itens. Assim as linhas "Taxa de urgência" existem no documento.
2. Junta os números de opção dos itens que não são secção.
3. Com **mais de uma opção**, cria um grupo por opção: os itens sem opção (comuns a todas) mais os itens dessa opção. Com uma opção ou nenhuma, há um só grupo.
4. Pagina cada grupo com `paginar`. Cada grupo termina na sua última página, com o seu SUB-TOTAL / IVA / TOTAL.

Foi uma decisão do Momad (17 de Outubro de 2026): não quis as opções como linhas no bloco de totais. Quer cada opção nas suas páginas, como nas cotações dele.

A numeração "página p/t" e o aviso "continua na página n →" contam todas as páginas, de todas as opções seguidas.

Bloco de totais: `blocoTotais()`

`blocoTotais(c, taxaIva, urgenciaPct)` chama `totaisCotacao` e devolve:

- `linhas`: SUB-TOTAL (`bruto`), "DESCONTO x%" a vermelho com o valor precedido de "-" (só quando há desconto), e "IVA x%";
- `barra`: TOTAL.

A urgência não é uma linha de totais. É uma linha própria da tabela, logo a seguir ao item urgente.

Geometria (`BLOCO`, `CAIXAS`):

- As linhas de totais sobem a partir da última, em y = 740,46, de 32,72 em 32,72 pt. Cada linha tem 26,5 pt de altura e duas caixas: rótulo (359,98–454,56) e valor (459,79–564,51).
- Mais linhas (o desconto) empurram o bloco para cima. A barra do TOTAL (texto em y 799,31) nunca se mexe.
- O bloco abre com uma linha amarela em `H = topoBloco(n) = 740,46 − 32,72 × (n − 1) − 39,92`, que fica em 667,82 sem desconto e em 635,10 com desconto. Com o fundo original, esta linha já vem na imagem. Com o fundo livre, a aplicação desenha-a.
- A última linha (IVA) usa Medium e as outras Book. O valor do desconto sai a vermelho.

Texto final, notas e garantias (última página)

A função `termos()` arruma o texto final (`Cotacao.notas`):

Caso	Onde sai	Regras
É o texto padrão (comparado com `normal()`)	Coluna à esquerda dos totais, a partir de H + 47,24	Quebras e larguras fixas do original (`NOTA_PADRAO_LINHAS`, `NOTA_LARGURAS`). Linhas 1 e 3 justificadas, as outras forçadas à largura medida
Outro texto que cabe na coluna	Mesma coluna, largura 320,42	Cabem 5 linhas sem desconto e 8 com desconto. Todas justificadas excepto a última
Outro texto que não cabe	A toda a largura (534,51), por cima do bloco de totais	Altura = linhas × 12 + 12. Os itens da última página acabam mais acima. Todas as linhas justificadas excepto a última

A justificação (`justificada`) calcula o `wordSpacing` que leva a linha à largura pedida: (largura alvo − largura medida) ÷ número de espaços.

Outros textos da última página:

- **Nota do material** (`Cotacao.notaMaterial`). São três linhas por cima dos totais:

  1. "Todos os Materiais serão impressos em vynil com aplicação do respectivo protector material 100% dedicado a wrap.";
  2. "TAKSIM® STX570 + STX571 (60 Microns)" a vermelho sublinhado, seguido de "- Não Certificado";
  3. a garantia "até 2 anos".

  É o modelo antigo, para cotações de wrap cujos itens não têm materiais próprios. Em `src/pages/Cotacoes.tsx`, a caixa "Nota do material no rodapé" só aparece quando nenhum item tem materiais e algum título ou descrição fala de "taksim" ou "wrap". Nesses casos fica ligada por defeito. Se algum item tiver materiais, a nota fica sempre desligada.

- **Nota das garantias**: duas linhas logo abaixo da linha amarela ("** As garantias sobre a duração e qualidade do material serão dadas com base nos certificados..."). Só aparece quando a cotação é de material (`comMaterial`), ou seja, quando a nota do material está ligada, quando algum item tem materiais, ou quando o título, a linha 2 ou a descrição de algum item fala de vinil/vynil, wrap, branding, laminação, PPF, Taksim, Mactac, Avery ou Hexis.

Páginas de continuação

Uma página que não é a última:

- usa o fundo livre, sem bloco de totais;
- desenha uma linha amarela em y 812,77;
- escreve "continua na página n →" alinhado à direita (x 564,2), em y 804.

Preview ao vivo

`PreviewCotacao({ c, cliente, empresa })` em `src/pages/cotacaoFormulario.tsx`:

- espera por `carregarFontes()` (até lá não mostra nada) e carrega o logotipo do cliente;
- chama `paginasCotacao(c, medirCanvas, empresa.urgenciaPct)` e desenha cada página com `PaginaCotacao` dentro de `.a4-prev` (o SVG ocupa a largura disponível).

Em `src/pages/Cotacoes.tsx` o Preview aparece em dois sítios:

- na coluna da direita ("Como sai a cotação"), escondida no telemóvel;
- na janela do botão **Preview** da barra de cima, com os botões Fechar e PDF.

O Preview mostra a `vista`, que é a cotação a ser editada (mesmo sem Guardar) **mais** o rascunho do gerador de wrap/branding antes de "Juntar à cotação". Na vista, as linhas do rascunho ficam no fim, depois da Deslocação. Ao juntar, passam para antes das linhas extra (função `ordenar`). Os totais do formulário também incluem o rascunho.

Impressão: `imprimirCotacao()`

`imprimirCotacao(c, d)` em `src/print.tsx`:

1. Carrega as fontes (`carregarFontes`) e o logotipo do cliente.
2. Descodifica antes as imagens (`cotacao-fundo.jpg`, `cotacao-fundo-livre.jpg`, `cotacao-barra-total.jpg`, `logo.svg` e o logotipo do cliente). As `<image>` do SVG carregam por conta própria e a função de impressão só espera pelas `<img>`. Sem este passo, a impressão podia sair sem a arte.
3. Calcula as páginas com `paginasCotacao` e desenha cada uma num `div.pr-svg` (210 × 297 mm).
4. A função interna `imprimir()`:
   - desenha tudo em `#print-root` (definido em `index.html`);
   - espera pelas `<img>`;
   - troca `document.title` pelo nome do ficheiro, que o browser propõe ao guardar em PDF;
   - chama `window.print()` 250 ms depois e limpa tudo no evento `afterprint`.
5. Nome do ficheiro: `Puzzle_Stickers_Cotação_{cliente}_{linha 2 ou linha 1 do primeiro item}`, com espaços, "/" e ":" trocados por "_". O número COT- não entra no nome.

O CSS de impressão (`src/styles.css`, `@media print`):

- define `@page { size: A4; margin: 0 }`;
- esconde a aplicação (`#root`, avisos e janelas) e mostra só `#print-root`;
- força `print-color-adjust: exact`.

Notas:

- Os botões **PDF** (barra de cima e janela de Preview) gravam primeiro a cotação (`salvar({ notaMaterial })`) e imprimem o registo gravado. Um rascunho do gerador que ainda não foi juntado **não** sai no PDF.
- A cotação ainda não passa pela janela "PDF pronto" da requisição. O ficheiro PDF é criado pelo "Guardar em PDF" do sistema. Limitação conhecida: a captura com html2canvas não aplica as fontes web dentro de SVG, por isso a cotação continua no diálogo de impressão.

Conteúdos fixos: `src/materiaisCotacao.ts`

`MATERIAIS_COTACAO`: materiais das cotações do Momad (17 de Outubro de 2026)

Material (como sai na cotação)	Certificado	Garantia	`marca` (título da opção)
TAKSIM® STX570 + STX571 (60 Microns)	Não	até 2 anos	Taksim®
Mactac® JT10700 WG BFG XR + LF10700 UV Series (80 Microns)	Sim	até 5 à 7 anos	Mactac®
Avery Dennison® MPI 1105 SuperCast Series + DOL1460Z (82 Microns)	Sim	até 7 anos	Avery Dennison®
Taksim® Economy White Vinyl + Premium Lamination (80 Microns)	Não	acima de 12 meses	Vynil Comum

Material (como sai na cotação)	Certificado	Garantia	`marca` (título da opção)
TAKSIM® Premium Bubble Free + Premium Lamination (80 Microns)	Não	acima de 3 anos	Premium
Vinyl Holográfico Genérico (50 Microns)	Não	—	—
Avery Dennison® Supreme™ PPF Xtreme Gloss (165µm)	Sim	—	—
HEXIS® BODYFENCE X	Sim	—	—
HEXIS® HEXLIGHTS2 (70µm)	Sim	—	—
HEXIS® BODYFENCE WINDSHIELD (70µm)	Sim	—	—

Só os cinco primeiros têm `marca`, por isso só esses aparecem como opções no gerador de wrap/branding.

Outros conteúdos

Constante	Conteúdo
`NOTAS_ITEM`	"** Layout ainda por definir", "** Layout já definido", "** Layout já definido pelo cliente", "** Layout já definido pelo Cliente (Inclui melhoria, ou criação de novo Layout)"
`TEXTOS_FINAIS`	Cinco textos finais: Padrão, Pedir layouts, Criar layouts, Agendar viatura, Projecto. O "Projecto" é longo e, por isso, sai a toda a largura
`IMPRESSO.tipos`	Full Wrap ("Cobertura de 100% dos painéis"; Laterais, Capom, Bumper Frontal, Bumper Traseiro, Tejadilho, Bagageira, Stickers Complementares) · Half Wrap ("Cobertura de toda a lateral"; Laterais, Stickers Complementares) · Branding ("Cobertura até 30% dos painéis"; Laterais, Logotipos)
`IMPRESSO.coberturas`	100% dos painéis, toda a lateral, e "até" 30, 40, 50, 55, 60, 75, 85 e 100% dos painéis
`IMPRESSO.partes`	16 partes: Frente, Traseira, Laterais, Portas, Capom, Bumper Frontal, Bumper Traseiro, Tejadilho, Bagageira, Guarda-lamas, Retrovisores, Pilares, Spoiler, Logotipos, Logotipo no "Chapéu", Stickers Complementares
`IMPRESSO.estilos`	Artístico, Desportivo, Corporativo
`IMPRESSO.impressao`	"Impresso em vynil com aplicação do respectivo protector material 100% dedicado a wrap."
`DESLOCACAO`	Até 20 km de ida (até 40 km no total): 1.500 MT · 21–35 km (42–70 km): 2.500 MT · 36–50 km (72–100 km): 3.500 MT

`frasePartes(sel)` escreve a frase "Inclui …" pela ordem do Momad:

- primeiro as partes na ordem de `IMPRESSO.partes`, depois as partes escritas à mão;
- "Stickers Complementares" vai sempre para o fim;
- os dois bumpers juntam-se em "Bumper Frontal e Traseiro";
- liga as partes com vírgulas e põe "e" antes da última. Sem partes, devolve "".

Exemplo com as partes do Full Wrap: "Inclui Laterais, Capom, Bumper Frontal e Traseiro, Tejadilho, Bagageira e Stickers Complementares".

Formulário: `src/pages/cotacaoFormulario.tsx`

`EditorCotacao` segue a organização definida a 18 de Outubro de 2026: cada linha da cotação é um serviço ou produto.

Secção	O que faz
Cliente	Escolher ou criar cliente ("+ Novo cliente..."). Cidade e país gravam-se no cliente ao sair do campo (`separarMorada` / `juntarMorada` em `src/format.ts` ; país por defeito Moçambique). Carregar o logotipo do cliente. Data e validade (dias)
Linhas	Cada linha fechada mostra um resumo. Aberta, mostra `ItemEditor` : opção (Nenhuma / 1 / 2...), linha 1, linha 2 (a negrito), detalhes (uma ideia por linha), materiais, nota final, quantidade, valor unitário sem IVA
"Adicionar linha"	Wrap / Branding (gerador `NovoImpresso`), Criação de imagem (`CatalogoCriacao`), Outro serviço ou produto (linha em branco), Título de secção
Deslocação, urgência e desconto	Deslocação por botões ("Sem deslocação", as três bandas de `DESLOCACAO` , ou "Outro valor" com km e valor). Urgência ligada linha a linha. Desconto em %. IVA fixo, "sempre incluído"
Texto final	Botões com `TEXTOS_FINAIS` e área de texto livre
Totais	Com várias opções, uma linha por opção (sub-total + IVA, total). Sem opções: Sub-total, Desconto, IVA, TOTAL

Regras do formulário:

- A deslocação é uma linha `{ qt: 1, titulo: 'Deslocação', subtitulo: banda ou km, extra: true }` . As linhas `extra` ficam sempre no fim da cotação (função `ordenar`).
- A urgência marca `ItemCotacao.urgente` . O botão mostra o acréscimo: `r2(precoUnit × urgenciaPct / 100)` .
- **Gerador Wrap / Branding (`NovoImpresso`)**:
  - escolhe-se o serviço (Branding passa o estilo para Corporativo), os estilos, a viatura ou objecto ("É uma viatura") com a quantidade, a cobertura, as partes e o layout;
  - marcam-se os materiais com `marca` (os três primeiros vêm marcados) e escreve-se um preço para cada um;
  - cria **um item por material marcado**. O título é a marca e a linha 2 é "{serviço} {viatura}". A descrição tem a primeira linha automática ("{serviço} {estilos} em Viatura {viatura}", editável), "** {cobertura}", a frase das partes e o texto de impressão. A nota é o layout escolhido;
  - com mais de um material, cada item recebe a sua opção (1, 2, 3...). O botão diz "Juntar à cotação (N opções)".
- **Criação de imagem**: lista os serviços activos do catálogo (`Servico`) por categoria. Ao escolher um, cria uma linha com o nome, os "inclui" numa lista com "·" e o preço do catálogo.
- O seletor de materiais usa sempre `MATERIAIS_COTACAO` , porque a página `Cotacoes.tsx` não passa outra lista. Um material escrito à mão ("Outro material (escrever)...") entra só nessa linha. A caixa "Guardar na lista de materiais" ainda não grava nada.

Requisição de prints

Ficheiros

Ficheiro	O que faz
`src/requisicaoPdf.tsx`	Preçário da Académica 2026, nomes oficiais, preço por linha, totais (`totaisLinhas`), a página (`PaginaRequisicao`) e o CSS (`CSS_REQUISICAO`)
`src/pdfPartilha.tsx`	`renderizarPdf` (HTML → PDF no aparelho), janela "PDF pronto" (`PdfHost`), `emitirRequisicao`
`src/print.tsx`	`imprimirLista` , que chama `emitirRequisicao`

O PDF abre a partir de vários sítios:

- os botões "PDF" e "PDF da lista" em `src/pages/RequisicaoForm.tsx` , que gravam a requisição antes de gerar o PDF;
- os ícones nas listas de `Requisicoes.tsx` , `Fornecedores.tsx` e `Trabalho.tsx` .

Só existe para requisições do tipo lista. As encomendas (`tipo: 'encomenda'`) não têm PDF.

Preçário e materiais

`PRECARIO_ACADEMICA_2026` tem os preços por m², sem IVA, com os nomes do preçário (16 de Outubro de 2026):

Nome	Tipo	Preço/m²
Impressão com material da Puzzle Stickers	impressão	800
Vinil comum	impressão	950
Vinil comum laminado	impressão	1.400
Vinil de wrap	impressão	1.150
Vinil branco premium	impressão	1.050
Vinil transparente premium	impressão	950
ContraVision	impressão	1.150
Banner	impressão	950
Laminação de wrap	laminação	850
Laminação transparente premium	laminação	450

Esta lista serviu para carregar o preçário no registo do fornecedor, através de um script de administração que não está no repositório. Na aplicação, o formulário lê os preços de `Fornecedor.materiais` (dados na nuvem), não desta constante.

Funções de apoio:

Função	Recebe	Devolve / regra
`nomeOficial(nome)`	Nome antigo	"Taksim…", "vinil de wrap" ou "vynil de wrap" → "Vinil de wrap"; laminação + transparente → "Laminação transparente premium"; laminação + premium/wrap → "Laminação de wrap"; qualquer outro nome fica igual

Função	Recebe	Devolve / regra
`linhasComMaterial(r)`	Requisição	Linhas com material próprio (`impressao` ou `laminacao` definidos). As requisições anteriores a 16 de Outubro de 2026 tinham um material para a lista inteira: cada linha recebe esse material (nome oficial) e os preços da lista
`nomeMaterial(l)`	Linha	"Bundle Wrap" quando é vinil de wrap + laminação de wrap. Senão "impressão + laminação", ou "—"
`precoM2(l)`	Linha	`precoImpressao + precoLaminacao` (Bundle Wrap = 1.150 + 850 = 2.000)
`valorLinha(l)`	Linha	`r2(m2Linha(l) × precoM2(l))`

`totaisLinhas(linhas, taxaIva)`

- Usa só as linhas com quantidade, largura e comprimento maiores que zero (`usadas`). Só estas saem no PDF.
- `porMaterial` : m², valor e preço agrupados por `nomeMaterial` . É calculado mas não aparece no PDF, porque o Momad não quis o resumo por material.
- `m2` : soma dos m². Também não aparece no PDF, porque os materiais são diferentes.
- `subtotal = r2(soma de valorLinha)` , `iva = r2(subtotal × taxa / 100)` , `total = r2(subtotal + iva)` .

A página: `PaginaRequisicao`

Recebe `numero` , `data` , `nomeTrabalho` , `fornecedor` , `empresa` , `linhas` e `notas` . É uma página HTML com 210 mm de largura e pelo menos 297 mm de altura.

Zona	Conteúdo
Topo	Logotipo `public/img/logo-doc.svg` (62 mm) · "REQUISIÇÃO DE PRINTS" · número numa etiqueta amarela (#ffc406) · data (`dataDoc`) · barra amarela por baixo
Caixa do fornecedor	Só o logotipo do fornecedor: `public/img/academica.png` quando o nome contém "Académica" (ou "Academica"). Para outro fornecedor, o nome em negrito
Caixa do trabalho	"Trabalho", `nomeTrabalho` com "_" trocados por espaços, e "N prints"
Tabela	Nº, Ficheiro, Material, Qt., L × C (m), m², Preço m², Valor. Larguras fixas: 7 / 64 / 33 / 8 / 21 / 10 / 20 / 21 mm
Ficheiro	10 pt, negrito, preto, com barra amarela à esquerda. Depois de cada "_" há um espaço invisível (U+200B) para o nome poder quebrar
Material	"Bundle Wrap" com a linha "Vinil de wrap + laminação de wrap" por baixo. Nos outros casos, a impressão (ou a laminação, se for só laminação) e, quando há as duas, a linha "+ {laminação}" por baixo
Fim	Notas à esquerda. À direita: Sub-total, "IVA x%" e TOTAL a amarelo
Rodapé	"Preços por m², sem IVA, do preçário 2026 da {fornecedor}." e o nome, os telefones e o email da empresa

Decisões do Momad (16 de Outubro de 2026):

- título "REQUISIÇÃO DE PRINTS";
- caixa do fornecedor só com o logotipo;
- sem resumo por material e sem total de m²;

- nome do ficheiro em destaque;
- célula do material em duas linhas.

O logotipo usa `logo-doc.svg`, que tem largura, altura e cor sólida definidas, porque o html2canvas corta um SVG sem largura e altura.

Do HTML ao PDF: `renderizarPdf(node, css)`

Devolve um `Blob` PDF. Funciona offline e nada sai do aparelho. As bibliotecas são ficheiros `.js` da aplicação, guardados pela PWA.

1. Importa `jspdf` e `html2canvas` só quando é preciso (import dinâmico).
2. Junta o CSS à página e desenha o documento numa `div` fora do ecrã (left −12000 px, 210 mm de largura).
3. Carrega o Gotham nos pesos 300, 400, 500 e 700 e espera por `document.fonts.ready`.
4. Espera pelas imagens com `decode()`, no máximo 2,5 s cada, porque com a aplicação em segundo plano o `decode()` nunca termina.
5. Captura com `html2canvas` (escala 2,5, fundo branco).
6. Corta a imagem em fatias com a proporção A4 (altura = largura × 297/210). Cada fatia vira uma página JPEG (qualidade 0,92) de 210 × 297 mm. Ignora uma sobra de menos de 4 px no fim.
7. Limpa tudo no `finally`.

Janela "PDF pronto"

`PdfHost` é montado uma vez em `src/App.tsx`. Recebe um `PdfPronto` `{ blob, nome, texto, telefone?, destino? }` através da variável interna `mostrar` de `src/pdfPartilha.tsx`. Hoje só `emitirRequisicao` a usa. Um documento novo que queira esta janela deve ser emitido por uma função nesse ficheiro. A janela existe porque os telemóveis só abrem a folha de partilha a partir de um toque directo.

Botão	Quando aparece	O que faz
Partilhar (WhatsApp, email…)	O aparelho consegue partilhar ficheiros (`navigator.canShare({ files })`)	Abre a folha de partilha com o ficheiro, o título e o texto
Descarregar e abrir o WhatsApp da {destino}	O aparelho não partilha ficheiros e o fornecedor tem telefone	Descarrega o PDF e abre o WhatsApp na conversa, com a mensagem escrita: `wa.me` no telemóvel, `whatsapp://send` no computador. O PDF tem de ser anexado à mão a partir das Transferências
Descarregar	Sempre	Grava o ficheiro
Ver	Sempre	Abre o PDF num separador

A janela mostra também o nome do ficheiro e o tamanho em KB. O link do ficheiro é criado uma vez por janela e libertado ao fechar.

`emitirRequisicao(r, d)`

- Desenha `PaginaRequisicao` com `linhasComMaterial(r)` e `CSS_REQUISICAO`.
- Nome do ficheiro: `{número com "/" trocado por "-"}_{nome do trabalho com espaços, "/", "\" e ":" trocados por "_"}.pdf`, por exemplo `REQ-003-OUT-26_Nome_Do_Trabalho.pdf`. Sem nome do trabalho, usa "Requisicao".
- Texto da mensagem: "Bom dia! Segue a requisição de prints {número} — {nome do trabalho, com "_" trocados por espaços} ({n} prints). Aguardo a pró-forma. Obrigado — Puzzle Stickers".
- Telefone: o do fornecedor. Destino: a primeira palavra do nome do fornecedor.

Diferenças a conhecer

- **IVA**: o PDF calcula sempre o IVA (`totaisLinhas`). `totaisRequisicao` , usado no resto da aplicação, só o soma quando `comIva` está ligado.
- **Arredondamento**: o PDF arredonda linha a linha e soma. `totaisRequisicao` soma sem arredondar e só arredonda no fim. Com um só preço, multiplica os m² totais pelo preço, como a folha Excel do Momad. Pode haver diferenças de cêntimos.
- **Quebra de página**: o corte em páginas é feito em píxeis. Uma requisição muito longa pode partir uma linha da tabela entre duas páginas.
- **Telefone**: para o WhatsApp, esta janela só tira os dígitos, por isso o número do fornecedor tem de estar gravado com o indicativo (258…). A função `waLink` de `src/format.ts` acrescenta o 258 aos números de 9 dígitos, mas esta janela não a usa.
- **Contagem de prints**: a mensagem conta as linhas com quantidade maior que zero. O PDF conta as linhas com quantidade e medidas.

Relação do IVA (PDF e CSV)

Em `src/print.tsx` , com os números de `ivaDoMes` (ver Cálculos). Os botões "PDF" e "Excel" estão em `src/pages/Iva.tsx` .

- `imprimirIva(mes, r, d)` faz uma página A4 em HTML (fonte Montserrat) com:
  - logotipo, razão social e NUIT;
  - "Relação do IVA", o mês e "Regime normal · taxa {taxa da empresa}%";
  - três caixas: IVA liquidado, IVA dedutível, e IVA a pagar ou crédito a reportar;
  - duas tabelas, "Faturas emitidas (vendas)" e "Faturas recebidas (compras)", com as colunas Data, Tipo, Número, Entidade, NUIT, Base, IVA e Total;
  - um rodapé a dizer que é um resumo de apoio e não substitui a declaração oficial.

  Sai pelo diálogo de impressão, com o título "Relacao IVA {mes} Puzzle Stickers". Nas notas de crédito, cada linha mostra o IVA negativo e a base sem sinal. A linha de totais de cada tabela subtrai base e IVA.
- `exportarIvaExcel(mes, r, d)` gera um CSV em UTF-8 com BOM, separador ";", fim de linha CRLF e números no formato 1.234,56. No topo leva o título, a razão social com o NUIT e os três valores. Depois vêm as colunas Direção, Data, Tipo, Número, Entidade, NUIT, Base, IVA e Total. Nas notas de crédito, base e IVA saem negativos e o Total sai sem sinal. O ficheiro chama-se `Relacao_IVA_{mes}_Puzzle_Stickers.csv` .

O CSS deste ficheiro (`CSS`) ainda tem estilos de versões antigas da lista e da cotação (classes `li-*` e `ct-*`) que já não são usados.

Catálogos

Os catálogos estão em `src/pages/Catalogos.tsx` , no separador **Cotações → Catálogos**. O separador fica escondido para o papel de trabalhador (`naoTrab` em `src/App.tsx`). O Momad pediu-os em 18 de Outubro de 2026, para os enviar em poucos toques.

id	Nome	Ano	Páginas	Tamanho	Notas
`criacao-de-imagem-2026`	Criação de Imagem	2026	21	4,7 MB	Capa deitada
`sticker-kits-2025`	Sticker Kits	2025	10	5,7 MB	

id	Nome	Ano	Páginas	Tamanho	Notas
`producao-2020`	Produção	2020	10	3,1 MB	Marcado "antigo"

Os ficheiros estão em `public/catalogos/{id}.pdf` e as capas em `public/catalogos/{id}.jpg`. Vão com a aplicação, por isso o link é o mesmo em todos os aparelhos.

Acção	O que faz
Partilhar PDF	Descarrega o PDF e abre a folha de partilha. Se o aparelho não partilhar ficheiros, descarrega com o nome "Catálogo {nome} · Puzzle Stickers {ano}.pdf"
Enviar link	Abre o WhatsApp sem contacto escolhido, com o texto "Olá! Segue o nosso catálogo de {nome}: {endereço}/catalogos/{id}.pdf"
Ver	Abre o PDF num separador (a capa também o abre)
Copiar link	Copia o link. Se a cópia falhar, mostra o link num aviso

Configuração da PWA (`vite.config.ts`):

- `navigateFallbackDenylist: [/^\/catalogos\//]` faz os links abrirem o PDF e não a aplicação.
- Os padrões de pré-cache do Workbox não incluem `catalogos/`. Os PDFs e as capas precisam de rede.

Para juntar um catálogo:

1. Pôr `{id}.pdf` e `{id}.jpg` em `public/catalogos/`.
2. Acrescentar uma entrada a `CATALOGOS`.

Cálculos: `src/calc.ts`

Arredondamento

`r2(n) = Math.round((n + Number.EPSILON) × 100) / 100`. As quantias passam por aqui. Os m² não são arredondados.

Valores por defeito da empresa (`EMPRESA` em `src/seed.ts`): IVA 16%, sinal 50%, urgência 45%, validade da cotação 30 dias.

Cotação

`comUrgencia(itens, pct)`

- **Recebe**: os itens da cotação e a percentagem de urgência.
- **Devolve**: os itens, com uma linha nova logo a seguir a cada item urgente (`urgente`, desde que não seja secção). A linha nova tem:
  - `titulo` : "Taxa de urgência (45%)";
  - `subtitulo` : a linha 2 do item, ou a linha 1;
  - `qt: 1` ;
  - `precoUnit = r2(precoUnit do item × pct / 100)` ;
  - a mesma `opcao` do item;
  - `extra: true` , `gerada: true` .
- **Regras**:

- cobra-se 45% de **uma** unidade, uma só vez: num item com 5 carros não se cobra 5 vezes (Momad, 18 de Outubro de 2026);
- sem percentagem, ou se já houver uma linha `gerada`, devolve os itens sem mudar nada. Por isso pode ser chamada duas vezes sem duplicar (`paginasCotacao` e depois `totaisCotacao`);
- as linhas geradas não se gravam na cotação, porque são criadas sempre que é preciso. Na conversão em trabalhos, a linha de urgência passa a ser uma parte do trabalho do seu item.

`totaisCotacao(c, taxaIva, urgenciaPct = 0)`

- **Recebe**: a cotação (itens e `desconto` em %), a taxa de IVA e a urgência.
- **Devolve**: `{ bruto, desconto, sub, iva, total, opcoes }`. `opcoes` é uma lista de `{ opcao, bruto, desconto, sub, iva, total }`. Quando há opções, os valores principais são os da opção mais barata e vêm também com o seu `opcao`.
- **Regras**:
  1. Aplica `comUrgencia` e ignora as secções.
  2. `bruto = r2(Σ qt × precoUnit)`, `desconto = r2(bruto × desconto% / 100)`, `sub = r2(bruto − desconto)`, `iva = r2(sub × taxa / 100)`, `total = r2(sub + iva)`.
  3. O desconto sai **antes** do IVA e aplica-se a todas as linhas, incluindo a urgência e a deslocação.
  4. Toda a cotação leva IVA (Momad, 17 de Outubro de 2026).
  5. Com opções, cada opção soma os seus itens mais os itens sem opção (os comuns, deslocação incluída).
  6. O valor principal (`total` e os outros) é o da cotação inteira ou, quando há opções, o da **opção mais barata**. A lista de cotações mostra "desde X" até o cliente escolher a opção (`opcaoAceite`).

Na conversão em trabalhos (`src/pages/Cotacoes.tsx`), o trabalho fica com `comIva: true` e o desconto já aplicado aos preços unitários das partes (`precoNoTrabalho`). Assim o trabalho soma o mesmo que a cotação.

Requisição

`m2Linha(l)`

Devolve `qt × largura × comprimento`. Os campos em falta contam como 0.

`totaisRequisicao(r, taxaIva)`

- **Recebe**: a requisição (linhas, preços da lista, `comIva` e, opcionalmente, `tipo` e `itens`) e a taxa de IVA.
- **Devolve**: `{ m2, precoM2, subtotal, iva, total }`.
- **Regras**:
  - **Encomenda** (`tipo: 'encomenda'`): `subtotal = r2(Σ qt × precoUnit)`. `m2` e `precoM2` são 0.
  - **Lista**:
    - `m2` é a soma dos m², sem arredondar.
    - Preço de cada linha: se a linha tem material próprio (`impressao` ou `laminacao` definidos, mesmo vazios), usa `precoImpressao + precoLaminacao`. Senão, usa o preço da lista (`precoImpressaoM2 + precoLaminacaoM2`).
    - Se todas as linhas com área têm o mesmo preço, `precoM2` é esse preço e `subtotal = r2(m2 × precoM2)`. É a conta da folha Excel do Momad.
    - Com preços diferentes, `precoM2 = 0` e `subtotal = r2(Σ m² da linha × preço da linha)`.
  - `iva = r2(subtotal × taxa / 100)` só com `comIva`. `total = r2(subtotal + iva)`.

Trabalho

`totaisTrabalho(t, taxaIva, urgenciaPct)`

- **Recebe**: o trabalho (`itens`, `comIva`, `urgente`), a taxa de IVA e a urgência.

- **Devolve**: `{ base, urgencia, subtotal, iva, total }`.
- **Regras**:
  - `base = r2(Σ qt × precoUnit)`;
  - `urgencia = r2(base × pct / 100)` quando o trabalho é urgente;
  - `subtotal = r2(base + urgencia)`. O IVA só entra com `comIva`.
  - A urgência do trabalho aplica-se a toda a base. Um trabalho criado a partir de uma cotação traz `urgente: false`, e a urgência entra como uma parte.

`resumoTrabalho(t, pagamentos, requisicoes, custos, cfg, hoje)`

- **Recebe**: o trabalho, todos os pagamentos, requisições e custos, `cfg = { taxaIva, urgenciaPct, sinalPct }` e o dia de hoje.
- **Devolve**: `ResumoTrabalho`. Em `src/state.tsx` é calculado para cada trabalho (`d.resumo`).

Campo	Regra
`subtotal`, `iva`, `total`	De `totaisTrabalho`
`pago`	Soma dos pagamentos do trabalho que não foram apagados
`emDivida`	`max(0, total − pago)`. É 0 se o trabalho foi cancelado
`sinalEsperado`	`r2(total × sinalPct / 100)`
`sinalPago`	`pago + 0,005 ≥ sinalEsperado`
`custoRequisicoes`	Soma dos `subtotal` (sem IVA) das requisições do trabalho que não foram apagadas. Inclui rascunhos
`m2`	Soma dos m² dessas requisições
`custoOutros`	Soma dos `Custo` do trabalho
`custoTotal`, `lucro`, `margem`	`lucro = subtotal − custoTotal`; `margem = lucro / subtotal` (0 sem subtotal)
`dias`	Dias desde `dataEntrada` até à data da fase "entregue", ou até hoje
`atrasado`	Não cancelado, não entregue, e `prazo` ou `prazoAtual` já passou

O custo mede-se sem IVA, porque o IVA das faturas dos fornecedores recupera-se na declaração mensal. A mão de obra não é custo do trabalho: os salários são despesas fixas e entram no resultado do mês.

Ciclo do fornecedor

`FASES_REQ` e `faseRequisicao(r, taxaIva = 16)`

Fase	Nome no ecrã	Cor
`rascunho`	Rascunho	#8a8f98
`enviada`	Lista enviada	#1e9bff
`proforma`	Pró-forma por pagar	#ff8a00
`credito`	A crédito	#e84393
`paga`	Paga	#7c5cff

Fase	Nome no ecrã	Cor
`faturada`	Faturada	#00b3a4
`levantada`	Levantada	#2bb673

A fase é calculada, nunca gravada. Ordem de decisão:

1. `temValor` é verdadeiro quando há pró-forma, pagamento a crédito ou pagamentos. `pago` é `temValor` com saldo ≤ 0,005.
2. Levantada (`levantadaEm` , ou o estado antigo `recebida`): "credito" se tem valor e ainda não está paga, senão "levantada".
3. Fatura recebida (`faturaDocId`) e paga: "faturada".
4. Paga: "paga".
5. `pagamento === 'credito'` : "credito".
6. Tem pró-forma: "proforma".
7. Enviada (`enviadaEm` ou estado `enviada`): "enviada".
8. Nos outros casos: "rascunho".

Valor, pago e saldo

Função	Regra
`valorRequisicao(r)`	Valor da pró-forma, se existir. Senão, o `total` de `totaisRequisicao`
`pagoRequisicao(r)`	Soma de `r.pagamentos` . Os registos antigos marcados `pago` sem lista de pagamentos contam como pagos por inteiro
`saldoRequisicao(r)`	`max(0, valor − pago)`
`estadoPagamento(r)`	"Pendente" (nada pago), "Parcial" (saldo > 0,005) ou "Pago"

`contasAPagar(reqs, taxaIva)`

Devolve o que se deve aos fornecedores: as requisições não apagadas, com pró-forma ou a crédito, com saldo > 0,005. Para cada uma devolve `{ r, valor (saldo), total, pago, desde, credito }` . `desde` é `creditoDesde` , ou a data da pró-forma, ou a data da requisição. A lista é ordenada por `desde` e depois pelo número da pró-forma (ou da requisição, quando não há pró-forma).

`mapaCredito(reqs, taxaIva)` : o "Mapa de controlo de crédito"

- **Entra**: requisições não apagadas com pró-forma, crédito, pagamentos ou marca `pago` .
- **Cada linha**: `{ r, valor, pago, saldo, estado, fase }` , ordenada pela data da requisição e pelo número da pró-forma.
- **Totais**: `total` , `pago` e `aberto` , arredondados.
- O teste confirma os totais de um mapa de exemplo: 100.133,60 / 12.000 / 88.133,60.

Caixa

`Movimento = { data, em?, tipo: 'entrada' | 'saida', origem, valor, texto, ref? }` . A origem é `cliente` , `compra` , `fixa` , `fornecedor` , `pessoal` ou `reposicao` .

`movimentosCaixa(pagamentos, despesas, requisicoes, taxaIva = 16)`

Fonte	Movimento
Cada `Pagamento` não apagado	Entrada `cliente`. O texto é a referência, ou "Pagamento de cliente"
Cada `Despesa` não apagada	Saída `pessoal` se `pessoal`, `fixa` se tem `fixaId`, `compra` nos outros casos. Valor com IVA
Despesa pessoal reposta (`pessoalEstado: 'reposto'` com `pessoalEm`)	Entrada `reposicao` no dia da reposição, com o texto "Reposição: …"
Cada pagamento de uma requisição	Saída `fornecedor`, com o texto "{número} · {trabalho}"
Requisição antiga marcada paga, sem lista de pagamentos, com `pagoEm`	Saída `fornecedor` com o valor da requisição

Ordena por data e depois pela hora de criação (`em`). Uma despesa pessoal autorizada (`pessoalEstado: 'autorizado'`) sai da caixa e não volta.

`saldoCaixa(movs, caixa?)`

- Sem `Empresa.caixa` (o saldo real que o Momad escreve uma vez em "Acertar caixa"), devolve `undefined`.
- Com caixa, devolve `caixa.valor` + entradas − saídas **depois** desse momento. Conta os dias seguintes e os movimentos do mesmo dia criados depois (`em > caixa.em`).
- Os movimentos do mesmo dia sem hora (pagamentos a fornecedores) consideram-se já incluídos no saldo.

`resumoCaixaMes(movs, mes)`

Devolve `{ entrou, reposicoes, compras, fixas, fornecedores, pessoal, saiu, saldo, movimentos }` do mês (AAAA-MM):

- `entrou` = todas as entradas, reposições incluídas;
- `saiu = compras + fixas + fornecedores + pessoal`;
- `saldo = entrou − saiu`.

É uma vista de caixa: cada valor conta no dia em que o dinheiro se mexe. O salário de Outubro pago a 2 de Novembro conta em Novembro.

Resultado do mês

`resultadoDoMes(mes, docs, requisicoes, custos, despesas, taxaIva)`

Devolve `{ vendas, custosTrabalhos, despesas, resultado, porCategoria }`, tudo sem IVA.

Parcela	Regra
`vendas`	Soma das bases dos documentos emitidos no mês que contam para IVA. As notas de crédito subtraem
`custosTrabalhos`	Subtotais das requisições do mês (pela data da requisição) que **não** estão em rascunho, mais os `Custo` do mês
`despesas`	`despesaSemIva` das despesas do mês que não são pessoais. O mês é `Despesa.mes` (despesa fixa) ou o mês da data
`resultado`	`vendas − custosTrabalhos − despesas`
`porCategoria`	Despesas sem IVA por categoria

`despesaSemIva(d) = r2(valor − iva)`.

Há duas vistas do custo do fornecedor:

- o **resultado** conta-o na data da requisição;
- a **caixa** conta-o quando é pago.

As despesas pessoais nunca são custo da empresa.

IVA

Função	Regra
`contaParaIva(tipo)`	Contam `fatura`, `fatura-recibo`, `venda-dinheiro` e `nota-credito`. Os recibos não contam
`ivaDoMes(docs, mes)`	Documentos não apagados do mês que contam para IVA, separados em emitidos e recebidos. As somas levam sinal (nota de crédito = −1). Devolve `emitidos`, `recebidos`, `baseVendas`, `baseCompras`, `liquidado`, `dedutivel`, `apagar` (se o saldo for > 0), `credito` (se for < 0) e `ignorados` (os documentos do mês que não contam)
`ivaDeBase(base, taxa)`	`r2(base × taxa / 100)`
`baseDeTotal(total, taxa)`	`r2(total / (1 + taxa / 100))`

Despesas fixas

Função	Recebe	Devolve / regra
`CATEGORIAS_DESPESA`	—	12 categorias, de "Salários" a "Outros"
`addMeses(mes, n)`	"AAAA-MM", n	O mês n meses depois (ou antes)
`janelaFixa(f, mes)`	Despesa fixa (`dia`, `abre`), mês	`{ abre, vencimento }`. Se `abre > dia`, o vencimento passa para o mês seguinte. É a regra do Momad: paga do dia 25 ao dia 5 do mês seguinte. O dia fica limitado ao último dia do mês (31 → 28 em Fevereiro)
`vencimentoFixa(f, mes)`	O mesmo	Só o vencimento
`fixasDoMes(fixas, despesas, mes, hoje)`	Despesas fixas e despesas	Fixas não apagadas que já começaram (`inicio ≤ mes`) e estão activas, ou que foram pagas nesse mês. Estado: `paga` (há despesa com `fixaId` e `mes`), `atrasada` (o vencimento já passou), `a-pagar` (janela aberta), `pendente`. Ordenadas por vencimento. Sem `abre`, passa de pendente a atrasada sem passar por "a pagar"

Numeração: `numeroDoMes(prefixo, existentes, data)`

- **Recebe**: "COT" ou "REQ", os números que já existem e a data.
- **Devolve**: o número seguinte do mês, ex.: `COT-003-OUT/26`, `REQ-001-NOV/26`.
- **Regras**:
  - o sufixo é o mês em três letras (JAN … DEZ) e o ano com dois dígitos;
  - procura o maior número com o mesmo prefixo e sufixo, soma 1 e escreve-o com 3 dígitos;

    - a contagem recomeça em cada mês;
    - os números noutro formato (ex.: `REQ-2026-013`) são ignorados.
- O formato foi definido pelo Momad em 16 de Outubro de 2026. Não serve para faturas.

Fases e agenda

Tabelas

- `ETAPAS`: as 7 fases, cada uma com nome, nome curto e cor: Pedido, Em negociação, Layout, Produção, Montagem, Entrega, Entregue.
- `TIPOS`: os 4 tipos de trabalho e as fases de cada um.

Tipo	Fases (`seq`)	Nomes próprios (`ROTULOS`)
`branding` (Branding / wrap)	pedido, negociacao, layout, producao, montagem, entrega, entregue	—
`criacao` (Criação de imagem)	pedido, negociacao, layout, entrega, entregue	layout = "Design"
`fornecimento`	pedido, negociacao, layout, producao, entrega, entregue	producao = "Encomenda"
`montagem` (só mão de obra)	pedido, negociacao, montagem, entrega, entregue	—

O tipo antigo `producao` é lido como `branding` (`tipoInfo`).

Funções

Função	Regra
`normalizarEtapa(e)`	Lê os nomes de fase anteriores a 16 de Outubro de 2026 como os novos: entrada → pedido; cotacao e aprovacao → negociacao; design → layout; marcacao e acabamento → montagem; encomenda e recebido → producao; pronto → entrega. É aplicada na leitura (`src/state.tsx`) e os registos antigos não são reescritos
`etapasDoTipo(tipo)`	A sequência de fases do tipo
`etapaInfo(e, tipo?)`	Nome e cor da fase, com o nome próprio do tipo
`proximaEtapa(t)` / `etapaAnterior(t)`	Fase seguinte ou anterior na sequência do tipo, ou `null`
`confirmado(t, pago)`	Verdadeiro se o cliente confirmou (`confirmadoEm`), se já pagou alguma coisa, ou se a fase já passou da negociação
`contaParaReceber(t, r)`	Entra em "a receber" só se não está cancelado, tem total, ainda deve (> 0,005) e está confirmado. Os trabalhos em negociação ficam de fora
`prazoAtual(t)`	Prazo da fase actual. Entregue: nenhum. Entrega: `prazo`. Montagem: `montagemAte`, `montagemDe` ou `prazos.montagem`. Outras fases: `prazos[etapa]`

Função	Regra
`itensAgenda(trabalhos, tarefas)`	Para cada trabalho não cancelado e não entregue, junta as datas das fases que faltam, a actual incluída: montagem (com intervalo), "Entrega prevista" e "{fase} — data limite" ("Responder ao pedido" na fase de pedido). Um pedido sem data aparece como "Pedido por tratar". Junta também todas as tarefas por fazer
`secaoAgenda(i, hoje)`	`semData` ; `atrasado` (o fim já passou); `hoje` (começou e ainda não acabou, por isso uma montagem em curso fica em Hoje); `semana` (próximos 7 dias); `depois`
`semDocumento(pagamentos, trabalhoId?)`	Pagamentos sem fatura-recibo ou recibo (`reciboId`), ordenados por data

Datas

Função	Regra
`diaISO` , `hojeISO`	Data local AAAA-MM-DD
`diasEntre(a, b)`	Dias de a até b
`addDias(iso, n)`	Soma n dias. Usa o meio-dia para evitar saltos de hora
`addDiasUteis(iso, n)`	Soma n dias de segunda a sexta. Não conta feriados. O catálogo fala em "dias úteis". Um trabalho convertido de uma cotação recebe um prazo de 10 dias úteis

Testes que fixam estes números

Correm com `npm test` (Vitest, ambiente `node`).

`src/calc.test.ts`

Grupo	O que fixa
Lista de impressão	Uma folha de exemplo: 10,54 m² × 2.000 = 21.080,00, mais IVA 3.372,80, dá 24.452,80. Com materiais misturados por linha: `precoM2` 0, subtotal 9.700, IVA 1.552, total 11.252
Trabalho	Uma cotação de exemplo: 110.000 + 17.600 = 127.600. Urgência de 45% antes do IVA: 10.000 → 14.500 → 16.820. `resumoTrabalho` : total 116.000, dívida 58.000, sinal pago, custo 20.000 + 5.000, lucro 75.000, atrasado, 9 dias
IVA mensal	Liquidado 17.440 (com a nota de crédito descontada), dedutível 3.372,80, a pagar 14.067,20, um recibo ignorado. Crédito de 500 quando as compras passam as vendas
Agenda	Dias úteis (sexta → segunda). Fases por tipo e o tipo antigo `producao` . `normalizarEtapa` . `contaParaReceber` . `itensAgenda` e `secaoAgenda` . `numeroDoMes` (recomeço mensal e formato antigo ignorado). Caixa: saldo de 100.000 a 16 Out → 129.840; resumo de Outubro e salário de Outubro pago em Novembro. `prazoAtual`
Despesas	Vencimento no dia 31 → 28 de Fevereiro. Estados paga / atrasada / pendente. Janela 25 → 5, incluindo Dezembro → Janeiro, com os estados pendente, a pagar, atrasada e paga. O salário de Outubro pago em Novembro conta no resultado de Outubro. `resultadoDoMes` = 100.000 − 12.500 − 29.000 = 58.500, sem contar rascunhos

Grupo	O que fixa
Ciclo do fornecedor	Totais de quatro pró-formas de exemplo (26.726,40; 15.683,20 com subtotal 13.520, valor de duas delas; 6.264) e de uma lista (15.340 / 17.794,40). Sequência rascunho → enviada → pró-forma → crédito → levantada. Encomenda: 2 × 12.500 + IVA = 29.000
Mapa de crédito	Totais de um mapa de exemplo: 100.133,60 / 12.000 / 88.133,60. Estados Parcial e Pendente. Fase crédito
Cotação	Urgência como linha própria + desconto: 146.500 / 14.650 / 131.850 / 21.096 / 152.946. Linha de urgência com qt 1 e 67.500 para 2 × 150.000. Desconto antes do IVA: 100.000 − 10% → 104.400. Opções com itens comuns: 47.560 / 52.200 / 59.160 (valor principal 47.560). Seis carros com duas opções: sub-totais 515.000 / 810.000

Testes de render

- `src/cotacaoPdf.render.test.tsx`:
  - pagina uma cotação de exemplo com `paginar` e confirma que cabe numa página, com as caixas dos itens em 219,4 / 303,5 / 411,1 / 518,2 (diferença menor que 1,2 pt);
  - escreve `test/out/cotacao.html`, para imprimir com o Chrome e comparar com o modelo da empresa;
  - para medir sem browser, usa `test/fixtures/gotham-advances.json`, com a largura de cada caractere do Gotham. Este ficheiro está no repositório.
- `src/requisicaoPdf.render.test.tsx`:
  - confirma `totaisLinhas` com materiais misturados: Bundle Wrap 15.840 (7,92 m² × 2.000), ContraVision 1.380, vinil transparente premium + laminação transparente premium 896, Banner 2.850, subtotal 20.966;
  - escreve `test/out/requisicao.html` para uma verificação visual.

Mudar `TOPO_ITENS`, `ITEM`, `BLOCO` ou a largura das descrições (`COL.totX1`, de que depende `LARGURA_DESC`) em `src/cotacaoPdf.tsx` altera as posições que o teste da cotação verifica.

Os testes que lêem PDFs e listas reais de fornecedores não estão neste repositório, porque usam documentos reais. Em `src/leitura.test.ts` fica o teste de leitura de números.

Pontos em aberto ligados a este capítulo

Planeados, à espera de decisão do Momad

Estão planeados mas **não implementados**.

- **Fatura da Académica sem pró-forma**:
  - campo `Requisicao.fatura` (número, data, valor com IVA) e uma fase nova, "Fatura por pagar", em `FASES_REQ` / `faseRequisicao`;
  - `valorRequisicao` passaria a preferir a fatura, depois a pró-forma, depois a lista;
  - `contasAPagar` e `mapaCredito` passariam a incluir as faturas;
  - os pagamentos ao fornecedor ganhariam um comprovativo (`ficheiroId`).
  - Falta decidir se o custo do trabalho passa a usar a base da fatura quando difere da lista.
- **Cotação negociada**: datas de envio (`enviadaEm`) e versões ligadas (`substituiId`, `substituidaPorId`, estado `substituida`). Falta decidir:
  - se a versão nova leva número novo ou o mesmo número com "v2";
  - se o PDF deve dizer "Substitui a cotação …";

    - se corrigir uma cotação já enviada só avisa ou fica bloqueado.

- **Requisição como a cotação**: faixa de estado, Preview A4 ao vivo, "Guardar" só quando há alterações e uma caixa "Enviei" na janela "PDF pronto" (pré-marcada, nunca automática) que grava a data de envio. Falta decidir se chega melhorar a janela actual ou se a requisição passa a ter página inteira.
- **Ficheiros na nuvem** (logotipos de clientes, faturas, comprovativos). Hoje ficam no aparelho onde foram carregados.

Pendente técnico

- **Cotação como ficheiro PDF** na janela "PDF pronto", como a requisição. É preciso outro caminho que não o html2canvas, porque este não aplica as fontes web dentro do SVG.

Notas para quem continua

- Não grave linhas com `gerada: true` na cotação. `comUrgencia` não acrescenta urgência quando já há linhas geradas.
- Carregue sempre as fontes antes de medir texto (`carregarFontes`). Medir com a fonte de recurso desarruma as quebras e a paginação.
- O resumo do trabalho conta as requisições em rascunho como custo. O resultado do mês não as conta.
- O Gotham é uma fonte comercial e não está neste repositório. Os ficheiros da empresa copiam-se para `public/fonts/` .